pattern	count_in_file
0-1 BFS (Deque)	1
0-1 BFS (Deque) or Dijkstra's Algorithm with a 3D State Space `(row, col, direction/momentum)`.	1
0-1 BFS / Dijkstra's Algorithm	1
0-1 BFS or Dijkstra	1
0-1 BFS or Dijkstra's Algorithm on a modified graph where every directed edge (u -> v) with weight 0 has a "ghost" edge (v -> u) with weight 1.	1
0-1 Trie (Binary Trie).	1
0/1 Knapsack DP (1D) Use a boolean array `dp` where `dp[i]` is true if sum `i` is possible.	1
0/1 Knapsack Dynamic Programming	1
0/1 Knapsack Dynamic Programming (Inverse Problem). Find subsets summing to strictly less than K, and subtract from total subsets.	1
1D Array Direct Indexing (1-based to 0-based mapping)	1
1D Array Extraction & Modular Rotation	1
1D Array tracking with consecutive free-block counting and slice assignment.	1
1D DP (Character Bucket State)	1
1D DP (Fibonacci) + Math Independence	1
1D DP + Bitmasking (State tracking per bit).	1
1D DP / Two Pointers If arr[i]-arr[i-1] == arr[i-1]-arr[i-2], current slices = prev slices + 1. Add to total sum.	1
1D Dynamic Programming	3
1D Dynamic Programming (2 States)	1
1D Dynamic Programming (Boolean State)	1
1D Dynamic Programming (Bottom-Up)	1
1D Dynamic Programming (Climbing Stairs Variant)	1
1D Dynamic Programming (Fibonacci-like)	1
1D Dynamic Programming (Iterative Tabulation).	1
1D Dynamic Programming (Knapsack Variation)	1
1D Dynamic Programming (Knapsack)	1
1D Dynamic Programming (Memoization)	1
1D Dynamic Programming (Right to Left).	1
1D Dynamic Programming (Rolling Array) calculated strictly from right-to-left.	1
1D Dynamic Programming (State Machine).	1
1D Dynamic Programming (Suffix/Prefix Array)	1
1D Dynamic Programming (Tabulation).	1
1D Dynamic Programming (Two States)	1
1D Dynamic Programming (Unbounded Knapsack).	1
1D Dynamic Programming + GCD Check	1
1D Dynamic Programming / Greedy	1
1D Dynamic Programming / Sliding Window	1
1D Dynamic Programming with 2D Palindrome Pre-processing.	1
1D Dynamic Programming with Frequency Map optimization.	1
1D Greedy / Sorting	1
1D Index Mapping (Modulo Arithmetic)	1
1D Projection (Interval Overlap).	1
1D Rolling DP (4 Stages)	1
26x26 Frequency Map Simulation.	1
2D Binary Indexed Tree (Fenwick Tree)	1
2D Bitset / Hash Set of Pairs	1
2D Diagonal Prefix Sums (Prefix Sum on two diagonal directions).	1
2D Difference Array (Prefix Sums).	1
2D DP (Knapsack Variant)	1
2D DP (Prefix Minimum)	1
2D DP (Subarray XOR Triangle + Range Max)	1
2D DP + 2D Prefix Sum (Suffix Sum)	1
2D DP with Binary Search OR "Moves to Floors" Inverse DP.	1
2D DP: Longest Common Subsequence (LCS)	1
2D Dynamic Programming	8
2D Dynamic Programming (Bottom-Up Tabulation).	1

2D Dynamic Programming (Bottom-Up)	2
2D Dynamic Programming (Bottom-Up).	2
2D Dynamic Programming (dp[i][j])	1
2D Dynamic Programming (HashMap/Array).	1
2D Dynamic Programming (Interval DP)	1
2D Dynamic Programming (Layered)	1
2D Dynamic Programming (LCS Variant)	1
2D Dynamic Programming (LCS).	1
2D Dynamic Programming (LPS on word1 + word2).	1
2D Dynamic Programming (Memoization).	1
2D Dynamic Programming (Palindrome Table).	1
2D Dynamic Programming (Path Counting)	1
2D Dynamic Programming (Source Idx vs Pattern Idx)	1
2D Dynamic Programming (Space Optimized)	1
2D Dynamic Programming (State Switching)	1
2D Dynamic Programming (State: Index, Last Value)	1
2D Dynamic Programming (Suffix-Sum style)	1
2D Dynamic Programming (Tracking mod pairs)	1
2D Dynamic Programming + Hash Set.	1
2D Dynamic Programming + Prefix Sums.	1
2D Dynamic Programming / Memoization.	1
2D Dynamic Programming with Memoization + Math Pruning (Probability limit).	1
2D Dynamic Programming with Prefix Sum Optimization	1
2D Dynamic Programming with Suffix Max Optimization (State: dp[num][diff]). (O(N * M))	1
2D Dynamic Programming.	1
2D Frequency Matrix (Counting)	1
2D Interval Dynamic Programming	1
2D Knapsack DP `dp[i][j]` is max strings using `i` zeros and `j` ones. Update backwards to save space.	1
2D Prefix Sum (DP) + Quickselect/Min-Heap.	1
2D Prefix Sum (In-Place)	1
2D Prefix Sum (Inclusion-Exclusion Principle)	1
2D Prefix Sum + 1D TreeSet Range Search	1
2D Prefix Sum + 2D Difference Array.	1
2D Prefix Sum + Binary Search (or Greedy Side Length)	1
2D Prefix Sum Array	1
2D Prefix Sum Matrix	1
2D Prefix Sums (Factors of 2 and 5)	1
2D Prefix Sums (Inclusion-Exclusion).	1
2D Prefix Sums (Left and Up counts)	1
2D Prefix Sums (Rows, Cols, and Diagonals).	1
2D Prefix Sums + Binary Search for Radius, treating the "spiral rings" as concentric square boundaries.	1
2D Prefix Sums + Hash Map / Binning.	1
2D Prefix Sums / Precomputation of "Smaller" and "Greater" counts.	1
2D Prefix/Suffix Max + Greedy	1
2D Sliding Window / Submatrix Hashing	1
2D-to-1D Reduction + Subarray Sum Logic.	1
3-Way Dijkstra's Algorithm	1
3D Dynamic Programming	9
3D Dynamic Programming (Grid + Modulo State)	1
3D Dynamic Programming (Index, Last Move, Score Diff)	1
3D Dynamic Programming (Knapsack Variant)	2
3D Dynamic Programming (Knapsack variant).	1
3D Dynamic Programming (Knapsack with Sign State).	1
3D Dynamic Programming (Memoization).	1
3D Dynamic Programming (Simultaneous Path)	1
3D Dynamic Programming (Spatial State).	1
3D Dynamic Programming (State Machine)	1

3D Dynamic Programming (State: i, j, kRemain). O(n^2 · k)	1
3D Dynamic Programming (Top-Down with Memoization).	1
3D Dynamic Programming (Top-Down)	1
3D Dynamic Programming / Grid DP (State: row, col, neutralizations_left). O(M · N · K)	1
3D Dynamic Programming / Memoization	2
3D Dynamic Programming.	1
5D Dynamic Programming + Sorting	1
8-Directional Raycasting from Move	1
`CASE` statement or `IF` function + `MOD()` + `LIKE`.	1
Accumulating Closure Scopes (Each timeout creates a new closure, potentially keeping old variables in memory longer).	1
Adjacency List + BFS (Graph Conversion)	1
Adjacency Matrix + Degree Precomputation.	1
Adjacency Set Intersection (Triangle Counting)	1
Advanced Combinatorics + Inclusion-Exclusion	1
Advanced Digit DP + Precomputed Combinatorics Modulo 10^9+7. Find valid popcounts up to Length L, then use nCr to place bits.	1
Advanced Digit DP with Modulo/State constraints	1
Aggregate Comparison (HAVING Clause)	1
Aggregate Math (`MAX` - `MIN`)	1
Aggregated Subquery / IFNULL	1
Aho-Corasick + Greedy Jump / Segment Tree	1
Aho-Corasick / Trie + 1D Dynamic Programming	1
Algebraic Factorization (N = kx + frac{k(k-1)}{2}).	1
Algebraic Simplification + Binary Search / Fenwick Tree. Re-write the property as a range: arr[j] must be in [arr[i]/2, 2*arr[i]] .	1
Amortized Linear Scan / Precomputation	1
Angular Sweep / Fixed Circle Centers	1
Argument Object Cloning (Visualizing the memory used to copy the arguments list for every call).	1
Arithmetic Computation	1
Arithmetic Digit Extraction	1
Arithmetic Digit Summation	1
Arithmetic Progression (AP) Formula	1
Arithmetic Series Segmentation (Modulo Math)	1
Arithmetic Snapping / Modulo Math	1
Array Grouping by Set Bits + Min/Max Tracking	1
Array Indexing (Direct Access)	1
Array Length Property (`this.length`)	1
Array Math (Grade School Multiplication)	1
Array modification tracking (snapshotting the entire array state at each simulated step).	1
Array of 26 Stacks / Deques mapping available indices. (O(N))	1
Array of Linked Lists (Chaining)	1
ASCII Bit Manipulation / Offset	1
AWK Column Accumulation.	1
Backtracking (Constrained DFS)	1
Backtracking (Counter Only)	1
Backtracking (DFS + In-Place Mark)	1
Backtracking (DFS + Lexicographic)	1
Backtracking (DFS for DAG)	1
Backtracking (DFS)	2
Backtracking (DFS) / Pick & Skip State Tracking	1
Backtracking (DFS) with Time Pruning	1
Backtracking (Half-String Construction)	1
Backtracking (In-Place Swap)	1
Backtracking (Include/Exclude)	1
Backtracking (Keep or Abbreviate)	1
Backtracking (Row-by-Row + Bitsets)	1
Backtracking (Warnsdorff's Rule optional)	1
Backtracking + Frequency Array.	1
Backtracking + Greedy (Place largest available number first).	1

Backtracking + Memoization (Top-Down DP)	1
Backtracking + Memoization (Top-Down DP).	1
Backtracking + Recursive State Validation	1
Backtracking + Sorting Groups.	1
Backtracking / DFS	1
Backtracking / Divide and Conquer.	1
Backtracking / Greedy (Small N)	1
Backtracking / Set	1
Backtracking DFS with Column-by-Column Pruning	1
Backtracking Iterate through the 10 LEDs, choosing to turn them on/off until `turnedOn` is met.	1
Backtracking on Net Balances Flatten everyone into their net balance. Try to settle one person's balance with any other person of the opposite sign.	1
Backtracking with Bi-directional Mapping	1
Backtracking with Coordinate Set Maintain a `visited` Set of `(row, col)`. Always try to clean, move, recurse, then return and face original direction.	1
Backtracking with DP/Memoization (pre-checking palindromes).	1
Backtracking with Early Pruning	1
Backtracking with Evaluation-on-the-fly	1
Backtracking with Fibonacci Pruning & Integer Overflow check.	1
Backtracking with Frequency State Management	1
Backtracking with Memoization (Bitmasking)	1
Backtracking with Memoization (DFS + Map).	1
Backtracking with Path Tracking	1
Backtracking with Pre-calculated Jump Table	1
Backtracking with Pruning	2
Backtracking with Pruning & Skyline State	1
Backtracking with Pruning + State Compression.	1
Backtracking with Pruning + Visited Array (or Bitmask).	1
Backtracking with Pruning / BFS for State Space	1
Backtracking with Pruning / Iterative Prefix Checking	1
Backtracking with Pruning / Network Flow.	1
Backtracking with Pruning.	1
Backtracking with State (Hamiltonian Path)	1
Backtracking with State Pruning (Bitmask)	1
Backtracking with Value Pruning.	1
Backtracking/Recursion or Bit Manipulation (Math: result is OR(all_elements) x 2^{n-1}).	1
Backward Pointer Scan	1
Backward Pointer Scan.	1
Backward Scan + Boolean Frequency Array	1
Backwards Traversal (Target to Start)	1
Balance Counter (Greedy)	1
Balanced Binary Search Tree (TreeSet) + Sliding Pointer	1
Balanced BST / TreeMap (Ordered Map)	1
Base 3 Representation (Ternary Logic).	1
Base-26 Conversion (with Offset Correction).	1
Base-9 Conversion.	1
Bellman-Ford / BFS (Level-based)	1
BFS (Breadth First Search) for Unweighted Shortest Path.	1
BFS (Breadth-First Search)	1
BFS (Breadth-First Search) + Visited Array (0-1000)	1
BFS (Digit Permutation Generation)	1
BFS (Level Order Traversal)	3
BFS (Level Order Traversal) but strictly Right-to-Left.	1
BFS (Level Order Traversal) tracking max per level.	1
BFS (Level Order Traversal) with an early exit on the first leaf node encountered.	1
BFS (Level Order Traversal).	1
BFS (Level-Order Expansion) or Bucket Sort.	1
BFS (Level-Order Traversal)	1
BFS (Level-Order Traversal).	1

BFS (Queue) Use a Queue. Process all nodes in the current queue (one level) before moving to the next.	1
BFS (Queue-based Level Order)	1
BFS (Reverse Level Order) + Hash Set	1
BFS (Shortest Path) Use a Queue. For each mutation, check if it's in the "bank". Use a visited set to avoid cycles.	1
BFS (Shortest Path) using a Queue and a Distance 2D Array / Dijkstra.	1
BFS (to find shortest distances) + DFS/Backtracking (to reconstruct paths).	1
BFS + Cycle Decomposition (Minimum Swaps)	1
BFS + Ordered Sets (Parity sets)	1
BFS + Priority Queue (Min-Heap / Max-Heap).	1
BFS + Priority Queue / Monotonic Stack	1
BFS / DFS (State Space Generation)	1
BFS / DFS Digit Construction	1
BFS / DFS with Edge Direction Tracking	1
BFS / DFS with Positional Indexing.	1
BFS / DFS with Visited State Set	1
BFS / Dijkstra (Day-by-Day)	1
BFS / Dijkstra on Grid State.	1
BFS / Dijkstra on Sorted Checkpoints.	1
BFS / Dijkstra on State Space (Grid + Bitmask).	1
BFS from Buildings (Multi-Source Distance Accumulation)	1
BFS from every node (or Edge Removal)	1
BFS on (CurrentNode, VisitedMask) State Space.	1
BFS on 3D State (row, col, k)	1
BFS on Busses/Routes (Bipartite-like Graph).	1
BFS on Matrix State (Bitmasking)	1
BFS on State (x, y, bitmask_of_keys).	1
BFS Precomputation + Bitmask DP	1
BFS Queue (Visualizing the search space widening with every time step).	1
BFS using a Queue.	1
BFS with a Double-ended Queue (Deque) and a boolean direction flag.	1
BFS with a Queue + Prepending to a Deque/Linked List for the result (or standard array reversal, which is practically optimal in many languages).	1
BFS with Character Mutation (a to z).	1
BFS with Multi-State Tracking	1
BFS with Multi-Value Jump Optimization	1
BFS with State Pruning (Shortest Path in String Graph).	1
BFS with Symmetry + Pruning (or Math)	1
BFS with Two-Condition Entry (Keys + Possession)	1
BFS/DFS with Hostname Filtering	1
BFS/DFS with Sliding Logic From a cell, move in one direction using a `while` loop until hitting a wall. Treat the *stop point* as a graph node.	1
Bi-directional Dynamic Programming (Meet-in-the-Middle) / Graph BFS. State = (u, v) .	1
Bi-directional Linear Scan with Last-Seen Map. O(N + Q)	1
Binary Exponentiation	1
Binary Exponentiation (Fast Power)	1
Binary Exponentiation (Modular Pow)	1
Binary Indexed Tree (Fenwick Tree) / Segment Tree	1
Binary Lifting (Sparse Table)	2
Binary Offset Mapping	1
Binary Search	1
Binary Search (bisect) on a sorted copy of nums1 to find the closest value to nums2[i].	1
Binary Search (Flattened Index)	1
Binary Search (if sorted) or Single Pass Min-Tracker.	1
Binary Search (Iterative Descent)	1
Binary Search (Left-most target)	1
Binary Search (Leftmost Boundary)	1
Binary Search (O(log N)).	1
Binary Search (on 1D projections of the 2D grid)	1
Binary Search (Range [0, x])	1

Binary Search (Range [1, num])	1
Binary Search (Range-based Gap Detection)	1
Binary Search (Range-Based)	1
Binary Search (Range-based).	1
Binary Search (Shrinking Bound)	1
Binary Search (Sorted Heaters) Sort heaters. For each house, binary search for the two closest heaters (one left, one right).	1
Binary Search (Ternary Splitting)	1
Binary Search (Triple Pass).	1
Binary Search (Two Bounds)	1
Binary Search + Bit Counting Math	1
Binary Search + Greedy Priority Queue	1
Binary Search + Inclusion-Exclusion (PIE)	1
Binary Search + Max-Heap (Priority Queue)	1
Binary Search + Sort Store all `start` values in a sorted list with original indices. For each `end`, binary search the list.	1
Binary Search + Two Pointers.	1
Binary Search / Jump Search	1
Binary Search / Math Search for k such that frac{k(k+1)}{2} le N or use the quadratic formula directly.	1
Binary Search on Answer	5
Binary Search on Answer (k) + Two Pointers.	1
Binary Search on Answer (Range [1, max(piles)]).	1
Binary Search on Answer (Range)	2
Binary Search on Answer (Y-coordinate). O(N log(Range/Epsilon))	1
Binary Search on Answer + Bit Digit DP (Math)	1
Binary Search on Answer + Difference Array	1
Binary Search on Answer + Disjoint Set Union (DSU) / Connected Components Check.	1
Binary Search on Answer + Edge Cases	1
Binary Search on Answer + Fenwick Tree (BIT) / Merge Sort for Inversions.	1
Binary Search on Answer + Greedy	2
Binary Search on Answer + Greedy Math (Arithmetic Progression sum).	1
Binary Search on Answer + Greedy Traversal. O(N log(Max_Score))	1
Binary Search on Answer + Greedy Verification	1
Binary Search on Answer + Greedy with Monotonic Queue (Deque)	1
Binary Search on Answer + Greedy.	1
Binary Search on Answer + Greedy/Flow.	1
Binary Search on Answer + Inclusion-Exclusion Principle.	1
Binary Search on Answer + Intersection Checks	1
Binary Search on Answer + Pruned Backtracking.	1
Binary Search on Answer + Reversed Graph Traversal. O(E log W)	1
Binary Search on Answer + Sliding Window	2
Binary Search on Answer + Sliding Window / Greedy Check.	1
Binary Search on Answer + Sliding Window / Segment Tree.	1
Binary Search on Answer + Sliding Window.	1
Binary Search on Answer + Validation (Greedy/Bipartite)	1
Binary Search on Answer / Math	1
Binary Search on Answer / Priority Queue	1
Binary Search on Answer Binary search the "max sum" (Low: max(arr), High: sum(arr)). Greedily count chunks needed for each guess.	1
Binary Search on Answer Range	4
Binary Search on Answer Range + Inclusion-Exclusion Principle	1
Binary Search on Answer Range + Two Pointers/Binary Search for Counting	1
Binary Search on Answer Range.	1
Binary Search on Answer Space (Time).	1
Binary Search on Answer Space + Greedy Counting.	1
Binary Search on Answer Space + Number Theory (Inclusion-Exclusion Principle using LCM).	1
Binary Search on Answer.	6
Binary Search on Candidate Points	1
Binary Search on Columns/Rows	1
Binary Search on Cubic Formula	1

Binary Search on Days + BFS/DFS	1
Binary Search on Distance + Sliding Window	1
Binary Search on Even Indices.	1
Binary Search on H-Boundary	1
Binary Search on Leaf Nodes + Tree Height Properties	1
Binary Search on Length + Rolling Hash (Rabin-Karp).	2
Binary Search on Length + Rolling Hash (Rolling Hash)	1
Binary Search on Length The number of '1's can only be between 2 and 64. For each fixed length m, binary search for base k.	1
Binary Search on Monotonic Property	1
Binary Search on Partitions	1
Binary Search on Range + Upper Bound	1
Binary Search on Slope Direction.	1
Binary Search on Slope.	1
Binary Search on the Answer Space + Greedy Sliding Window.	1
Binary Search on the Answer.	1
Binary Search on the Answer. Guess a "Finish Time" T. Write a greedy O(N) checker to see if all rides can be completed within time T.	1
Binary Search on Value or Min-Heap Merge	1
Binary Search on Value Range	2
Binary Search on Value Threshold / Greedy Math	1
Binary Search with Duplicate Handling (O(log N) average, O(N) worst).	1
Binary Search.	1
Binary Tree Symmetry / Recursion	1
Bipartite Check + Component BFS Max Depth	1
Bipartite Coloring (Parity Counting)	1
Bipartite Graph Matching / Min Vertex Cover.	1
Bipartite Graph Maximum Matching (Kuhn's Algorithm / DFS with Backtracking).	1
Bit contribution counting array tracking 1s, coupled with right-shift carry propagation logic.	1
Bit Count Frequency + Inclusion-Exclusion	1
Bit Manipulation (bitCount) + Constant Set Check	1
Bit Manipulation (Bitmasking)	1
Bit Manipulation (Column-wise Bit Counting)	1
Bit Manipulation (Common Prefix Match)	1
Bit Manipulation (Exponential Subtraction)	1
Bit Manipulation (Extract and Shift).	1
Bit Manipulation (Flipping Trailing Ones)	1
Bit Manipulation (Greedy Bit-by-Bit)	1
Bit Manipulation (Left Shifting)	1
Bit Manipulation (Modulo 3 Summing).	1
Bit Manipulation (Popcount + Bit Length)	1
Bit Manipulation (Popcount) / Math	1
Bit Manipulation (Power of 2 - 1)	1
Bit Manipulation (Right Shift) + Distance Tracker.	1
Bit Manipulation (Shift and Add)	1
Bit Manipulation (XOR + Rightmost Set Bit)	1
Bit Manipulation (XOR Masking).	1
Bit Manipulation (XOR Shift).	1
Bit Manipulation (XOR) or Character Summation	1
Bit Manipulation (XOR).	1
Bit Manipulation (XOR, AND)	1
Bit Manipulation (Zero-Bit Filling)	1
Bit Manipulation + Masking	1
Bit Manipulation / Math	1
Bit Manipulation Group every 4 bits using bitwise AND (num & 15), then right shift by 4 (num >>> 4).	1
Bit Manipulation: (n > 0) and (n & (n-1) == 0)	1
Bit-by-Bit Counting / Mathematical Combinatorics.	1
Bit-by-Bit Counting For each bit position (0-31), count how many numbers have a '1' (k) and how many have a '0' (n-k). Distance contribution is k x (n-k).	1
Bit-by-Bit Reconstruction	1

Bit-Counting + Greedy Reconstruction	1
Bit-Position Counting	1
Bitmask + Backtracking Find character differences as bitmasks. Use DFS to find the shortest abbreviation mask that covers all difference constraints.	1
Bitmask + Prefix XOR	1
Bitmask + Prefix XOR (First Occurrence)	1
Bitmask / Binary Enumeration	1
Bitmask / Profile DP	1
Bitmask DP	3
Bitmask DP (Submask Iteration)	1
Bitmask DP (Ternary/Base-3 or Base-2 with double slots).	1
Bitmask DP (TSP Variant)	1
Bitmask DP + Floyd-Warshall	1
Bitmask DP + Sorting Values	1
Bitmask DP + State Compression.	1
Bitmask DP + Submask Iteration	1
Bitmask DP / Backtracking with State Compression.	1
Bitmask DP / State Compression.	1
Bitmask DP over Target Array + LCM. O(N x 2^|target|)	1
Bitmask Dynamic Programming (DP on Subsets).	1
Bitmask Dynamic Programming (Inverted)	1
Bitmask Dynamic Programming (Knapsack Variant)	1
Bitmask Dynamic Programming (Primes as Bits).	1
Bitmask Dynamic Programming + GCD Precomputation.	1
Bitmask Dynamic Programming.	1
Bitmask Enumeration + Precomputed Palindromes	1
Bitmasking (Bitwise AND Comparison)	1
Bitmasking (Brute Force is the only way, but Bitmasking is the production-ready implementation).	1
Bitmasking + Array.	1
Bitmasking + Backtracking (Pruned)	1
Bitmasking + Disjoint Set Union (DSU).	1
Bitmasking + Gosper's Hack	1
Bitmasking + Hash Map / Advanced State Tracking	1
Bitmasking + Prefix XOR + Frequency Hash Map	1
Bitmasking + Subset Enumeration	1
Bitmasking / Backtracking Iterator	1
Bitmasking / Set Hashing	1
Bitmasking / Sorting + Hash Set.	1
Bitmasking / State Space Search	1
Bitmasking for character presence + Hash Map frequency counting.	1
Bitmasking Subset Enumeration + DFS / Floyd-Warshall	1
Bitwise AND (&) Operator in SELECT/WHERE	1
Bitwise AND Properties / Hash Set	1
Bitwise Evaluation / Prefix Counts.	1
Bitwise Logic (Total XOR Property).	1
Bitwise Logic / CASE statement with ID manipulation.	1
Bitwise Mask XOR Find the highest set bit. Create a mask of all 1s of that length. Result is `num ^ mask`.	1
Bitwise Operations & Masking	1
Bitwise OR (Math / Bit Manipulation Insight)	1
Bitwise Parity Counting	1
Bitwise Parity Math / Combinatorics	1
Bitwise Powers of 2 Math.	1
Bitwise Prefix XOR & Frequency Counting.	1
Bitwise Shift & Subtract	1
Bitwise Shift and Masking.	1
Bitwise Simulation (Carry Manipulation).	1
Bitwise Trie Insert all numbers into a Binary Trie. For each number, traverse the trie trying to pick the opposite bit.	1
Bitwise XOR (Sum) & AND-Shift (Carry) Loop	1

Bitwise XOR + Bit Counting XOR the two numbers (x oplus y). Count the set bits using Brian Kernighan’s algorithm.	1
Bitwise XOR + Brian Kernighan's Alg	1
Bitwise XOR Properties (`A ^ B = C` implies `A ^ C = B`).	1
Bitwise XOR Properties + Hamming Distance	1
Blocking Execution Context (One task occupies the stack while others wait in the queue).	1
Blocking Main Thread (Visualizing the CPU stuck in a loop waiting for the async task).	1
Boolean Array + Step Increment	1
Boolean Array Masking	1
Boolean Frequency Array / Hash Set	1
Boolean Indexing: df[df['id'] == 101]	1
Boolean Mask + Interval Merging.	1
Border-to-Interior DFS (Inverse Flood Fill).	1
Bottom-Up DFS (Post-order Traversal)	1
Bottom-Up DFS (Post-order Traversal) with a Global Max tracker.	1
Bottom-Up DFS (Post-order Traversal).	1
Bottom-Up DFS (returning pairs).	1
Bottom-Up DFS. Return the height of the subtree if it is balanced. If it is ever unbalanced, return -1 immediately to short-circuit.	1
Bottom-Up Dynamic Programming (In-place or rolling 1D array).	1
Bottom-Up Dynamic Programming (Reverse Tabulation).	1
Bottom-Up Greedy (Post-order Traversal)	1
Bottom-Up Post-Order DFS	1
Bottom-Up Post-Order DFS (Tree DP)	1
Bottom-Up Tree DP (Post-order Traversal).	1
Boundary Analysis (Sorting).	1
Boundary Contraction	1
Boundary Shrinking	1
Bounded BFS / Geometric Limitation.	1
Bounded Knapsack Dynamic Programming.	1
Boyer-Moore Voting + Random Sampling / Segment Tree	1
Boyer-Moore Voting Algorithm (Extended)	1
Boyer-Moore Voting Algorithm.	1
Brain Teaser / Math (Length Comparison).	1
Brain Teaser / Math (Parity).	1
Brainteaser / Array Math (Ghost Ants)	1
Brainteaser / Logical Deduction	1
Breadth-First Search (BFS)	3
Breadth-First Search (BFS) + Mathematical Formula	1
Breadth-First Search (BFS) / Bidirectional BFS	1
Breadth-First Search (BFS) / Level-order traversal with a Visited HashSet.	1
Breadth-First Search (BFS) / Shortest Path	1
Breadth-First Search (BFS) Shortest Path on a 3D State Graph	1
Breadth-First Search (BFS) String Building + Frequency Pruning	1
Breadth-First Search (BFS) with Column Tracking (Hash Map)	1
Brian Kernighan’s Algorithm (n & (n-1)).	1
Brute Force (Small Constraints)	1
Brute Force (Small Constraints) / Trie (Optimal)	1
Brute Force / Constraint Optimization	1
Brute Force Path Memory (Storing every conceivable route configuration simultaneously).	1
Brute Force with Early Exit (Small Constraints)	1
BST Property Navigation (Binary Search).	1
BST Property Navigation (Succ = smallest of right subtree)	1
BST Property Traversal (Value Comparison)	1
BST Traversal (Iterative)	1
BST Traversal leveraging Parent Pointers.	1
Bucket Sort (Counting Citations)	1
Bucket Sort (Pigeonhole Principle).	1

Bucket Sort / Counting Tally	1
Bucket Sort / Frequency Map + Greedy Flattening	1
Bucket Sort / Greedy with Priority.	1
Bucket Sort / Prefix Sum	1
Bucket Sort / Sliding Window	1
Bucket Sort of Iterators / Waiting Lists	1
Bucket Sort or Min-Heap	1
Bucket Sort Use an array where the index is the frequency. Put characters into the bucket at `frequency[char]` index.	1
Buffer Scan (Visualizing the database engine reading the text column repeatedly for each word).	1
Buffered Chunk Processing with Bound Checking.	1
Built-in Bit Manipulation (popcount)	1
Built-in O(N log N) Sort with a strictly defined Custom Comparator function returning -1, 0, or 1 based on a prioritized rule cascade.	1
Calculate average, throw array into a Hash Set, and start checking from ceil(avg) + 1 .	1
Call Stack & Arguments Object frame allocation.	1
Call Stack & Redundant Object Allocation (Draw stacked boxes for deep recursion + identical clone structures in heap).	1
Canonical Hashing (DFS + Sort)	1
Cantor's Diagonalization	1
Cascading / Bitmasking	1
Case When + Correlated Subquery (or In/Not In)	1
CASE WHEN + CROSS JOIN (Static Ranges)	1
CASE WHEN + SUM (Pivoting)	1
CASE WHEN / Aggregate Filtering	1
Case When Logic (Geometric Constraints)	1
CASE WHEN with IN clause (Subquery).	1
Case-Based Boundary Analysis. O(n)	1
Case-Based Comparison (Counter + Mismatch Pointer).	1
CEIL + SUM + LEFT JOIN	1
Center Point + Diagonal Property	1
Chained Operations: df.loc[...].sort_values(...)[['name']]	1
Chaining (Array of Linked Lists / BSTs)	1
Character Frequency Counting + Greedy Symmetric Placement.	1
Character Frequency Signatures (Even/Odd split).	1
Chebyshev Distance (Math/Geometry)	1
Chebyshev Distance (Max of ΔX, ΔY)	1
Chunked Transfer (Length + Delimiter)	1
Circular Buffer (Fixed-size Array)	1
Circular Buffer / Fixed-size Array with Head/Tail Pointers.	1
Closure + clearTimeout / setTimeout	1
Closure + Hash Map (Caching)	1
Closure + setTimeout + Stored Arguments	1
Closure / Higher-Order Function	1
Closures / Factory Pattern	1
Column-Wise Linear Scan	1
Column-Wise Max Pre-calculation	1
Combinatorial Construction. Generate palindromes directly by mirroring the "first half" of the binary bits.	1
Combinatorial Counting (Pascal's Triangle logic)	1
Combinatorial Counting + Digit-by-Digit Construction. O(n)	1
Combinatorial Row Storage (Visualizing all row combinations in memory).	1
Combinatorics (Cantor Expansion) + Modular Multiplicative Inverse + Precomputed Factorials.	1
Combinatorics (Contribution of Each Element)	1
Combinatorics (Handshake Lemma)	1
Combinatorics (Inclusion-Exclusion Principle)	1
Combinatorics (nCr mod10) using Lucas Theorem & CRT. O(N)	1
Combinatorics (nCr) + Bit-by-Bit Construction.	1
Combinatorics (nCr) + Fast Power Math. (O(N))	1
Combinatorics (Pascal's Triangle / Math)	1

Combinatorics (Permutations with Repetition) + Modular Multiplicative Inverse.	1
Combinatorics (Permutations)	1
Combinatorics (Stars and Bars)	1
Combinatorics + Fenwick Tree (BIT)	1
Combinatorics + Frequency Counting.	1
Combinatorics + Modular Inverse	1
Combinatorics + Prefix/Suffix Maps	1
Combinatorics / 4-State DP (Markov Chain approach)	1
Combinatorics / DP (Parity Switch)	1
Combinatorics / Gap Counting	1
Combinatorics / Greedy Math.	1
Combinatorics / Hash Map	1
Combinatorics / Math (DP)	1
Common Denominator (LCM) + Final GCD.	1
Common Table Expression (CTE) + LEFT JOIN filtering.	1
Common Table Expression (CTE) / GROUP BY	1
Common Table Expressions (CTE) + Aggregation	1
Common Table Expressions (CTE) + Window Functions (RANK/DENSE_RANK)	1
Common Table Expressions (CTEs) + GROUP BY aggregation + Self JOIN.	1
Common Table Expressions (CTEs) to group by employee_id, year, week , aggregate sums, filter > 20 , then self-join.	1
Compare Current vs Next	1
Complement Counting (Total - Good)	1
Complementary Counting (Total - Odd)	1
Complementary Counting + Binary Exponentiation.	1
Component Labeling (Island ID) + Hash Map.	1
Concurrency: 3 Semaphores	1
Concurrency: Mutexes / Semaphores / Promises	1
Concurrency: Two Alternating Semaphores	1
Condition Variables / Semaphores	1
Conditional Aggregation (SUM + CASE)	2
Conditional Aggregation (SUM + IF/CASE)	2
Connected Components (BFS/DFS) + Global Totals	1
Connected Components (DFS/BFS) + Combinatorics	1
Constrained Backtracking / DFS	1
Constructive Algorithm / Greedy Pattern Filling.	1
Constructive Pattern / Mathematical Observation	1
Contribution of Each Character (Index tracking).	1
Contribution Technique (Position-wise Counting)	1
Controlled Stack-based DFS (Lazy Loading).	1
Convex Hull + 3 Pointers (Shoelace Formula).	1
Convex Hull + Rotating Calipers.	1
Coordinate Compression + Segment Tree / Sparse Map.	1
Coordinate Compression / Line Sweep	1
Coordinate Compression / Sweep Line Map	1
Coordinate Geometry + Single Pass Accumulation.	1
Coordinate Mapping (Direct Indexing).	1
Coordinate Mapping (Precomputation)	1
Coordinate Math (i, j) Simulation	1
Coordinate Math (Manhattan Distance Pathing)	1
Coordinate Min-Max Comparison	1
Coordinate Rotation (Chebyshev Transformation)	1
Coordinate Sorting + Monotonic Optimization	1
Corner Count Set + Area Summation	1
Count Factors of 5 (Legendre's Formula).	1
COUNT(DISTINCT customer_id)	1
COUNT(DISTINCT) with GROUP BY	1
Counting & Filling (Stable Parity). O(N)	1

Counting + Combinatorics (Three Cases)	1
Counting / Frequency (Linear Scan)	1
Counting 1s + Suffix Matching	1
Counting Sort (Frequency Map)	1
Counting Sort / Frequency Array	1
CROSS JOIN	1
CROSS JOIN (Cartesian Base) + LEFT JOIN	1
Cross Join (Master Template) + Left Join	1
CROSS JOIN (Template) + LEFT JOIN	1
CROSS JOIN / SELF JOIN	1
Cross Join with Duplicate Filtering (Upper Triangular Check).	1
Cross Product (Orientation) For every three consecutive points, calculate the cross product. All cross products must have the same sign (all positive or all negative).	1
Cross-Multiplication (Slope Equality)	1
CTE + Interval Merging (Sweep Line SQL)	1
CTE + Window Functions (RANK)	1
CTE / UNION for Bidirectional Friends + JOIN	1
CTE / Window Functions	1
CTE Aggregation + Window Function (LEAD)	1
CTE with `COUNT(*) OVER(PARTITION BY salary)` + `DENSE_RANK()`.	1
CTEs + Not In / Except (Set Difference)	1
Custom Comparator / Lambda Sorting.	1
Custom Comparator / Partitioning	1
Custom Comparator with __builtin_popcount	1
Custom Comparator with Bitwise Manipulation	1
Custom Greedy Sorting	1
Custom Greedy Sorting (Comparator: S1+S2 vs S2+S1).	1
Custom Sort (Length then Lexicographical)	1
Custom Sort + Two Pointers (Subsequence Check).	1
Cycle Decomposition (Permutation Graph).	1
Cycle Decomposition (Permutation Theory).	1
Cycle Detection (Hash Map)	1
Cycle Detection (Pigeonhole) Record the `index` in s_2 after each s_1. If the same index repeats, a cycle is found. Calculate remainder math.	1
Cyclic Sort (Index as Map)	1
Date Filtering + Grouping	1
Datetime library or Formula-based difference. `(Date2 - Date1).days * rate`.	1
Days Since Epoch (1971) Calculation	1
Deep Call Stack (Visualize memory frames for every nested level of the JSON structure).	1
Deep Call Stack Grid (Showing massive stack frames for O(N) depth and O(2^N) repeated states).	1
Deep Recursion Stack (Visualizing maximum stack depth O(N) but an exploding number of string copies across states).	1
Degree Array + Edge Adjacency Matrix	1
Denary Tree Traversal Visualize numbers as a 10-ary tree (1, 10, 100...). Calculate "steps" between prefixes to skip whole subtrees.	1
Depth-First Search (DFS) / BFS	1
Depth-First Search (DFS) or Breadth-First Search (BFS)	1
Deque + HashSet (Constant-Time Movement and Collision)	1
Descending Order Sorting or Single-Pass Mapping.	1
Deterministic Finite Automaton (DFA)	1
Deterministic One-Pass (DFA)	1
df.drop_duplicates(subset=['id'])	1
df.dropna(subset=['name'])	1
df.head(3) (Slicing)	1
df.melt(id_vars=['product'], ...)	1
df.pivot(index='month', columns='city', values='temperature')	1
df.rename(columns={...})	1
df.shape Attribute Access	1
df['grade'].astype(int)	1
df['quantity'].fillna(0)	1
df['salary'] *= 2 (Vectorized Math)	1

DFS (Leaf-only Extraction).	1
DFS (Parent Tracking) + Pointer Manipulation	1
DFS (Post-order Traversal) with Existence Flags	1
DFS (Post-order) + Target Set	1
DFS (Pre-order Traversal of a 10-ary Tree)	1
DFS (Pre-order) with target subtraction (passing `target - node.val` down the recursion).	1
DFS (to Map) + BFS (to Find Shortest Path).	1
DFS + Ancestor Hash Map	1
DFS + Ancestor Value Tracking (Value-based Map).	1
DFS + Backtracking (Parent Tracking)	1
DFS + Bit Manipulation	1
DFS + Combinatorics (Product of Subtree Counts)	1
DFS + Degree Counting	1
DFS + Fenwick Tree (BIT) / Merge Sort Tree.	1
DFS + Frequency Array (Counting)	1
DFS + Hash Set.	1
DFS + Memoization (or Sorting + DP)	1
DFS + Memoization (Topological Sort Pattern)	1
DFS + Memoization + Pruning Use string compression (remove 3+ matches) and a Map to store "State: MinBalls". Skip redundant placements.	1
DFS + Memoization / DP (State: r, c, dir, turn_taken, next_val). O(N · M)	1
DFS + Path Serialization.	1
DFS + Precomputation (Level-wise Max Depths)	1
DFS + Sliding Window on Path + Hash Map for Last Depth. O(N)	1
DFS + Subtree Size Math	1
DFS + Trie (Bitwise)	1
DFS / BFS (Parental Context)	1
DFS / BFS + Grid Traversal (Visited tracking)	1
DFS / BFS from the click point with strict boundary/number checks.	1
DFS / BFS or Union-Find (Disjoint Set Union).	1
DFS / BFS with Connectivity Logic.	1
DFS / BFS with Node Mapping (Memoization)	1
DFS / Preorder Traversal + Hash Set	1
DFS mapping (to reveal grid) + Dijkstra's Algorithm (Shortest Path).	1
DFS Post-order Traversal + Tracking Node with Value 1	1
DFS Recursion (Post-Order)	1
DFS Subtree Precomputation + Ancestor Check	1
DFS Tree Flattening + Manacher's Algorithm	1
DFS with Arithmetic Accumulation.	1
DFS with Backtracking	1
DFS with Backtracking / String Builder	1
DFS with Backtracking. Maintain a single shared path list: `push` node, recurse, `pop` node.	1
DFS with Height-Based Grouping	1
DFS with In-place Sink (or BFS/Union-Find).	1
DFS with In-Place Sinking (Visited marking).	1
DFS with Local Set Use backtracking. At each recursive level, use a `HashSet` to skip duplicate numbers to avoid redundant subsequences.	1
DFS with Max-So-Far Parameter	1
DFS with Memoization / Dependency Graph Resolve. O(Final Length)	1
DFS with Path Tracking (Bottom-Up Logic)	1
DFS with Probability Propagation	1
DFS with Pruning / Bitmask DP Sort matchsticks descending. Try placing in a side; if it exceeds `target`, backtrack immediately.	1
DFS with Range Tracking (Top-Down).	1
DFS with Running Sum/State	1
DFS with Sliding Window on Tree Paths + Hash Map. O(N)	1
DFS with State Marking (3-Coloring).	1
DFS with Subtree Size Precomputation	1
DFS with Timestamping (Kahn’s or Visited Array)	1
DFS/BFS Traversal (Same as Brute force, as you must visit every node to check).	1

DFS/BFS with a Hash Map (Mapping Original Nodes to Cloned Nodes).	1
DFS/BFS with Symmetrical Dual-Pointers	1
Diagonal Indexing (i - j) + HashMap/Bucket Sort. $O(N^2)$	1
Diagonal Iteration + Hash Set	1
Diagonal Point Matching + Hashset	1
Difference Array (Line Sweep)	1
Difference Array (Prefix Sum of Queries)	1
Difference Array (Prefix Sum of Shifts)	1
Difference Array (Prefix Sum Optimization)	1
Difference Array (Sweep Line)	1
Difference Array (Sweep Line) or Fenwick Tree (Binary Indexed Tree) to mathematically track shifts.	1
Difference Array + Greedy (Sweep Line)	1
Difference Array + Greedy Peak Tracking.	1
Difference Array / Greedy	1
Difference Array / Greedy Peaks	1
Difference Array / Lazy Propagation (Segment Tree).	1
Difference Array / Line Sweep.	1
Difference Array / Prefix Sum	1
Difference Array / Prefix Sum (Line Sweep).	2
Difference Array / Prefix Sums	1
Difference Array / Prefix Sums.	1
Difference Array / Sweep Line	2
Difference Array / Sweep Line Algorithm	1
Difference Array String Hashing + Frequency Map.	1
Digit DP (Combinatorial Counting).	1
Digit DP (Combinatorial Math)	1
Digit DP (Dynamic Programming)	1
Digit DP (Dynamic Programming) / Combinatorics.	1
Digit DP + KMP (Automata State)	1
Digit DP + Precomputed Sieve	1
Digit DP / Arithmetic Progression Formula.	1
Digit DP / Combinatorics.	1
Digit DP / Combinatorics. Precompute valid target "set bit counts", then build valid numbers bit by bit using combinations (nCr).	1
Digit DP / Constructive Greedy	1
Digit DP / Fibonacci Pattern.	1
Digit DP / Mathematical Counting (Digit-by-Digit)	1
Digit DP with Remainder State	1
Digit Dynamic Programming	1
Digit Dynamic Programming (Bitmask)	1
Digit Dynamic Programming (Digit DP)	2
Digit Dynamic Programming (State: index, tight, productMod, sum). O(digits · sum · prod)	1
Digit Dynamic Programming.	1
Digit Extraction (Modulo Arithmetic)	1
Digit Extraction (Pop & Push)	1
Digit Frequency Counting (Anagram Check).	1
Digit Manipulation + Hash Set	1
Digit-based DFS / Backtracking.	1
Digit-by-Digit Math w/ Carry	1
Digital Root (Mathematical Modulo 9)	1
Dijkstra (from Node N) + DAG DP (Topological Sort/Memoization).	1
Dijkstra + Auxiliary "Teleport Hub" nodes. Instead of V x V edges, connect all nodes to one "Hub" with specific entry/exit costs.	1
Dijkstra + DP (Counting)	1
Dijkstra / BFS + A* Search or Dynamic Programming.	1
Dijkstra / Binary Search + BFS	1
Dijkstra with State (City, DiscountsUsed)	1
Dijkstra with String Priority Use a Priority Queue. Store `(distance, path_string, x, y)`. Only update if distance is smaller OR distance is equal but string is lexicographically smaller.	1
Dijkstra's (Bi-directional) + Edge Relaxation	1

Dijkstra's Algorithm	1
Dijkstra's Algorithm (with Priority Queue)	1
Dijkstra's Algorithm + Parity Waiting Logic.	1
Dijkstra's Algorithm with Bitmask DP (State Space Search). State = (Current_Node, Visited_Mask) .	1
Dijkstra's on a Coordinate Graph	1
Dijkstra's x3 (Steiner Tree on 3 Nodes variant).	1
Dijkstra’s Algorithm (Min-Priority Queue)	1
Dijkstra’s Algorithm + Edge Coverage Math.	1
Dijkstra’s Algorithm + Prime Sieve	1
Dimension Reduction + Interval Merging	1
Dimension Sorting + LIS (Dynamic Programming)	1
Direct Accumulation (Math optional)	1
Direct Array Placement / Cyclic Sort	1
Direct Comparison / O(1) Access	1
Direct Digit Extraction (Modulo/Div)	1
Direct Hash Map / ASCII Array Indexing.	1
Direct Index Mapping (Modulo Arithmetic).	1
Direct Index Mapping (One-Pass)	1
Direct Indexing Compare `words[i].charAt(j)` with `words[j].charAt(i)` with boundary checks.	1
Direct Integer Accumulation	1
Direct Linear Scan with Set Lookup.	1
Direct Mapping + Pairwise Check	1
Direct Math (Division & Min). O(1)	1
Direct Math (Positional Notation).	1
Direct Math Operations / ALU	1
Direct SQL Filtering (Math inside WHERE clause)	1
Direct SQL WHERE Clause + ORDER BY.	1
Direct String Transformation + Regex	1
Directed Acyclic Graph (DAG) - Count In-Degrees	1
Directed Acyclic Graph (DAG) with Topological Updates.	1
Direction Array + Matrix Simulation	1
Directional Ray Casting + Primality Sieve	1
Disjoint Set Union (DSU)	2
Disjoint Set Union (DSU) + Sorting by Value	1
Disjoint Set Union (DSU) / Connected Components	1
Disjoint Set Union (DSU) / Connected Components + Hash Maps.	1
Disjoint Set Union (DSU) / Connected Components + Sorting	1
Disjoint Set Union (DSU) / Connected Components.	1
Disjoint Set Union (DSU) / Kruskal's Variant with DP.	1
Disjoint Set Union (DSU) / Union-Find	1
Disjoint Set Union (DSU) or Validated BFS	1
Disjoint Set Union (DSU) with Path Compression. For every session connection, union the two nodes and decrement the total cluster count.	1
Disjoint Set Union (DSU) with Restrictions Check	1
Disjoint Set Union (Union-Find)	1
Disposable HashSets (Draw hundreds of temporary Sets being created and garbage-collected for every individual cell).	1
Distance Array + Priority Queue (Drawing the standard Dijkstra memory footprint).	1
Distance Sorting / Hash Set of Lengths.	1
Distinct Counting + Null Handling.	1
Divide & Conquer + Combinatorics (Pascal's Triangle)	1
Divide & Conquer Recursively check if a quadrant is "leaf". If all 4 children are leaves and same value, merge.	1
Divide and Conquer (Odd/Even Separation)	1
Divide and Conquer (Quadtree Decomposition)	1
Divide and Conquer (Recursive Splitting).	1
Divide and Conquer (Split on Invalid Characters)	1
Divide and Conquer + Memoization (Top-Down DP)	1
Divide and Conquer / DFS. Find mid, make root, recurse left half, recurse right half.	1

Divide and Conquer / Sweep Line	1
Divide and Conquer or Segment Tree with "Monotonic Property" optimization. Focus on boundaries where the XOR value changes (at most 32 times).	1
Divisor Counting + Inclusion-Exclusion	1
Divisor Iteration + Freq Map Comparison	1
Divisor Search + Post-order DFS	1
Double Array Concatenation + Monotonic Queue/DP	1
Double BFS (or DFS with Height Tracking)	1
Double BFS / DFS	1
Double Binary Search	1
Double DFS / Max Flow Min Cut (Path Neutralization).	1
Double Ended Queue (Deque) / Two Pointers	1
Double Hash Maps (Count of Counts)	1
Double In-place Reversal (Reverse Whole + Reverse Words).	1
Double JOIN + CASE Statement	1
Double Logarithmic Loop (Logarithmic Bounds)	1
Double Loop with Square Root Check	1
Double Topological Sort (Kahn's Algorithm)	1
Double-Ended Queue (Deque) + Running Sum	1
Doubly Linked List + TreeMap (or Lazy Deletion PQ)	1
Doubly Linked List w/ Dummy Nodes	1
DP (1D Calendar / Days-based)	1
DP (1D/2D) `dp[i][j]` stores max score diff from subarray i...j. `dp[i][j] = max(nums[i]-dp[i+1][j], nums[j]-dp[i][j-1])`.	1
DP (Difference as State)	1
DP (Hash Map of Sets) Map each stone to a Set of jump lengths that reached it.	1
DP (Index, Remaining K, Last Parity)	1
DP (Interval 3D)	1
DP (Max Length per Char) Track the longest "consecutive" sequence ending at each character ('a' through 'z'). The sum of these 26 values is the result.	1
DP (Sorted Robots & Factories)	1
DP (Stirling Numbers of Second Kind)	1
DP (Subset Sum) Transform to finding a subset with sum (S + target)/2. Solve using 1D Knapsack DP.	1
DP + BFS / Optimized Recursion	1
DP + Binary Search (Weighted Interval Scheduling).	1
DP + Bitmask	1
DP + Bitset (Bit-Level Parallelism)	1
DP + Character Frequency Pre-computation	1
DP + Combinatorics (Digits, Count, Sum)	1
DP + Freq Map + Two Pointers	1
DP + Generating Functions / Polynomial Interpolation or NTT.	1
DP + Global Max Tracking (O(Nk))	1
DP + Largest Histogram (Q84)	1
DP + LCP (Suffix) + Prefix Sums	1
DP + Longest Common Prefix (LCP) with Rolling Hash or Suffix Array	1
DP + Median Distance Pre-computation	1
DP + Median Properties / Slopes	1
DP + Memoization (State: zeros, ones, last_bit)	1
DP + Memoization + Bitwise AND	1
DP + Memoization / Combinatorics	1
DP + Memoization or Monotonic Stack Sort	1
DP + Monotonic Deque (Sliding Window Maximum)	1
DP + Monotonic Deque / Sliding Window.	1
DP + Monotonic Queue (Deque)	2
DP + Monotonic Queue (Sliding Window)	1
DP + Prefix Sum Optimization	2
DP + Prime Factor Bitmasking	1
DP + Segment Tree or Fenwick Tree. Use a coordinate-compressed Segment Tree to query the sum of `Up/Down` counts for all values smaller/larger than the current element in O(log N).	1
DP + Sliding Window (Prefix Sums)	1
DP + Sorting + Binary Search	1

DP + Ternary Bitmask (State Compression)	1
DP / BFS (Visited Grid)	1
DP / Bitset Optimization	1
DP / Center Expansion + Greedy	1
DP / Central Expansion (Mismatch Count)	1
DP / Contribution of Each Character	1
DP / Middle-Element Counting	1
DP / Pre-calculated Jumps Join sentence with spaces. At each screen column start, pre-calculate which word the next row will start with.	1
DP on Strings / Bitmasking	1
DP with Bitmask	1
DP with Bitmask (Prime Representation)	1
DP with Bitmask (State: Mask of completed tasks)	1
DP with Bitmask / Memoization.	1
DP with Bitmasking.	1
DP with Bitset (Prefix/Suffix ORs)	1
DP with Bitset Optimization	1
DP with Convex Hull Trick (CHT) or prefix sum optimization. O(N^2)	1
DP with Frequency & Sum Mapping	1
DP with Frequency Map + Sliding Window Optimization	1
DP with Frequency Partitioning (i pmod K).	1
DP with Hash Map + Sorting.	1
DP with Interval Grouping (Bucket Sort style)	1
DP with Interval Precomputation	1
DP with Left/Right Prefix Max	1
DP with Lexicographical Greedy Path. O(26 x N)	1
DP with Map Matrix Use an array of HashMaps `dp[i]`, where `dp[i][diff]` stores count of slices ending at i with difference `diff`.	1
DP with Memoization (Suffix Sums)	1
DP with Minimum Tracking (Two-Minima Optimization)	1
DP with Obstacle Guard	1
DP with Prefix Sum Optimization + "Ongoing" State. O(n · k)	1
DP with Pruning (Capped Set size).	1
DP with Remainder State for each Divisor (1-9). O(10 · 10 · N)	1
DP with Sliding Window / Prefix-Suffix Max.	1
DP with Sliding Window Sum.	1
DP with State (lastRemainingIndex, nextIndex). O(N^2)	1
DP with Two Minimums (Optimization)	1
DSU (Disjoint Set Union) + Backtracking + Sorting	1
DSU + In-degree Tracking.	1
DSU + Multiples (Sieve-like)	1
Dual Frequency Array Comparison	1
Dual Frequency Maps (Visualizing two HashMaps being fully rebuilt for every possible split point).	1
Dual Hash Maps (Count of Counts)	1
Dual Hash Maps (Frequency Tracking)	1
Dual Hash Maps (In-Transit & Completed)	1
Dual-Map Bijection or Pattern Normalization.	1
Dual-Pass Dijkstra (Target Distance Matching)	1
Dummy Node + Linear Traversal	1
Dutch National Flag (3-Pointer)	1
Dynamic BFS / Incremental DP	1
Dynamic Prefix Maximum & Max-Difference Tracking	1
Dynamic Programming (0/1 Knapsack variation).	1
Dynamic Programming (1D Array).	1
Dynamic Programming (1D)	2
Dynamic Programming (1D) or BFS	1
Dynamic Programming (1D).	1
Dynamic Programming (1D/2D)	1
Dynamic Programming (2D Range DP)	1

Dynamic Programming (2D Table or 1D Optimized)	1
Dynamic Programming (2D Table)	1
Dynamic Programming (2D)	2
Dynamic Programming (2D) or Two-Pointer Slide	1
Dynamic Programming (2D/3D).	1
Dynamic Programming (3-State Machine)	1
Dynamic Programming (4-Directional Prefix)	1
Dynamic Programming (4-state variables) or Prefix/Suffix Profit Arrays.	1
Dynamic Programming (Base-X Representation)	1
Dynamic Programming (Bellman-Ford variant).	1
Dynamic Programming (Bi-directional Kadane's)	1
Dynamic Programming (Bottom-Up 1D)	2
Dynamic Programming (Bottom-Up)	2
Dynamic Programming (Catalan Number Variation)	1
Dynamic Programming (Catalan Number)	1
Dynamic Programming (Column by Column)	1
Dynamic Programming (Column State Transitions)	1
Dynamic Programming (Divide and Conquer with Memoization)	1
Dynamic Programming (DP Table with State).	1
Dynamic Programming (DP with Hash Table)	1
Dynamic Programming (DP) tracking the minimum and maximum possible sum of `1s - 0s` at each cell.	1
Dynamic Programming (DP). `dp[i][direction]` = number of zigzags ending at index i going "up" or "down".	1
Dynamic Programming (Ending-character Tally)	1
Dynamic Programming (Frequency Transitions)	1
Dynamic Programming (Generating Functions Reconstruction).	1
Dynamic Programming (Histogram Heights) + Greedy Sorting.	1
Dynamic Programming (House Robber Variation)	1
Dynamic Programming (In-place or 1D)	1
Dynamic Programming (In-place)	1
Dynamic Programming (In-Place)	1
Dynamic Programming (Interval DP).	1
Dynamic Programming (Interval DP). `dp[i][j]` stores whether the subarray from i to j can be transformed into the target.	1
Dynamic Programming (Iterative Space Optimized)	1
Dynamic Programming (Kadane's Algorithm Variant)	1
Dynamic Programming (Kadane's variation with Min/Max tracking).	1
Dynamic Programming (Knapsack on Log-Sums) / Prime Factorization.	1
Dynamic Programming (Knapsack variant) or Combinatorial Math. Since capping is a uniform rule, use the frequency of elements to calculate contributions.	1
Dynamic Programming (Knapsack variant) or Meet-in-the-Middle if N is around 30-40.	1
Dynamic Programming (LCS Variant)	1
Dynamic Programming (LCS Variation).	1
Dynamic Programming (Left & Right Prefix/Suffix Arrays)	1
Dynamic Programming (Linear House Robber Variant)	1
Dynamic Programming (Linear State)	1
Dynamic Programming (Linear).	1
Dynamic Programming (LIS Variant).	1
Dynamic Programming (LIS Variation)	2
Dynamic Programming (Longest Common Subsequence Variation)	1
Dynamic Programming (Longest Common Subsequence).	1
Dynamic Programming (Longest Palindromic Subsequence)	1
Dynamic Programming (Matrix Chain Multiplication style).	1
Dynamic Programming (Matrix Exponentiation)	1
Dynamic Programming (MCM - Matrix Chain Multiplication Style).	1
Dynamic Programming (MCM Variant)	1
Dynamic Programming (Min/Max Tracking)	1
Dynamic Programming (Most Significant Bit / Least Significant Bit)	1
Dynamic Programming (O(N^2 * k))	1
Dynamic Programming (Partition DP) / Generating Functions.	1

Dynamic Programming (Pascal's Triangle Variation)	1
Dynamic Programming (Pick/Skip State)	1
Dynamic Programming (Pre-computed Distances)	1
Dynamic Programming (Prefix Sum Opt)	1
Dynamic Programming (Prefix Sum Transition)	1
Dynamic Programming (Range DP).	1
Dynamic Programming (Remainder DP). State = `dp[i][remainder]`. Find the maximum sum we can delete that leaves a remainder of R.	1
Dynamic Programming (Remainder State)	1
Dynamic Programming (Right-to-Left).	1
Dynamic Programming (Row & Column Pre-computation)	1
Dynamic Programming (Sort + Longest Path in DAG)	1
Dynamic Programming (Space Optimized HashMap).	1
Dynamic Programming (Space Optimized)	1
Dynamic Programming (State Compression)	2
Dynamic Programming (State Independence)	1
Dynamic Programming (State Machine Kadane's).	1
Dynamic Programming (State Machine)	2
Dynamic Programming (State Machine) / Greedy.	1
Dynamic Programming (State Machine): 2 states per index (Swapped vs Not Swapped).	1
Dynamic Programming (State Transitions)	1
Dynamic Programming (State: [roads][skips]).	1
Dynamic Programming (State: City x Fuel)	1
Dynamic Programming (State: i, j, diff)	1
Dynamic Programming (State: Index + Prev Value)	1
Dynamic Programming (State: index, last_color_left, last_color_right). O(N)	1
Dynamic Programming (State: mile, lane, switchesLeft). O(N)	1
Dynamic Programming (State: Value & Direction)	1
Dynamic Programming (Stirling Numbers of the First Kind).	1
Dynamic Programming (Subproblem relation).	1
Dynamic Programming (Suffix Sum with Jump)	1
Dynamic Programming (Suffix-based)	1
Dynamic Programming (Tabulation). Building the current row iteratively from the directly previous row.	1
Dynamic Programming (Top-Down Memoization or 2D Bottom-Up).	1
Dynamic Programming (Top-Down with Memo)	1
Dynamic Programming (Two 1D Arrays).	1
Dynamic Programming (Two Passes)	1
Dynamic Programming (Two-State: Ends in 0, Ends in 1)	1
Dynamic Programming (Two-State: Same/Different)	1
Dynamic Programming (Viterbi-like)	1
Dynamic Programming + Bitmasking.	1
Dynamic Programming + Freq Counting	1
Dynamic Programming + Hash Map	1
Dynamic Programming + Hashing (Sorting by Length).	1
Dynamic Programming + Monotonic Queue (Sliding Window)	1
Dynamic Programming + Monotonic Queue/Segment Tree.	1
Dynamic Programming + Precomputed Best Times.	1
Dynamic Programming + Prefix Sums	1
Dynamic Programming + Sliding Window (Prefix Sum of Reachable Nodes).	1
Dynamic Programming / 1D State Tracking.	1
Dynamic Programming / Backtracking with Memoization	1
Dynamic Programming / BFS with Memoization & Pruning.	1
Dynamic Programming / BFS with State. State = `(row, col, direction)`. Since the path is "twisted", we track our entering vector.	1
Dynamic Programming / Combinatorics Math	1
Dynamic Programming / Greedy Matching	1
Dynamic Programming / Greedy Range Extremes	1
Dynamic Programming / Greedy State	1
Dynamic Programming / Greedy Tally	1

Dynamic Programming / Hash Map	1
Dynamic Programming / Linear Counting	1
Dynamic Programming / Math (Running Count).	1
Dynamic Programming / Mathematical Greedy	1
Dynamic Programming / Monotonic Stack.	1
Dynamic Programming / Prefix Sums	1
Dynamic Programming / Prefix Sums.	1
Dynamic Programming / Prefix-Suffix Min.	1
Dynamic Programming / State Machine	1
Dynamic Programming / State Machine. State = `(Day, HasStock)`. Use a simple greedy one-pass if unlimited trades are allowed.	1
Dynamic Programming on Trees (Tree DP).	1
Dynamic Programming or Mathematical Optimization (Rule of 3s)	1
Dynamic Programming with a Fenwick Tree or Segment Tree to find the maximum shipments ending at index `i`.	1
Dynamic Programming with Bitmask	2
Dynamic Programming with Bitmask (State Compression).	1
Dynamic Programming with Bitmask.	1
Dynamic Programming with Bitmasking	1
Dynamic Programming with Bitmasking (State Compression)	1
Dynamic Programming with Bitmasking / Dijkstra.	1
Dynamic Programming with Counting (Combinatorics). Group elements by their remainder when divided by the "beauty" difference K, then solve as a "House Robber" variant.	1
Dynamic Programming with Frequency Maps or Combinatorics. Calculate combinations of each element frequency F independently and sum them.	1
Dynamic Programming with Mathematical Contribution (Prefix Arrays).	1
Dynamic Programming with Memoization	1
Dynamic Programming with State (Changed vs. Unchanged).	1
Dynamic SQL / Information Schema	1
Early Exit Iterator	1
Early Short-Circuit Simulation. If any digit is '0', the product becomes 0 instantly (cannot divide by zero), so return false immediately.	1
Early-Exit Linear Comparison	1
Edge-Centric + Top 3 Neighbors	1
Enumeration by Zero-Count (O(N√N))	1
Euclidean Algorithm (Recursive/Iterative)	1
Euclidean Algorithm for GCD. `gcd(a, b)` where you repeatedly take the remainder until one reaches zero.	1
Event-Driven Simulation using 4 Priority Queues (Heaps).	1
Execution Context Overhead (Visualizing the engine creating and destroying complex scopes for every iteration).	1
Exhaustive Heatmap Search (Brute Force)	1
Expand Around Center	1
Expand Around Center + Difference Count.	1
Expand Around Center.	1
Exponential Expansion + Binary Search	1
Exponential State Memory (Visualizing the storage of all valid subarray combinations).	1
Exponential State Search (Visualizing all possible combinations of powers being summed).	1
Exponential State Tracking (Visualizing memory holding all subsets simultaneously or deep recursion tree).	1
Exponential String Allocation (Visualize the heap filling with modified versions of the string).	1
Extreme Frequency Identification. O(N)	1
Factorial Number System (Math)	1
Fast & Slow Pointers (One Pass)	1
Fast and Slow Pointers	1
Fast and Slow Pointers (Tortoise and Hare)	1
Fast Fourier Transform (FFT) or Karatsuba.	1
Fast/Slow Pointers + Reverse Linked List.	1
Fenwick Tree (Binary Indexed Tree)	1
Fenwick Tree (BIT) / Index Mapping	1
Fenwick Tree (BIT) / Segment Tree	1
Fenwick Tree (BIT) / Segment Tree + Coordinate Compression	1
Filtered Inner Join Pipeline	1
Filtered Select with Null Handling.	1
Find Dip, Swap, Reverse	1

Find Middle + Reverse Second Half + Interweave.	1
Finding Bound Indices (Last 1, First 3) + Linear locked-state count. O(N)	1
First-Occurrence String Search / Math	1
First/Last Index Pre-computation	1
First/Last Occurrence Hash Map.	1
Fisher-Yates Shuffle Algorithm	1
Fixed Size Sliding Window	1
Fixed Sliding Window (No Set).	1
Fixed Sliding Window + Frequency Map	1
Fixed Sliding Window + Persistent Hash Map (or Array Cache) for O(1) incremental score updates.	1
Fixed Sliding Window on a Concatenated Array.	1
Fixed-Size Max-Heap	1
Fixed-Size Sliding Window	1
Fixed-Space Iteration (100..998) + Frequency Map Tally. O(N + 450)	1
Fixed-Window Sliding (Multiple Passes)	1
Floating Closure Memory (The callback and its lexical environment stay in memory until the timer naturally finishes).	1
Flood Fill (DFS/BFS) + Edge Trimming	1
Floyd-Warshall (All-Pairs Shortest Path)	2
Floyd-Warshall (Precomputation) or Weighted Disjoint Set Union (DSU).	1
Floyd-Warshall (Transitive Closure)	1
Floyd-Warshall Algorithm (All-Pairs Shortest Path)	1
Floyd’s Cycle Finding (Tortoise and Hare)	1
Floyd’s Cycle-Finding Algorithm (Fast & Slow Pointers)	1
Floyd’s Cycle-Finding Algorithm (Tortoise and Hare).	1
Fractional Conversion (Math/String)	1
Frequency Array (Character Tally)	1
Frequency Array (Counting Sort Pattern)	1
Frequency Array (Counting Sort).	1
Frequency Array (Hash Buckets)	1
Frequency Array (Hash Map Pattern)	1
Frequency Array + Greedy Consolidation.	1
Frequency Array + Prefix Sums + Math (Harmonic Series block processing).	1
Frequency Array + Triple Loop (Values 1-100).	1
Frequency Array / Counting (Rows are Sorted)	1
Frequency Array / Hash Map	1
Frequency Array / Hashing (Long Int)	1
Frequency Array / Hashing (Modulo 24)	1
Frequency Array / Math (Sum & Squares)	1
Frequency Array / Matrix Exponentiation	1
Frequency Bucket Counting	1
Frequency Counter (Fixed-size Array)	1
Frequency Counting (Greedy)	1
Frequency Counting (Hash Map)	1
Frequency Counting (Hash Map/Array)	1
Frequency Counting (Odd Count Limit)	1
Frequency Counting (Since arrays are sorted & distinct)	1
Frequency Counting (Y vs. Non-Y)	1
Frequency Counting + Combinatorics	1
Frequency Counting + Greedy Math	1
Frequency Counting + Iterative Value Check	1
Frequency Counting + Last Occurrence Mapping	1
Frequency Counting + Permutation Invariance.	1
Frequency Counting + Top 2 Candidates.	1
Frequency Counting Map / Parity Math	1
Frequency Counting of Top 2 Elements (Even/Odd).	1
Frequency Dominance / Median Comparison	1

Frequency Hash Map (Value -> List of Indices).	1
Frequency Hash Map + Greedy Matching.	1
Frequency Hash Map + Sum Equation	1
Frequency Hash Map or Bitmasking on an Array	1
Frequency Hash Map or Sorted Two-Pointers.	1
Frequency Hash Map.	2
Frequency Intersection (Min-Tally)	1
Frequency Map (Counting Sort Style)	1
Frequency Map (Hash Table)	1
Frequency Map + Binary Search or Sliding Window.	1
Frequency Map + Divisibility Check	1
Frequency Map + Greedy Construction	1
Frequency Map + Greedy Sort	1
Frequency Map + Greedy Sorting	1
Frequency Map + Map of Stacks (Grouped by Freq).	1
Frequency Map + Multiples / Divisors	1
Frequency Map + Sorted Greedy	1
Frequency Map + Sorted Unique Elements.	1
Frequency Map / Bitset (Single Pass)	1
Frequency Map / Bottleneck Analysis	1
Frequency Map / Bucket Sort	1
Frequency Map / Counting Sort	1
Frequency Map / Hash Table	2
Frequency Map / Index Array	1
Frequency Map Compression (Maximum Requirements)	1
Frequency Map or Sort + Two Pointers	1
Frequency Map over Range (Visualizing a large array tracking every possible integer value).	1
Frequency Map Ratio	1
Frequency Mapping (Coordinate Transformation)	1
Frequency Mapping + Greedy Pairing + Min-Cost Observation.	1
Frequency Mapping by Index Parity	1
Frequency Maps + Character Set Comparison	1
Frequency Maps / Discrepancy Counting. (Compare counts of what you have vs. what you need).	1
Frequency of Frequencies (Count Map)	1
Frequency Sorting + Sliding Threshold	1
Frequency Tracking with Contribution Counting. For each element, mathematically calculate how many subarrays it serves as the mode.	1
Frequency-Based Math.	1
Frobenius Coin Problem (Math)	1
FULL OUTER JOIN (or UNION of LEFT JOINs)	1
Function with Local Variable Offset (N-1) and LIMIT .	1
Function.prototype.apply / Symbol Key	1
Functional Graph Decomposition (Cycles & Tails)	1
Game Theory (Minimax) + 3D Memoization (Topological Sort from End States).	1
Game Theory (Sprague-Grundy/Nim) & Modulo Math Pattern	1
Gaps and Islands (Row Number Difference)	2
Gaps and Islands (SQL Window Functions) / Sequence Hashing.	1
Gauss' Sum Formula OR XOR Gates	1
GCD + Factor Counting	1
GCD + Sorting	1
Generators (`yield`)	1
Geometry / Gradient Descent (or Weiszfeld's)	1
Geometry Math (Column-based Calculation).	1
Global Bounding Box Scan	1
Global State Buffer (Persistent Pointer/Counter).	1
Global Sum Maintenance (Delta Tracking)	1
Graph (Functional Graph / Cycle Detection + Topological Sort).	1
Graph (Prim's or Kruskal's MST)	1

Graph + BFS/DFS or Disjoint Set Union (DSU) with Weights	1
Graph / DFS with Simultaneous Verification	1
Graph / Dijkstra's (Max-Heap)	1
Graph Adjacency List + Degree Finding + DFS/Linear Traversal.	1
Graph BFS / DFS with Visited Array	1
Graph Coloring (3-Color DFS) OR Kahn's Algorithm (Topological Sort on reversed edges).	1
Graph Coloring (BFS/DFS)	1
Graph Coloring (Bipartite Graph Check).	1
Graph Connectivity (BFS/DFS or Union-Find)	1
Graph connectivity (Union Find / BFS on Circles)	1
Graph Connectivity (Unique Source Tracking)	1
Graph Conversion (Parent Pointers) + BFS.	1
Graph Conversion + BFS	1
Graph Cycle Decomposition	1
Graph Cycles + Median Math	1
Graph Degree Analysis + BFS	1
Graph Degree Counting (Array-based Frequency Map).	1
Graph Degree Logic (Out-degree vs In-degree) / Slot Counting	1
Graph In-Degree Count + BFS/DFS	1
Graph In-Degree Counting / Row Sum Optimization	1
Graph Precomputation (Anchored Weights) / Component Ratios.	1
Graph Reachability / SCC (Tarjan's)	1
Graph Theory (Articulation Points / 2-Day Rule)	1
Graph Theory (Degree counting). Odd-degree nodes must total 0, 2, or 4. Check specific adjacency rules.	1
Graph Theory (In-Degree Zero)	1
Graph Theory (Out-degree & Cycles)	1
Graph Theory (Topological Sort)	1
Graph Theory / Degree Sorting (Constructive).	1
Graph Traversal (BFS or DFS) with Visited Set.	1
Graph Traversal (BFS) / Collaborative Filtering with Inverted Index.	1
Graph Traversal (DFS/BFS) + Reachability	1
Graph Traversal (Directed Graph) + BFS/DFS.	1
Graph Tree Diameter (2-Pass BFS/DFS)	1
Gray Code Generation + Bitwise XOR	1
Gray Code Reversal / Bit Manipulation	1
Greatest Common Divisor (GCD)	1
Greatest Common Divisor (GCD) Filter	1
Greatest Common Divisor (GCD) of the displacements. Find the distance each element needs to move and find the GCD of all those distances.	1
Greedy "Flip-and-Carry" or Difference Array. Start from the left; if the current bit doesn't match the target, flip the remainder of the string.	1
Greedy "Largest to Back"	1
Greedy "Unbalanced" Counter	1
Greedy (Adjacent Pair Checking)	1
Greedy (Adjacent Zero-Segment Merging). O(N)	1
Greedy (Always pick two largest) or Math.	1
Greedy (BFS-Style Window)	1
Greedy (Bit Manipulation)	1
Greedy (Complementary Minimum Tracking). O(N)	1
Greedy (Count Even Numbers)	1
Greedy (Dual Sorting)	1
Greedy (Extreme Finding)	1
Greedy (Fill from Back)	1
Greedy (First Mismatch).	1
Greedy (Heavy-Letter Priority)	1
Greedy (Huffman Coding / Min-Heap)	1
Greedy (Independent Max Gaps)	1
Greedy (Interval Scheduling / Jump Game logic).	1
Greedy (Interval Sorting)	1

Greedy (Kruskal's) or Tree DP (if constrained).	1
Greedy (Last Occurrence Map)	1
Greedy (Math-based Calculation).	1
Greedy (Max Reach)	1
Greedy (Maximum Digit)	1
Greedy (Min-Heap / Sorting)	1
Greedy (Min/Max Balance Tracking).	1
Greedy (modulo matches) + DP with Bitmask / Memoization for remainders.	1
Greedy (Monotonic Suffix Max)	1
Greedy (One-Pass Peak-Valley Counting)	1
Greedy (Pairs Comparison)	1
Greedy (Priority Queue / BFS) or Binary Search + DFS.	1
Greedy (Right-to-Left Scan).	1
Greedy (Right-to-Left Traversal)	1
Greedy (Right-to-Left)	1
Greedy (Right-to-Left) + Math Division	1
Greedy (Rightward free, Leftward paid).	1
Greedy (Running Offset)	1
Greedy (Skipping Alternate Stones)	1
Greedy (Smallest Even Strategy).	1
Greedy (Sort + Boundary Cap)	1
Greedy (Sort and Gap-Check).	1
Greedy (Sort by Difference)	1
Greedy (Sort by End Time) Sort intervals by their end times. Always pick the one that ends earliest to leave maximum room for others.	1
Greedy (Sort by End Time).	1
Greedy (Sort by End) Sort balloons by their end coordinates. Shoot one arrow at the end of the first balloon; skip all that overlap.	1
Greedy (Sort Neighbors per Node)	1
Greedy (Sorting + Linear Grouping)	1
Greedy (Sorting + Prefix Sums)	1
Greedy (Sorting by Combined Value)	1
Greedy (Sorting by Fractional Part).	1
Greedy (Sorting by Opportunity Cost).	1
Greedy (Sorting)	1
Greedy (Sorting).	4
Greedy (Sum of Unique Positive Numbers). O(N)	1
Greedy (Target Candidates).	1
Greedy (Two Pointers / Multi-Pass).	1
Greedy (Two Pointers).	1
Greedy (Two Scenarios)	1
Greedy (Two Scenarios: Even vs Odd indices)	1
Greedy (Two-Candidate Check).	1
Greedy (Two-Pass Scanning).	1
Greedy (Work Backwards from Target).	1
Greedy + Arithmetic Progression	1
Greedy + Backtracking + Prime Factorization	1
Greedy + Bitmask (or HashSet)	1
Greedy + Case Analysis Categorize by length: <6 (Insert), 6-20 (Replace), >20 (Delete then Replace). Priority on deleting from repeating triplets.	1
Greedy + Coordinate Modulo Counting	1
Greedy + Counting / Bucket Sort	1
Greedy + Decay Offset	1
Greedy + Delta Sorting (or Simple DP)	1
Greedy + Fenwick Tree / Segment Tree	1
Greedy + Frequency Array (Counting Sort).	1
Greedy + Frequency Counting (Bucket Sort).	1
Greedy + Frequency Fill (Even/Odd Interleaving).	1
Greedy + Frequency Sorting / Hash Set	1
Greedy + Greedy with Binary Search (TreeSet)	1

Greedy + Max-Heap + Difference Array	1
Greedy + Min-Heap (Bounded Priority Queue)	1
Greedy + Min-Heap (Priority Queue)	2
Greedy + Monotonic Stack + Frequency Counting.	1
Greedy + Priority Queue (Min-Heap of negative values)	1
Greedy + Remainder DP	1
Greedy + Set Intersection (Filtering Friendships).	1
Greedy + Sorting	2
Greedy + Sorting (Two-Pointer logic)	1
Greedy + Sorting.	3
Greedy + Stack (Two Passes).	1
Greedy + Stack + Suffix Minimums	1
Greedy + Suffix Matching Precomputation	1
Greedy + Two Pointers (Rolling Hash)	1
Greedy / Array Math	1
Greedy / Bit Manipulation	1
Greedy / Bubble Sort Logic	1
Greedy / Bucketing by Remainder	1
Greedy / Case Analysis.	1
Greedy / Cycle Counting with Costs.	1
Greedy / Cycle Decomposition (DSU)	1
Greedy / Event Simulation (State tracking).	1
Greedy / Fixed Window Flip	1
Greedy / Frequency Counting	1
Greedy / Independent Sorting	1
Greedy / Last-Position Tracking.	1
Greedy / Linear Scan	2
Greedy / Majority Element	1
Greedy / Math (Reverse Operations)	1
Greedy / Math (Zero Counting).	1
Greedy / Math Accumulator	1
Greedy / Math Analysis	1
Greedy / Math Observation	1
Greedy / Math-based Cycle Counting	1
Greedy / One-Pass Simulation	1
Greedy / One-Pass State Machine	1
Greedy / Parity based on Depth	1
Greedy / Parity Counting	1
Greedy / Post-Order DP (State Signaling)	1
Greedy / Prefix Counters	1
Greedy / Preprocessing	1
Greedy / Relative Order Tracking	1
Greedy / Running Balance Counter	1
Greedy / Running GCD	1
Greedy / Set Intersection	1
Greedy / Simulation (Peak Tracking)	1
Greedy / Single-Source Shortest Path (Dijkstra) or Prefix Min.	1
Greedy / Sorting	2
Greedy / Sorting Mirror Elements	1
Greedy / Stack Logic (Counter)	1
Greedy / State-Flip Parity	1
Greedy / Two Pointers	1
Greedy / Two Pointers / Preprocessing	1
Greedy / Virtual Deletion	1
Greedy Absolute Displacement	1
Greedy Accumulation (Target Sum Matching).	1

Greedy Accumulation / Prefix Sum tracking.	1
Greedy Algorithm (Coin Change variation)	1
Greedy Algorithm (Pointer Jumping)	1
Greedy Algorithm (Price-per-Weight Ratio).	1
Greedy Algorithm (Rearrangement Inequality): Sort one ascending, one descending.	1
Greedy Algorithm (Summing Positive Slopes).	1
Greedy Algorithm + Sorting	1
Greedy Algorithm + Sorting.	1
Greedy Algorithm + Sorting. Sort the array. To maximize medians, pair the largest available, the *next* largest (the median), and the smallest available.	1
Greedy Algorithm / Single Pass Array Scan.	1
Greedy Algorithm / Sorting. Match the longest land rides with the shortest water rides (or vice versa, depending on exact problem constraints) to minimize the maximum bottleneck.	1
Greedy Algorithm / String Parsing with continuous accumulation.	1
Greedy Algorithm with a Max-Priority Queue. At any step, activate the item that provides the highest immediate gain/value based on current state.	1
Greedy algorithm with a Queue or Monotonic Stack, focusing on the nearest available character that satisfies the activation rule.	1
Greedy Alternation	1
Greedy Analysis of Parity. An alternating sum is (EvenSum - OddSum) . Swapping only helps if you move a large number from an "Odd" (negative) position to an "Even" (positive) position.	1
Greedy AND-Reset Logic	1
Greedy Array Compression	1
Greedy Array Processing (Jump Game II Pattern)	1
Greedy Assignment + Frequency Sorting	1
Greedy Assignment + Hash Map of Max-Heaps + Special Worker Greedy choice. O((N+M) log M)	1
Greedy Billing (Counter-based).	1
Greedy Bit Alignment	1
Greedy Bit Construction	1
Greedy Bit Manipulation (Low-bit Analysis).	1
Greedy Bit Manipulation (Trailing Zeros)	1
Greedy Bit Manipulation + Min-Priority Queue (or Frequency Array)	1
Greedy Bit-by-Bit Construction (from MSB to LSB). Check if a partition can satisfy the current bit mask using a linear scan.	1
Greedy Bit-Construction + Trie.	1
Greedy Bitwise Construction (MSB to LSB).	1
Greedy Bitwise Masking (MSB to LSB)	1
Greedy Block Counting (Run-Length)	1
Greedy Boundary Detection (Top-Left to Bottom-Right)	1
Greedy Candidate Maintenance (Two-Variable Sweep)	1
Greedy Case-Based Logic.	2
Greedy Chunking (Math)	1
Greedy Coloring.	1
Greedy Column-wise Scan	1
Greedy Comparison (2 Target States).	1
Greedy Component Rounding	1
Greedy Constraint Matching	1
Greedy Constructive Construction.	1
Greedy Coordinate Accumulation (Math)	1
Greedy Counting (Compare with two possible targets: "0101..." and "1010...").	1
Greedy Counting + Sorting lengths	1
Greedy Counting / Cumulative Sum	1
Greedy Counting / Frequency Analysis	1
Greedy Degree-Based Assignment	1
Greedy Difference Tracking + Parity Check.	1
Greedy Digit Allocation	1
Greedy Digit Manipulation	1
Greedy Digit Substitution	1
Greedy Distance Calculation (Total Distance - Max Saving).	1
Greedy DP + Prefix Sums + Monotonic Queue	1
Greedy Elimination (Two Pointers)	1
Greedy Extremes Check	1
Greedy Factorization (Divisors 9 down to 2).	1

Greedy Filtering (One Pass).	1
Greedy First-Digit Replacement.	1
Greedy Frequency Counting + Min-Heap	1
Greedy Frequency Distribution. Sort elements by frequency. Place the most frequent elements into different groups first.	1
Greedy Frequency Sorting	1
Greedy Frequency Sorting + Combinatorics	1
Greedy Game State Simulation (Two Pointers)	1
Greedy Gap Filling / Pigeonhole Principle	1
Greedy Grouping (HashMap)	1
Greedy Half-Scan Mutation	1
Greedy Hash Map / Frequency Array	1
Greedy Interval Scheduling (Earliest Deadline First).	1
Greedy Interval Scheduling (End-Point Prioritization).	1
Greedy Left-to-Right / Math	1
Greedy Left-to-Right Minimization / Two Pointers	1
Greedy Linear Scan	1
Greedy Local Correction.	1
Greedy Math (Arithmetic Progression)	1
Greedy Math (Integer Break logic) + Fast Modular Exponentiation.	1
Greedy Math (Single Pass).	1
Greedy Math + Case Analysis	1
Greedy Math / Frequency Map	1
Greedy Math / Max Priority Queue	1
Greedy Math / Modulo Arithmetic	1
Greedy Math / Total Sum	1
Greedy Mathematical Balancing	1
Greedy Matrix Allocation	1
Greedy Min/Max Tracking.	1
Greedy Modular Distance	1
Greedy One-Pass	1
Greedy One-Pass (The Total Net Gain Rule).	1
Greedy One-Pass Comparison	1
Greedy Overlap Selection.	1
Greedy Pairing + Modular Exponentiation	1
Greedy Palindrome Generation Start from the largest possible palindrome (e.g., 999999) and work downwards. For each, check if it has two n-digit factors.	1
Greedy Parity Construction	1
Greedy Parity Property	1
Greedy Peak Finding (Single Element Identifier).	1
Greedy Pointer / Linear Jump	1
Greedy Pointer Matching.	1
Greedy Positional Logic.	1
Greedy Positioning (Two-Target Comparison).	1
Greedy Pre-reversal + Optimized Split Scan.	1
Greedy Prefix Matching	1
Greedy Prefix Maximum Tracking (O(N x M)).	1
Greedy Preorder Simulation	1
Greedy Prime Factorization (Reverse Digits)	1
Greedy Property (Row Head + Column Max).	1
Greedy Quadrant Invariance. O(N)	1
Greedy Range Expansion	1
Greedy Range Extension (Patching)	1
Greedy Range Removal (Set/Segment Tree)	1
Greedy Reverse Engineering (Work Backwards)	1
Greedy Reverse Matching (Work Backwards)	1
Greedy Selection (Math/Sorting)	1
Greedy Selection + Boolean Array/Set	1
Greedy Selection + Difference Array / Sorting.	1

Greedy Selection + Hash Set Lookup.	1
Greedy Selection + Hash Sets	1
Greedy Selection + Smallest-Odd/Even Swap	1
Greedy Selection on Sorted Array. O(N log N)	1
Greedy Selection with Math Constraints	1
Greedy Set Partitioning	1
Greedy Simulation	1
Greedy Simulation (Two Cases)	1
Greedy Simulation (While Loop)	1
Greedy Simulation with BFS/DFS	1
Greedy Single Pass / State Machine	1
Greedy Single Pass with Hash Set.	1
Greedy Single-Pass / Max-Tracker.	1
Greedy Sliding Window	1
Greedy Sorting (Descending Order).	1
Greedy Sorting (Largest Cost First)	1
Greedy Sorting (Smith's Rule)	1
Greedy Sorting + Last Digit Swap	1
Greedy Sorting + Stack-based Substitution	1
Greedy Sorting / Hash Map counting. Pair cards based on their first and second letters to maximize your "counter-plays".	1
Greedy Sorting / Two Pointers	1
Greedy Sorting Alignment	1
Greedy Sorting by "Time-to-Arrival"	1
Greedy Sorting.	1
Greedy Sorting. Sort the array descending and pick the largest values until you hit the K distinct limit.	1
Greedy Stack (Simulation).	1
Greedy Strategy	1
Greedy Strategy (Tian Ji's Horse Racing).	1
Greedy Strategy + Sorting.	1
Greedy Strategy: Earliest Deadline First (EDF) or Shortest Processing Time (SPT).	1
Greedy String Matching (Split by wildcard, search Prefix then Suffix). (O(N))	1
Greedy String Mutation (Right-to-Left)	1
Greedy Subtraction	1
Greedy Subtraction (Coin Change)	1
Greedy Sum Matching	1
Greedy Summation / Math	1
Greedy Swapping / Parity-Based Reordering. O(N)	1
Greedy Tally + Remainder Math	1
Greedy Template Filling + Contradiction Check (KMP/Suffix). O(n · m)	1
Greedy Time Calculation (Hash Map)	1
Greedy Traversal (Right-to-Left) + Smallest Prime Factor (SPF)	1
Greedy Traversal / Peak Finder	1
Greedy Two-Pointer Placement	1
Greedy Virtual Deletion (Two Pointers)	1
Greedy with Difference Array / Sliding Window	1
Greedy with Frequency & Vacancy Maps.	1
Greedy with Frequency Maps or Dynamic Programming. Count the excess of each item type and discard only the necessary surplus.	1
Greedy with Max-Heap (Priority Queue).	1
Greedy with Max-Priority Queue.	1
Greedy with Min-Heap (Priority Queue).	1
Greedy with Modular Arithmetic	1
Greedy with Priority Queue (Max-Heap)	1
Greedy with Set Cover or Bitmask DP (if dimensions are small). Focus on covering the "bottleneck" (uncovered cells with the fewest neighbors).	1
Greedy with Sortedness Tracking	1
Greedy with Suffix Comparison.	1
Greedy Word Packing + Round Robin Spacing	1

Greedy Zero-ing (Place Value Increment)	1
Grid Expansion + DFS/BFS (or Union-Find)	1
Grid Partitioning + Bounding Box	1
Group By + Aggregate COUNT(DISTINCT)	1
Group By + Aggregate Ratio + Order By.	1
GROUP BY + Aggregation (SUM)	1
Group By + CASE WHEN + ROUND	1
GROUP BY + COUNT	1
GROUP BY + COUNT(DISTINCT)	2
GROUP BY + COUNT(DISTINCT) + AVG	1
GROUP BY + DATE_FORMAT	1
GROUP BY + DATEDIFF / BETWEEN	1
GROUP BY + GROUP_CONCAT / STRING_AGG	1
Group By + Having (Conditional Sums).	1
Group By + Having (Min/Max Dates).	1
Group By + Having (Set Membership)	1
Group By + Having (Subquery Max).	2
Group By + Having Clause.	2
Group By + Having Count (Distinct).	1
Group By + Having Count Clause	1
GROUP BY + HAVING COUNT()	1
GROUP BY + HAVING COUNT(DISTINCT)	1
Group By + Having.	1
GROUP BY + MAX() + EXTRACT/LIKE/YEAR().	1
Group By + Order By Count + Limit 1.	1
Group By + Scalar Subquery / Cross Join	1
GROUP BY + SUM(CASE WHEN)	1
Group By / Sort by ID & Date + Linear Scan. O(N log N)	1
Group by First Letter + Venn Diagram Intersection	1
Group by Length/Last Char + Trie for Longest Common Prefix (LCP).	1
GROUP BY with HAVING COUNT(*) > 1 .	1
Group By.	1
Group Consecutive Counts.	1
Group line segments by slope using a Hash Map. A trapezoid is formed by picking 2 segments with the same slope (bases) and ensuring they don't share points.	1
Grouping + `COUNT(DISTINCT)` with `HAVING`	1
Grouping + Sliding Window/Case Analysis	1
Grouping by Vowel Count + Reverse Pointers	1
Hamiltonian Path (Backtracking + Pruning)	1
Hash Aggregation / Index Join	1
Hash Anti-Join (LEFT JOIN + IS NULL)	1
Hash Join / Hash Map Filtering	1
Hash Map (Aggregate & Track Max)	1
Hash Map (Counter) + Custom Stable Sort.	1
Hash Map (Counting) + Hash Set (Uniqueness)	1
Hash Map (Dictionary) for O(1) Access.	1
Hash Map (Dictionary) for O(1) CRUD.	1
Hash Map (First Word Mapping)	1
Hash Map (First/Last/Count).	1
Hash Map (Frequency Count) / Single Pass	1
Hash Map (Frequency Counter) + Second Pass	1
Hash Map (Frequency Counter) with GCD reduction for fractions	1
Hash Map (Frequency Counting)	1
Hash Map (Frequency Counting) + Math Combinatorics.	1
Hash Map (Frequency Counting).	1
Hash Map (Frequency Map) + Two Sum variant for 22 static targets.	1
Hash Map (Frequency) + String Prefix/Suffix Splitting	1
Hash Map (ID to Rank) or Tuple Sorting. Store (Time, ID) pairs and sort them directly.	1

Hash Map (Inventory Counter).	1
Hash Map (Meet-in-the-Middle logic)	1
Hash Map (Meet-in-the-Middle) Calculate all A+B and store in a Map `sum: count`. Then iterate C+D and check for -(C+D) in the Map.	1
Hash Map (Object Literal) Accumulator	1
Hash Map (One Pass)	1
Hash Map (Parent Pointer) + Set Intersection	1
Hash Map (Piece-Start Mapping)	1
Hash Map (Remainder Frequency)	1
Hash Map (Sorted Key / Char Count)	1
Hash Map (Sparse Storage) + Direct Formula Parsing. O(1) set / O(L) get.	1
Hash Map (Stringified Rows)	1
Hash Map (Timestamp -> Price) + Two Heaps (or TreeMap)	1
Hash Map (User to Task-List) + Sorting/Filtering.	1
Hash Map (Val -> Node) + Set (Child Tracking).	1
Hash Map (Value rightarrow Frequency Count).	1
Hash Map + Balanced BST (or TreeMap/SortedSet)	1
Hash Map + Bitmask (Sparse Row Processing)	1
Hash Map + clearTimeout	1
Hash Map + Custom Comparator Sorting	1
Hash Map + DFS/BFS.	1
Hash Map + Digit Perturbation	1
Hash Map + Doubly Linked List (DLL).	1
Hash Map + Doubly Linked List of Buckets Map strings to nodes. Nodes are buckets of strings with the same count. Buckets are linked in order of count.	1
Hash Map + Frequency Counters	1
Hash Map + Grouping (Run-Length Encoding)	1
Hash Map + Linked List of DLLs Map keys to nodes. Have a second "Frequency Map" that points to a Doubly Linked List of all keys with that frequency.	1
Hash Map + Max-Heap (Priority Queue) with Lazy Deletion.	1
Hash Map + Max-Heap/SortedList with Lazy Deletion. (O(log N))	1
Hash Map + Max-Priority Queue (Lazy Removal)	1
Hash Map + Min-Heap (for ID reuse)	1
Hash Map + Min-Heap (size K).	1
Hash Map + Prefix Sum	1
Hash Map + Result Linked List Construction	1
Hash Map + Sieve of Eratosthenes.	1
Hash Map + Target XOR Decomposition	1
Hash Map + Word Split	1
Hash Map / Array Grouping by Max Digit	1
Hash Map / Array Substitution Cipher	1
Hash Map / Chunk Frequency	1
Hash Map / Combinatorics	1
Hash Map / Dictionary Counting	1
Hash Map / Fixed Array (First Occurrence)	1
Hash Map / Frequency Array	2
Hash Map / Frequency Array (Modulo Arithmetic).	1
Hash Map / Frequency Array Count char frequencies. Add all even counts. Add (odd-1) for odd counts. Add 1 if any odd exists.	1
Hash Map / Frequency Array.	1
Hash Map / Frequency Count (Modulo Arithmetic).	1
Hash Map / Frequency Counting	2
Hash Map / Frequency Indexing	1
Hash Map Adjacency List + BFS/DFS.	1
Hash Map Aggregation	1
Hash Map Aggregation / Group By (if SQL). Accumulate counts in a single pass, then compare against the precomputed average.	1
Hash Map Aggregation / Sorting by User	1
Hash Map Aggregation / SQL Group By with HAVING clause. Compute statistics for all books in one pass.	1
Hash Map Aggregation / SQL-style GroupBy.	1
Hash Map Binding / Array of Objects + Sort	1
Hash Map Count + Sentinel Node.	1

Hash Map Counting (Frequency of Arrays)	1
Hash Map Counting + Single-Pass Max Tracking.	1
Hash Map for Index Lookup.	1
Hash Map for List A (Value -> Index). Scan List B and look up values in the Map in O(1).	1
Hash Map for lookups + Single Pass StringBuilder.	1
Hash Map for O(1) inorder index lookup + Recursive DFS (Process postorder backwards: Root, Right, Left).	1
Hash Map for O(1) inorder index lookup + Recursive DFS.	1
Hash Map for Row Patterns.	1
Hash Map for Slopes (per starting point).	1
Hash Map Frequencies + Greedy Math Downward Search	1
Hash Map Frequencies + Mathematical Chaining	1
Hash Map Frequency + Sorting	1
Hash Map Frequency Counting	1
Hash Map Frequency Tracking.	1
Hash Map grouping (Content -> List of Paths).	1
Hash Map Grouping + Circular Maximum Gap Calculation	1
Hash Map Grouping + Single Pass Validation.	1
Hash Map Grouping + Sort & Sweep.	1
Hash Map Grouping + Sorting	1
Hash Map Grouping by `Row + Col`	1
Hash Map Indexing	1
Hash Map mapping User ID to a Hash Set of minutes.	1
Hash Map of Bitmasks. For each unique bitmask, store the maximum integer found. Then compare all pairs of bitmasks.	1
Hash Map of Distances For each point i, calculate distances to all other points j. Store counts in a Map `dist: count`. Result is sum count x (count - 1).	1
Hash Map of Edge Frequencies.	1
Hash Map of Hash Maps & Tree Sets	1
Hash Map of Hash Sets. O(N)	1
Hash Map of Indices + Sliding Window	1
Hash Map of Lists (Greedy Accumulation)	1
Hash Map of Lists (Indexed Access)	1
Hash Map of Lists + Sorting + Sliding Window (Size 3)	1
Hash Map of Max Values	1
Hash Map of Sets (Callback Registry)	1
Hash Map of Sorted Indices + Binary Search	1
Hash Map of Sorted Sets (TreeMap)	1
Hash Map Point Frequencies + Diagonal Anchoring	1
Hash Map Pre-processing	1
Hash Map Pre-processing + Two Pointers	1
Hash Map Remapping (Virtual Array)	1
Hash Map Root Lookup + DFS Validation	1
Hash Map State Tracking + DFS Pruning	1
Hash Map Traversal / Parent-Child Mapping.	1
Hash Map with Canonical Form Key	1
Hash Map with Expiration Timestamps	1
Hash Map with Index Lists / Sliding Window of Size 3	1
Hash Map with Math Ratio (GCD) and Middle-element Fixing (O(N^2)).	1
Hash Map with Random Base62 Encoding.	1
Hash Map with Size-3 Deque (Sliding Window of Indices)	1
Hash Map with Suffix Tracking	1
Hash Map with Tuple Keys / Bitsets	1
Hash Set (Backward Scan)	1
Hash Set (Coordinate Encoding).	1
Hash Set (Impossible Value Filtering).	1
Hash Set (Intelligent Starting Points).	1
Hash Set (Seen Filter)	1
Hash Set (Single Pass)	2
Hash Set (Target Difference math).	1

Hash Set + Extremity Midpoint	1
Hash Set + Greedy Growth	1
Hash Set + Min/Max Math (No Sorting)	1
Hash Set + Pre-processing.	1
Hash Set + Single BFS/DFS	1
Hash Set + While Loop	1
Hash Set / Bitmask + Math.min	1
Hash Set / Bitmasking	1
Hash Set / Boolean Array	1
Hash Set / Boolean Array (Bitmask)	1
Hash Set / Boolean Array Lookup	1
Hash Set / Boolean Arrays	1
Hash Set / Boolean Lookahead	1
Hash Set / Direct Indexing	1
Hash Set / Frequency Counting	1
Hash Set / Sorting + Binary Search.	1
Hash Set / Two Pointers (after Sort)	1
Hash Set Aggregation. For each customer, collect `transaction_month` into a Set. Filter customers where `Set.size == 12`.	1
Hash Set for O(1) word lookup + Custom Comparator Sorting (or Min-Heap for top K selection).	1
Hash Set Intersection (Difference Checking)	1
Hash Set Lookup	1
Hash Set Lookup + Single Pass Traversal.	1
Hash Set Lookup / Single Pass Tokenization.	1
Hash Set Lookups	1
Hash Set of Pair Strings	1
Hash Set of Visited Points	1
Hash Set or Boolean Array Check	1
Hash Set or Boolean Frequency Array.	1
Hash Set Validation (or Boolean Array Tracking).	1
Hash Set.	1
Hash Sets / Set Difference	1
Hash Table + Alphabetical Sorting	1
Hash Table / Array Mapping (Single Pass)	1
Hashing / Rolling Hash	1
HashMap + Dynamic Array (Swap with Last)	1
HashMap + Sorted List (Binary Search)	1
HashMap of Sets + Dynamic Array	1
HashMaps for Line Counting	1
HashSet + Dummy Node 1-Pass	1
HashSet Lookup	1
Havel-Hakimi Algorithm. Sort the degree sequence, then repeatedly "consume" the highest degree by connecting it to the next highest degrees.	1
Head Re-linking (Two-Pointer Shift)	1
Heap Allocation Churn (Visualizing the heap filling with intermediate string fragments: "a", "aa", "aaa"...).	1
Heap Allocation Overload (Visualize multiple intermediate string objects piling up in memory due to immutable string slicing).	1
Heap Array Allocation (Visualizing thousands of string objects being stored simultaneously).	1
Heap Fragment Memory (Visualizing thousands of temporary string objects created during slicing).	1
Heap memory allocation map (Drawing separate object allocations in memory for every closure context created).	1
Hierholzer's Algorithm (Eulerian Path via DFS)	1
Hierholzer’s Algorithm (Eulerian Path)	2
Higher-Order Function (Factory)	1
History Tracking (Binary Search on Versions)	1
Horizontal Shrinking	1
Horner's Method for Polynomial Evaluation. Process the string character by character, multiplying the current result by the base and adding the new digit.	1
Implicit Treap (Randomized Binary Search Tree) / Splay Tree with Lazy Propagation.	1
IN Clause with Multi-Column Key	1
In-Degree Accumulator (Array)	1
In-order DFS + Two-Pointer Merge	1

In-Order DFS Traversal with Level Grouping.	1
In-Order Flattening + Midpoint Reconstruction	1
In-Order Traversal (natively sorted) + Track `prev` Node.	1
In-Order Traversal (Sorted Property)	1
In-order Traversal + Binary Search (Sorted Array)	1
In-order Traversal with In-place Re-linking.	1
In-Place "Leapfrog" Reversal	1
In-place array mutations (Unnecessary write operations to memory).	1
In-place Array Mutations (Visualizing the CPU repeatedly writing to memory for every range update).	1
In-place Array Transformation.	1
In-Place Encoding (Math / Modulo)	1
In-Place Extraction + Sorting.	1
In-Place Group Iteration & Re-linking	1
In-Place In-order Maintain a `first` and `last` pointer. Connect `last.right = current` and `current.left = last` during traversal.	1
In-Place Iterative Reduction	1
In-Place Link Manipulation	1
In-place Marking For each number x, go to index |x|-1 and flip the value to negative. If already negative, it's a duplicate.	1
In-place Matrix Rotation (Transpose + Reverse) or Index Mapping.	1
In-place modification of Character Array.	1
In-place Negation For each value x, treat |x|-1 as an index and make the value at that index negative. Positives remaining are the gaps.	1
In-Place Pointer Reversal	1
In-Place Pointer Splicing + Euclidean Algorithm	1
In-place row swapping within the specified submatrix boundaries.	1
In-Place Simulation (Two Pointers/Copying)	1
In-Place State Encoding (Bit Manipulation)	1
In-place State Flagging & Two-Pointer Drop	1
In-Place State Marking	1
In-Place Two Pointers / Link Manipulation.	1
In-Place Two-Pointer Swap	1
Inclusion-Exclusion (Math)	1
Inclusion-Exclusion Principle (Area1 + Area2 - Overlap)	1
Inclusion-Exclusion Principle (Math)	1
Inclusion-Exclusion Principle with Modular Exponentiation	1
Inclusion-Exclusion Principle. (Total Trapezoids = All shapes with at least 1 pair of parallel sides MINUS Parallelograms which have 2 pairs).	1
Incremental Accumulation + Upper Bound Pruning (O(N^2)).	1
Independent Precomputed BFS	1
Independent Row/Column Counting	1
Independent Subqueries / Common Table Expressions (CTE)	1
Index Mapping + Linear Scan.	1
Index-as-Hash / Negation Marker.	1
Information Theory (N-Dimensional Hypercube) Each pig represents a dimension. The number of states a pig can have is (Tests + 1). Solve (States)^{Pigs} ge Buckets.	1
Inner Join or Group By with IN clause.	1
INNER JOIN with GROUP BY and HAVING	1
INNER JOIN with Inequality Filtering	1
Inner Join.	1
Inorder Simulation (DFS). Calculate list length. Build left subtree, assign current list node as root, advance list pointer, build right subtree.	1
Inorder Traversal (Morris Traversal) with State Tracking variables.	1
Inorder Traversal to Hash Set (or Two Pointers on Arrays)	1
Insertion Sort with Dummy Head and Predecessor Pointers.	1
Integer Modulo Sum + Frequency Bucket	1
Integer State Mutation (Visualizing the variables main and additional changing over time).	1
Interactive Bit Manipulation	1
Intermediate Array Allocation (Visualize a potentially massive array of strings being created in memory just for a boolean check).	1
Intermediate Array Reallocations (Visualizing the heap creating new arrays for every merge pass).	1
Intermediate Subarray Slices (Visualizing the heap filling with transient array segments).	1
Interval Consolidation + Greedy Activity Selection. O(N)	1

Interval DP (Matrix Chain Multiplication) + HashSet	1
Interval DP (Top-Down Memoization)	1
Interval DP + Partition DP	1
Interval DP `dp[i][j]` stores the shortest encoded string for `s[i...j]`. Check for patterns using `(s+s).indexOf(s, 1)`.	1
Interval Dynamic Programming	1
Interval Dynamic Programming (Minimax)	1
Interval Expansion + Greedy Search	1
Interval Merging	1
Interval Merging / Prefix Sums	1
Interval Merging / Segment Tree (TreeMap)	1
Interval Sorting + Monotonic Scan	1
Interweaving Nodes (In-place).	1
Invariance & Hamming Distance	1
Inverse Sliding Window (Two Pointers) seeking the maximum middle subarray to *leave behind*.	1
Inverse Transform (Polar) Generate theta in [0, 2π] and r = R √{rand()}. The square root is crucial for uniform density.	1
Inverted Index (Hash Map)	1
Inverted Index + Bitsets / Matrix Multiplication.	1
Iterative Accumulator Update	1
Iterative Bottom-Up Dynamic Programming (State variables).	1
Iterative Brute Force (Optimized Range)	1
Iterative Cleanup + Block Processing	1
Iterative Comparison	1
Iterative Digit Extraction (Modulo & Stack).	1
Iterative Division (Prime Squeezing)	1
Iterative DP (Fibonacci)	1
Iterative DP / Simulation	1
Iterative Dynamic Programming (1D).	1
Iterative Dynamic Programming (Bottom-Up). `dp[i] = dp[i-1] + dp[i-2] + dp[i-3]`.	1
Iterative Dynamic Programming (Substring Check)	1
Iterative GCD + Greedy Expansion	1
Iterative Grid Simulation.	1
Iterative In-order Traversal with Early Stop	1
Iterative In-Place Reversal (Three Pointers: prev, curr, next)	1
Iterative Mapping (New Array)	1
Iterative Mirroring / Bitwise Formula	1
Iterative Modulo (%) and Division (/)	1
Iterative Modulo and Division.	1
Iterative New Array Push	1
Iterative Pointer Manipulation (Bottom-Up Logic).	1
Iterative Pointer Mutation (Visualizing the CPU cycle overhead for millions of small increments).	1
Iterative Simulation with Window Functions.	1
Iterative Slicing (Step = Size)	1
Iterative Stack (Depth Tracking).	1
Iterative Stack / Pointer Weaving When you hit a `child`, "insert" the child branch between `current` and `current.next`.	1
Iterative StringBuilder / Two-Pointer	1
Iterative Traversal using Two Stacks (or One Stack + Reverse).	1
Iterative Traversal with a Stack.	1
Iterative with Stack (or Morris)	1
JavaScript Generators & Promises	1
JavaScript Prototype Chain Traversal	1
JavaScript Proxy (`get` trap)	1
JOIN + Aggregation (HAVING Count Match)	1
JOIN + Conditional Aggregation	1
JOIN + Distinct Filtering	1
Join + Group By (Install Date Logic).	1
Join + Group By + AVG.	1
JOIN + GROUP BY + CASE/IF logic	1

JOIN + GROUP BY + HAVING	2
Join + Group By + Limit.	1
Join + Group By with Filtered Aggregation	1
JOIN + SUM Aggregation	1
JOIN + Window Function (RANK)	1
Join Filtering + Order By Clause	1
Join with a GROUP BY subquery or Window Function.	1
K-way Merge (Dynamic Programming with Multiple Pointers)	1
Kadane's (Min/Max Complement)	1
Kadane's + Total Sum Logic	1
Kadane's Algorithm	1
Kadane's Algorithm on Difference Array	1
Kadane's Algorithm or Sliding Window with Prefix Sums. If Value = Sum, it's strictly Kadane's.	1
Kadane’s Algorithm (Greedy/DP)	1
Kadane’s Algorithm Variation	1
Kahn's Algorithm (Topological Sort) + Multi-source BFS	1
Key Normalization + Frequency Map	1
KMP (LPS Array) + Three-Pointer Greedy Slide. O(N + M)	1
KMP (String Matching) + Matrix Exponentiation (DP)	1
KMP / String Matching (or O(N^2) if constraints allow)	1
KMP / Z-Algorithm on Trend Array	1
KMP Algorithm (LPS Array)	1
KMP Algorithm (Prefix Function)	1
KMP or Optimized Built-in Search	1
KMP Partial Match Table (LPS Array)	1
KMP Pattern Matching on Tree	1
KMP Prefix Function (LPS Array)	1
KMP Prefix Function (LPS) + Monotonic Two Pointers	1
KMP String Matching + Binary Search / Two Pointers	1
KMP with Stream State	1
Knuth-Morris-Pratt (KMP) Automaton	1
Kruskal's / Prim's with Virtual Root	1
Kruskal's Algorithm (Greedy + DSU)	1
Kruskal's Algorithm with Disjoint Set Union (DSU) or Tarjan's Bridge Finding Algorithm.	1
Kruskal's MST + LCA (Lowest Common Ancestor) with Binary Lifting.	1
Kruskal’s + Edge Exclusion/Inclusion	1
Language-native sort with a Custom Key/Comparator.	1
Large Intermediate Join (Visualizing the DB loading every single booking record into memory before counting).	1
Largest Suffix Algorithm (Two Pointers) + Substring Truncation. (O(N))	1
Lazy Evaluation / Two Pointers / State Machine.	1
Lazy Flipping (Flag + Two Arrays).	1
Lazy Iteration + Stack/List Cache	1
Lazy Iterator with Stack (Depth-First Search)	1
Lazy Propagation (Difference Array Logic)	1
LCA (Lowest Common Ancestor) + Binary Lifting + Prefix Bitmasks.	1
LCA (Lowest Common Ancestor) + Binary Lifting + Prefix Frequencies	1
LCA (Lowest Common Ancestor) + Path Comparison	1
LCA + Distance Formula	1
LCP Array (Longest Common Prefix) / Z-Algorithm	1
LCRS (Recursive Pointer Re-assignment) Use a recursive DFS to transform pointers without creating new nodes.	1
LEFT JOIN (Hash Join)	1
LEFT JOIN + Aggregation	1
Left Join + CASE Statement + Aggregate Functions	1
LEFT JOIN + COALESCE (Conditional Logic)	1
LEFT JOIN + COALESCE / IFNULL	1
LEFT JOIN + Conditional Aggregate	1
Left Join + Filtering + Having.	1

Left Join + Group By (SQL)	1
LEFT JOIN + GROUP BY + COALESCE	1
LEFT JOIN + GROUP BY + DISTINCT	1
LEFT JOIN + GROUP BY + IFNULL/COALESCE	1
Left Join + Group By.	1
LEFT JOIN + NULL Filtering	1
LEFT JOIN / WHERE IS NULL	1
LEFT JOIN on Interval + GROUP BY	1
LEFT JOIN where Orders.Id IS NULL (or NOT IN with subquery).	1
LEFT JOIN with COALESCE / IFNULL	1
LEFT JOIN with COALESCE / IFNULL.	1
LEFT JOIN with Conditional Aggregation	1
LEFT JOIN with NULL check / EXCEPT operator	1
Left Join with Null/Value Filtering.	1
LEFT JOIN with time range condition + IS NULL check.	1
Length Counting + Mathematical Chunking	1
Length Sort + KMP (or Suffix Tree)	1
Level Order Traversal (BFS) + Sorting/Min-Heap.	1
Level-by-Level Linked List Traversal (using existing `next` pointers to traverse horizontally).	1
Level-Order BFS (Null Detection)	1
Level-Order BFS (Same Level, Different Parent)	1
Level-Order BFS + Frequency Map Sort	1
Lexical Scope / Closures	1
Line Sweep + Segment Tree (or Coordinate Compression).	1
Line Sweep / Prefix Sums / 2D Difference Arrays.	1
Line Sweep Algorithm (Prefix Sum over time) OR Min-Heap	1
Line Sweep Algorithm + DP with Segment Tree.	1
Line Sweep Algorithm + TreeMap (Difference Array)	1
Line Sweep Algorithm + TreeMap (Event Sorting)	1
Line Sweep Algorithm / Difference Array	1
Linear Algebra (System of Equations)	1
Linear Array Scan with Modulo Arithmetic (comparing absolute distance vs circular wrap-around distance).	1
Linear Array Scanning	1
Linear Basis (Gaussian Elimination for XOR). Find a set of basis vectors that can represent any possible XOR sum of the array.	1
Linear Boundary Comparison	1
Linear Count Tracker	1
Linear Counter + Math	1
Linear DP (Climbing Stairs Variant)	1
Linear DP + Monotonic Optimization	1
Linear Dynamic Programming (DP) - Staircase variation.	1
Linear Expression Reduction (Parsing).	1
Linear Filter Pipeline + Custom Sort	1
Linear Iteration (Math/Simulation)	1
Linear loop checking `nums[i]-nums[i-1]` + one explicit edge check for `nums[N-1]-nums[0]`. O(N)	1
Linear Memory Scan (Visualizing the CPU reading the entire marbles array for every single instruction).	1
Linear One-Pass Update	1
Linear Row Scan + Tracker	1
Linear Scan (Manual Digit Check)	1
Linear Scan (Max Property Observation)	1
Linear Scan (Min/Max Tracking)	1
Linear Scan + Digit Math.	1
Linear Scan / Adjacent Identical Pairs Math	1
Linear Scan / Counter	1
Linear Scan / Fixed Sliding Window	1
Linear Scan / String Splitting.	1
Linear Scan / Substring Search.	1

Linear Scan of Adjacent Elements.	1
Linear Scan with Consecutive Counter. O(N)	1
Linear Scan with Mathematical Digit Extraction.	1
Linear Scan with State Tracker	1
Linear Search Overhead (Visualizing indexOf being called repeatedly).	1
Linear Search Pointers (Drawing the search range for every individual query).	1
Linear Simulation / Iteration	1
Linear Strike Counting (Math)	1
Linear Traversal (Bidirectional)	1
Linked Hash Map / Queue + Hash Map	1
Linked List Traversal with a Dummy Head (tracking the start of the next level).	1
LIS (Longest Increasing Subsequence) + LDS (Decreasing)	1
LIS-style DP + Coordinate Compression + Fenwick Tree	1
List Maintenance + Pointer Tracking	1
List of Coordinates + Delta Frequency Map.	1
Local Incremental Update	1
Logarithmic Digit Counting / Range Check	1
Logical Comparison of First Two Edges.	1
Logical Invariance (Existence of '1').	1
LogTrick / Hash Map (Monotonic Bitwise AND)	1
Long Division + Hash Map (Remainder Tracking).	1
Longest Common Substring (LCS Variant)	1
Longest Consecutive Sequence (Greedy)	1
Longest Non-Decreasing Subsequence (Patience Sorting / Binary Search).	1
Look-Ahead (Caching)	1
Loop pointers footprint (Draw i and j pointers constantly jumping back and forth across an array).	1
Lowest Common Ancestor (LCA) + Node Depth.	1
Lowest Common Ancestor (LCA) via integer division (node / 2) in a Complete Binary Tree.	1
Lowest Common Ancestor (LCA) with Binary Lifting.	1
Manacher's Algorithm + Prefix/Suffix Max	1
Manacher's Algorithm + Suffix Automaton / Hash.	1
Manhattan Distance Comparison	1
Map of Min-Heaps (Group by i - j)	1
Massive NxN Grid Matrix (Highlighting the memory waste of storing every direct edge).	1
Massive Set Allocation (Visualizing thousands of HashSets being created and destroyed in memory).	1
Massive Sort Buffer Allocation (Drawing the memory used to sort N^2 subarrays).	1
Massive String Heap (Visualizing huge strings being copied and stored at every level of recursion).	1
Massive String Reallocations (Immutable strings mean creating a brand new string in memory for every deletion).	1
Math	1
Math (Algebraic Equation).	1
Math (Arithmetic Progression Series).	1
Math (Arithmetic Progression)	1
Math (Combinatorics + Stars and Bars)	1
Math (Combinatorics) + Filtering.	1
Math (Contribution to Sum) + Combinatorics	1
Math (Cross Product / Slope Comparison).	1
Math (Digit Extraction and Remainder).	1
Math (Direct Search is already efficient given N <= 10^5)	1
Math (Distance Squared Comparison).	1
Math (Distributive Law): (a^b) AND (c^d).	1
Math (Euclidean GCD Logic).	1
Math (Euclidean-like division).	1
Math (Expression Expansion / 8 Scenarios)	1
Math (FOIL Method) + String Split.	1
Math (Median Finding) + Divisibility Check	1
Math (Modular Inverse) + Global Modifiers	1
Math (Modulo 3 Categorization) & Case Analysis	1

Math (Modulo Arithmetic / Bitwise)	1
Math (Modulo Arithmetic / Pigeonhole Principle).	1
Math (Modulo Arithmetic)	1
Math (Modulo with Negative Base).	1
Math (Observation)	1
Math (Parity check): Sum of ASCII value of letter and integer value of number.	1
Math (Parity tracking). Count negative numbers and check for zeros.	1
Math (Perfect Square Property)	1
Math (Running Modulo).	1
Math (Sieve-like approach). Check if a target GCD 'x' can be formed by iterating its multiples.	1
Math (Stars and Bars) / DP	1
Math (Suffix Length Precalculation)	1
Math (Summation / XOR)	1
Math (Symmetry / Bit Manipulation).	1
Math (The Josephus Problem) / Bottom-Up DP.	1
Math (Trailing Zeros + Modulo/Logarithms).	1
Math (Triangular Numbers) or DP.	1
Math + Prefix Sums of "Division Counts". O(Q log(Range)) or O(Q · 1).	1
Math / Binary Pattern Recognition.	1
Math / Binomial Coefficients	1
Math / Combinatorics (Stirling Numbers of the Second Kind)	1
Math / Contiguous Counting	1
Math / Contribution Technique	1
Math / Digit DP.	1
Math / Edge-Delta Optimization	1
Math / Geometry (Closest Point)	1
Math / Geometry (Line of Sight checks)	1
Math / Greedy / Min-Max	1
Math / Greedy Distribution (Base + Remainder)	1
Math / Greedy Observation.	1
Math / Half-String Mirroring	1
Math / Hash Set (Min/Max finding)	1
Math / Index Hopping	1
Math / Line Intersection (Convert MMDD to Day of Year Integer)	1
Math / Modular Arithmetic	1
Math / Modulo Arithmetic.	1
Math / Number Theory (Global Minimum modulo reduction)	1
Math / Observation (O(1)).	1
Math / Pair Divisors up to Square Root of N.	1
Math / Parity Analysis.	1
Math / Permutation Pruning (Binary Search style)	1
Math / Prefix Sum (Tracking flow and bottlenecks).	1
Math / Simulation	1
Math / Simulation (Repeated Division and Modulo).	1
Math / Square Root Optimization	1
Math Bounds + Hash Set	1
Math Formula	1
Math Observation (Bubblesort Property).	1
Math Optimization + Prefix Map (Advanced)	1
Math Rearrangement + Hash Map: Check nums[i] - rev(nums[i]) == nums[j] - rev(nums[j]).	1
Math-Based Accumulator	1
Math-based column parity (Odd/Even columns)	1
Math-based digit extraction (Modulo/Division).	1
Math/Graph (Cycle Detection). Track only the movement of index 1.	1
Math/Index Mapping (i / n, i % n)	1
Math/Modulo Zero-Check Loop	1
Mathematical Base Conversion (N % K, N // K) using a StringBuilder/Byte Array, followed by a single Reverse operation.	1

Mathematical Binning (Floor Division)	1
Mathematical Bitwise Cancellation (Symmetry). The expression simplifies to just XORing all elements in the array.	1
Mathematical Ceiling Division.	1
Mathematical Constant Division (Base-10 Log or Max Int)	1
Mathematical Digit Decomposition	1
Mathematical Digit Extraction	1
Mathematical Digit Extraction (Modulo & Division)	1
Mathematical Digit Extraction + Hash Set	1
Mathematical Digit Extraction.	1
Mathematical Digit Summing + Array Frequency Map.	1
Mathematical DP (Linear Recurrence)	1
Mathematical Greedy + Parity Check	1
Mathematical Greedy Partitioning	1
Mathematical Induction / Constant Observation	1
Mathematical Insight (Constant Return)	1
Mathematical Insight (Elimination)	1
Mathematical Interval Binary Search	1
Mathematical Invariant / Game Theory	1
Mathematical Least Common Multiple (LCM).	1
Mathematical Modulo (Game Theory)	1
Mathematical Modulo-10 Digit Extraction.	1
Mathematical Observation (Cascade Effect).	1
Mathematical Observation (Parity) OR 2D DP.	1
Mathematical Observation (Powers of 2)	1
Mathematical Offset Calculation (f(R) - f(L-1)).	1
Mathematical Parity / Bottleneck Check	1
Mathematical Pattern (Square Root)	1
Mathematical Pattern Reduction.	1
Mathematical Progression (Sum of Arithmetic Series).	1
Mathematical Proof / Bit Manipulation (XOR properties).	1
Mathematical Property (Perfect Squares).	1
Mathematical Range Summation.	1
Mathematical Reduction Incrementing n-1 elements is mathematically equivalent to decrementing 1 element. Sum up the distance of each element from the minimum.	1
Mathematical Relative Position	1
Mathematical Remainder Reversal	1
Mathematical Series (Tetrahedral Numbers) + Greedy.	1
Mathematical Simulation (Step & Head Tracking)	1
Matrix Exponentiation	1
Matrix Exponentiation / 1D DP.	1
Matrix Marking / State Grid	1
Matrix Traversal (Local Constraint)	1
Matrix Traversal + O(√N) Prime Check	1
Max Heap (Priority Queue)	1
Max Heap + Absolute Value Sorting	1
Max Heap / Binary Search on Sums	1
Max Heap / Priority Queue (Greedy)	1
MAX() with Conditional Aggregation (CASE/IF)	1
Max-Heap (Priority Queue) of "Potential Gain".	1
Max-Heap (Priority Queue) of Intervals.	1
Max-Heap (Priority Queue) to maintain the largest element dynamically.	1
Max-Heap (Priority Queue).	2
Max-Heap (Size K) or QuickSelect	1
Max-Heap + Cooling Queue (Greedy)	1
Max-Heap / Greedy with Interleaving	1
Max-Heap / K-way Merge (Pull Model)	1
Max-Priority Queue (Heap)	1
Max-Value Tracking (Greedy)	1

Maximum Flow / Bipartite Matching. Model robots as one set of nodes, walls as another, and find the maximum matching/coverage.	1
Median + Palindrome Generation	1
Median Property (Independent Dimensions)	1
Median Target (Greedy) The point that minimizes the sum of absolute differences is the median. Sort and find the middle.	1
Median-Based Greedy approach. The total cost is minimized when all elements are changed to the median of the array.	1
Median-based Greedy.	1
Meet in the Middle + Binary Search	1
Meet-in-the-Middle + Binary Search / Trie.	1
Meet-in-the-Middle + Dynamic Programming / Math Pruning.	1
Meet-in-the-Middle + Two Pointers.	1
Memoized Recursion / Dynamic Programming on (n, firstPlayer, secondPlayer).	1
Merge Intervals + Binary Search	1
Merge Intervals + Combinatorics	1
Merge Intervals / Segment Tree / Jump Pointers (DSU).	1
Merge Sort (Bottom-Up Iterative for O(1) space, or Top-Down Recursive).	1
Merge Sort (Divide & Conquer) During the merge step, both halves are sorted. Use two pointers to count reverse pairs before merging.	1
Merge Sort (Divide & Conquer) or Fenwick Tree after transforming to (nums1[i]-nums2[i])	1
Merge Sort (Divide & Conquer) tracking original indices	1
Merge Sort / Quick Sort / Heap Sort (Divide & Conquer)	1
Merkle Hashing or String Serialization (KMP).	1
Middle Character Extraction	1
Middle Element Fix + Left/Right Max Search	1
Min Heap (Priority Queue)	1
Min-Cost Flow / Hungarian Algorithm	1
Min-Element Comparison / Math	1
Min-Heap (K-Way Merge)	1
Min-Heap (Priority Queue)	2
Min-Heap (Priority Queue) of End Times	1
Min-Heap (Priority Queue) tracking `[rot_day, count]`.	1
Min-Heap (Priority Queue) with Dijkstra-like Expansion	1
Min-Heap (Size K)	1
Min-Heap + BFS Push perimeter to min-heap. Pop lowest boundary, process neighbors, add to heap. "Shrinking perimeter".	1
Min-Heap + Boolean Visited Array	1
Min-Heap + Hash Set (or Min_Tracker)	1
Min-Heap + List of Stacks	1
Min-Heap + Map of Sets	1
Min-Heap / Priority Queue (Multi-way Merge)	1
Min-Max Coordinate Tracking	1
Min-Priority Queue (Heap)	2
Min-Priority Queue (Min-Heap) sorted by processing time and original index.	1
Min-Priority Queue (Min-Heap).	1
Min/Max Tracking.	1
Minimal Memory (Only tracking unique category representatives).	1
Minimal Variable State (Only index i).	1
Minimax / Information Theory (Heuristic Guessing).	1
Minimax / Topological Sort on States	1
Minimum Boundary Tracking.	1
Mirroring + Middle-Digit Increment. Construct the palindrome from the first half of N, then adjust based on "Special" digit constraints.	1
Mismatch Index Comparison.	1
Modified BFS (Dijkstra-like) with Top-2 Tracking + Traffic Light Math	1
Modified Binary Search	1
Modified Dijkstra (or 2D DP)	1
Modified Dijkstra (State-Space Expansion)	1
Modified Dijkstra / BFS with State	1
Modified Dijkstra / Min-Heap BFS	1
Modified Dijkstra's Algorithm (Min-Heap)	1

Modified Dijkstra's Algorithm (Shortest Path on weighted states).	1
Modified Kadane's Algorithm for each distinct value T. O(50 · N)	1
Modified Merge Sort (Divide & Conquer)	1
Modular Arithmetic & Hash Map (Prefix Modulos).	1
Modular Arithmetic (Cycles).	1
Modular Arithmetic (Iterative Remainder).	1
Modular Arithmetic + Hash Map (Digit Length Bins).	1
Modular Exponentiation + Recursive Property	1
Modulo Arithmetic (%)	1
Modulo Arithmetic (Circular Indexing)	1
Modulo Arithmetic Indexing	1
Modulo Backtracking (Reverse Engineering)	1
Modulo Digit Extraction	1
Modulo Frequency Mapping (Remainders).	1
Modulo Math + Sliding Window on Doubled Array	1
Modulo Simulation	1
Modulo Sliding Window (Size 3)	1
Modulo-based Frequency Array. Use `n % 10` to strip digits and an array of size 10 to store counts.	1
Monotone Chain Algorithm (Sorting + Cross Product).	1
Monotonic Decreasing Stack	1
Monotonic Decreasing Stack (Looping 2N times).	1
Monotonic Deque (Double-Ended Queue)	1
Monotonic Deque (Sliding Window Max)	1
Monotonic Deque + Prefix Sums.	1
Monotonic Deque / Two Pointers + Sliding Window Minimum.	1
Monotonic Stack	3
Monotonic Stack (Decreasing)	2
Monotonic Stack (Greedy)	1
Monotonic Stack (Increasing)	2
Monotonic Stack (Next Greater Element)	1
Monotonic Stack (Next Smaller Element)	1
Monotonic Stack (O(N)).	1
Monotonic Stack (Pair of Price and Span).	1
Monotonic Stack (Parent Tracking)	1
Monotonic Stack (Right-to-Left) + Collision Time Comparison.	1
Monotonic Stack (Right-to-Left) Keep a stack of potential "2" values (middle magnitude) and track the largest "3" value found so far.	1
Monotonic Stack (Row/Col wise)	1
Monotonic Stack (Simulation of DFS)	1
Monotonic Stack (Time-to-Die)	1
Monotonic Stack + 1D DP	1
Monotonic Stack + Contribution of Each Element + Math (Counting valid subarrays). O(N)	1
Monotonic Stack + DP (Suffix Jump Mapping)	1
Monotonic Stack + Dynamic Programming	1
Monotonic Stack + Frequency Map + Visited Array.	1
Monotonic Stack + Greedy (Frequency Map).	1
Monotonic Stack + Greedy + Frequency Map	1
Monotonic Stack + HashMap Pre-process nums2 using a stack to find all "Next Greater" pairs. Store in a Map `num: next_greater`.	1
Monotonic Stack + Lexicographical Merge	1
Monotonic Stack + Prefix Sum of Prefix Sums	1
Monotonic Stack + Prefix/Suffix DP	1
Monotonic Stack + Range Extension	1
Monotonic Stack + Sieve of Eratosthenes + Greedy (Contribution Technique)	1
Monotonic Stack + Two Pointers	1
Monotonic Stack + Two Pointers (Right-to-Left)	1
Monotonic Stack / Greedy	1
Monotonic Stack / Greedy Suffix Max	1
Monotonic Stack / Histogram	1

Monotonic Stack Keep digits in increasing order. Pop if current digit < top of stack and K > 0.	1
Monotonic Stack or Reverse Recursion	1
Monotonic Stack to find the next/previous greater/smaller elements in O(N), then Dijkstra/DP on those specific edges.	1
Monotonic Stack with Budget Constraints	1
Monotonic Stack.	3
Morris Inorder Traversal	1
Multi-Dimensional Dynamic Programming (State Compression).	1
Multi-Level LEFT JOIN with Grouping	1
Multi-Level Sorting (Frequency Matrix)	1
Multi-pass Two Pointers (In-Place).	1
Multi-Queue Interleaving	1
Multi-Source BFS	4
Multi-source BFS (Water) + Standard BFS (Player)	1
Multi-Source BFS + Arrival Time Comparison	1
Multi-source BFS + Max-Heap/Dijkstra (or Binary Search + BFS)	1
Multi-source BFS + Priority Queue (Dijkstra Variant).	1
Multi-Source BFS / Dijkstra with Coordinate Mapping.	1
Multi-Source BFS or 2-Pass DP.	1
Multi-Source BFS.	1
Multi-source BFS/DFS Start from the Pacific edges and "flow uphill." Do the same for Atlantic. Find the intersection.	1
Multi-source Breadth-First Search (BFS).	1
Multi-source DFS/BFS from Boundaries.	1
Multi-Source Dijkstra / Graph Reversal	1
Multiple Boolean Flags (Visualizing all the manual state variables needed for date math).	1
Multiple Statement Contexts (Each call is a separate statement in the execution stack).	1
Multiple TreeSets / Priority Queues + HashMaps	1
Native Pandas Constructor (Vectorized Internal Logic)	1
Nested BFS / State BFS (Box Pos + Person Pos)	1
Nested Char Loops.	1
Nested Loop Sliding Window with Running Set	1
Nested Loop with Running Frequency Array.	1
Nested Maps / Prefix Tree (Trie)	1
Net Displacement Counter.	1
Net Shift Calculation + Slicing	1
Network Socket Overload (Visualizing multiple open connections consuming system file descriptors).	1
New Grid Mapping.	1
Next Permutation (K times) + Greedy Swap Alignment.	1
Next Permutation Algorithm (Linear Scan).	1
Next Permutation Algorithm (Suffix Reversal)	1
Next Permutation Algorithm on the first half.	1
Next Permutation on Half-String	1
Number Theory (GCD + Frequency Map).	1
Number Theory (GCD + Power of Two).	1
Number Theory (GCD / Bézout's Identity)	1
Number Theory (Greatest Common Divisor)	1
Number Theory / Sieve of Eratosthenes	1
O(1) Combinatorics (Parity Math)	1
O(1) State Variables (Draw one sum and one i).	1
O(M^2) Simulation (M = instruction length).	1
O(M· N) Matrix Allocation (Massive memory waste for grids like 10^5 x 10^5).	1
O(N^2) 2D Array Allocation (Massive memory waste for large N).	1
Object-Oriented Design (Polymorphism) + Postfix Stack Tree	1
Object.entries() + Array.reduce()	1
Odd Frequency Character Counting	1
Offline Queries (Sort Queries) + Min-Heap (Priority Queue) + BFS to prevent re-processing nodes.	1
Offline Queries + Bitwise Trie.	1
Offline Queries + DFS + Binary Trie	1

Offline Queries + DSU with Sorting.	1
Offline Queries + Monotonic Stack / Priority Queue	1
Offline Queries + Sorting + Balanced BST (TreeSet).	1
Offline Queries + Sorting + Min-Priority Queue (Heap).	1
Offline Queries + Union-Find (DSU)	1
On-the-fly Trial Division + Hash Set deduplication.	1
One-Pass DP / Sliding Window (Arithmetic Progressions)	1
One-Pass Greedy (Peak-Valley).	1
One-pass Greedy / Kadane’s-like Sliding Window.	1
One-Pass Greedy / Simulation	1
One-Pass Greedy Iterate through attacks. If the gap between attacks < duration, add the gap. Else, add the full duration.	1
One-Pass Greedy Swapping	1
One-Pass Mathematical Summation	1
One-Pass Min-Max Tracking	1
One-Pass Neighbor Comparison	1
One-Pass Parity Tally	1
One-Pass Precomputation	1
One-Pass Running Maximum	1
One-pass Sliding Window / Greedy.	1
One-Pass String Comparison.	1
One-Pass with Toggle/Previous Tracking	1
Optimal Brute (Observation: GCD only decreases)	1
Optimised Monotonic Stack + Dynamic Programming	1
Optimized BFS / DFS on Digit States	1
Optimized Brute Force with LCM Early Exit	1
Optimized Linear Scan (Early Exit)	1
Optimized Sliding Window (Variable Frequency)	1
ORDER BY Multiple Columns	1
Ordered Map (TreeMap) / Balanced BST	1
Ordered Map / TreeMap with Granularity Masking.	1
Palindrome Construction (Math/Generator)	1
Palindrome Generation (Length-based)	1
Palindrome Generation + Combinatorial Counting	1
Palindrome Generation + Primality Test.	1
Parallel StringBuilder Appending	1
Parity Counting (Math/Greedy)	1
Parity Counting (Row/Col Arrays)	1
Parity Logic (Difference is even iff Total Sum is even). O(N)	1
Parity Summation	1
Parity-Based Character Comparison	1
Parity-Based Math (Even/Odd Geometry)	1
Partition Memory Sets (Visualizing two massive separate heaps for every single partition attempt).	1
Path Frequency Maps (Visualizing thousands of HashMaps created to check character counts for every path).	1
Patience Sorting (Binary Search + Tails Array)	1
Patience Sorting (LIS Variation)	1
Patience Sorting / Segment Tree (LIS variant)	1
Pattern Canonicalization / Hashing.	1
Pattern Categorization (6-Segment Window)	1
Pattern Conversion + KMP Algorithm	1
Pattern Searching (Presence of "01").	1
pd.concat([df1, df2])	1
Perimeter Modulo Math	1
Perimeter Unwrapping + Binary Search on Answer + Greedy. O(N log N + N log side)	1
Permutation Validation (Fixed Size)	1
Persistent Pointer (Amortized Analysis)	1
Pigeonhole Principle (Frequency Counting)	1

Pigeonhole Principle + Iterative Modulo	1
Pivot Table (CASE WHEN + GROUP BY).	1
Pointer Manipulation + Dummy Node	1
Pointer Simulation on Compressed Pairs.	1
Pointer-based Simulation	1
Polar Angle Sorting + Sliding Window	1
Polynomial Class / Map of Term-to-Coefficient	1
Polynomial Rolling Sum (Base-26 conversion).	1
Post-Order DFS (Bottom-Up Flagging)	1
Post-Order DFS (Bottom-Up Size Tally)	1
Post-Order DFS (Bottom-Up State)	1
Post-Order DFS (Bottom-Up)	2
Post-order DFS (Bottom-Up)	1
Post-order DFS (Bottom-Up).	1
Post-order DFS (Depth-Pair Return)	1
Post-Order DFS (Depth-Pair Return).	1
Post-Order DFS (Tree DP)	2
Post-Order DFS Sum Tracking	1
Post-order DFS Traversal + Hash Set	1
Post-Order Recursive Reconstruction	1
Post-order Subtree Sums + Hash Set.	1
Post-order Traversal (Bottom-Up DFS) + Hash Map.	1
Post-Order Traversal (Bottom-Up DFS).	1
Post-order Traversal (Bottom-Up DFS). 1 + max(left, right)	1
Post-order Traversal (Bottom-Up DP)	1
Post-order Traversal (Bottom-Up Information Passing)	1
Post-order Traversal (Bottom-Up Pruning).	1
Post-Order Traversal (Bottom-Up Summing)	1
Post-order Traversal (Bottom-Up Summing).	1
Post-order Traversal (Bottom-Up)	1
Post-Order Traversal (Bottom-Up)	1
Post-Order Traversal (DFS)	1
Post-Order Traversal (Excess/Deficit Flow)	1
Post-order Traversal + Serialization + Hash Map.	1
Power of Two Check (Bit Manipulation).	1
Pre-allocation + Linear Fill	1
Pre-calculation + Stable Sorting.	1
Pre-computation + Binary Search/Suffix Sum	1
Pre-computation with Hash Set	1
Pre-computed Array + Leap Year Math	1
Pre-evaluated Boolean Variables mapping to strict logic combinations.	1
Pre-mapped Char-to-Row Create a Map or Array of size 26 where `row[char]` is 1, 2, or 3. Check if all chars in a word return the same row ID.	1
Pre-order + Bounds Use Pre-order without null markers. Rebuild using BST property (current value must be within [min, max]).	1
Pre-order / In-order / Post-order Coordination.	1
Pre-order Traversal (Bitwise Accumulation).	1
Pre-Order Traversal (DFS) with Sentinel Markers	1
Pre-order with Child Count Serialize as `Value, ChildCount`. E.g., `1,3,2,0,3,0,4,0` (Node 1 has 3 children...).	1
Pre-processing (Index Collection)	1
Precomputation (2D Grid Greedy).	1
Precomputation (Prefix Products)	1
Precomputation (Row/Col Sum Arrays)	1
Precomputation + 2D Partition Dynamic Programming	1
Precomputation + Binary Search	1
Precomputation / Backtracking with Pruning	1
Precomputation / Bitmasking	1
Precomputation / Combinatorics	1
Precomputation of All Substrings (up to 30 bits) + Hash Map.	1

Precompute "Longest Decreasing" from left and "Longest Increasing" from right for every index.	1
Precompute dependencies and apply a Greedy "Earliest Start" calculation.	1
Precomputed Frequency Map + Single Pass. O(N)	1
Precomputed HashMaps (Priority Tiering)	1
Precomputed Pairwise Frequency (Hashmap)	1
Precomputed Prefix/Suffix Counts (DP)	1
Precomputed Row-Mins and Col-Maxes	1
Prefix & Suffix Arrays	1
Prefix & Suffix Arrays (State Tracking)	1
Prefix & Suffix Products (Flattened 2D to 1D)	1
Prefix and Suffix Accumulators	1
Prefix and Suffix Counts. Count existing '0's and '1's. Inserting '0' at the start pairs with all existing '1's. Inserting '1' at the end pairs with all existing '0's.	1
Prefix and Suffix Precomputation Arrays.	1
Prefix and Suffix Running Totals.	1
Prefix and Suffix Sums / Hash Map of Lists.	1
Prefix and Suffix Sums. Precalculate even/odd sums. When you "remove" an element, the suffix even/odd sums simply swap roles!	1
Prefix Expansion / Direct String Construction	1
Prefix Hash Set / Trie	1
Prefix Manipulation + Candidate Testing.	1
Prefix Max & Suffix Min	1
Prefix Max & Suffix Min Pass	1
Prefix Max / Suffix Max Arrays	1
Prefix Max and Suffix Min Arrays	1
Prefix Maximum & Suffix Maximum (or Pre-calculation)	1
Prefix Min & Suffix Min Arrays	1
Prefix Min & Suffix Min Arrays (Identical to 2908)	1
Prefix Parity State + Sliding Window on Pair Combinations (20 pairs). O(20 · N)	1
Prefix Product Array with Zero-Reset	1
Prefix Sum & Suffix Sum Arrays (Minimax pattern)	1
Prefix Sum (In-place)	1
Prefix Sum (Parity Violations)	1
Prefix Sum (Running Total).	1
Prefix Sum (Total Sum Balance)	1
Prefix Sum (Treat 0 as -1) + Hash Map.	1
Prefix Sum + Binary Indexed Tree (Fenwick)	1
Prefix Sum + Binary Indexed Tree (Fenwick) / Merge Sort	1
Prefix Sum + Binary Search (or Sliding Window).	1
Prefix Sum + Binary Search Calculate areas of all rectangles. Create a prefix sum array. Randomly pick a "total area" value and binary search for the rect.	1
Prefix Sum + DFS Use a Hash Map to store "Prefix Sum -> Frequency". If (CurrentSum - Target) exists in the map, a path is found.	1
Prefix Sum + Dynamic Programming (Suffix Max).	1
Prefix Sum + Hash Map (Diff Tracking)	1
Prefix Sum + Hash Map (First Occurrence)	1
Prefix Sum + Hash Map (Math Restructuring)	1
Prefix Sum + Hash Map (Minimum Prefix Tracking)	1
Prefix Sum + Hash Map (Modular Arithmetic)	1
Prefix Sum + Hash Map (Modulo Tracking)	1
Prefix Sum + Hash Map (One/Two Pass)	1
Prefix Sum + Hash Map (Two-Sum Variation)	1
Prefix Sum + Hash Map / Monotonic Stack	1
Prefix Sum + Hash Map.	1
Prefix Sum + Hash Set + Greedy	1
Prefix Sum + Hash Set for Mid-Point Optimization.	1
Prefix Sum + Hashmap (or Sliding Window)	1
Prefix Sum + Math (Parity)	1
Prefix Sum + Modulo Arithmetic + Binary Search.	1
Prefix Sum + Modulo Hash Map.	1
Prefix Sum + Monotonic Stack (Next Smaller Element).	1

Prefix Sum + Range Math.	1
Prefix Sum / Minimum Running Sum	1
Prefix Sum / Running Count.	1
Prefix Sum / Running Total	3
Prefix Sum / Total Sum Tracking	1
Prefix Sum Array	1
Prefix Sum Array + Binary Search.	1
Prefix Sum Array + Leap Year Logic	1
Prefix Sum Array.	1
Prefix Sum Array. O(N)	1
Prefix Sum Average (Greedy Ceiling)	1
Prefix Sum Balance	1
Prefix Sum Difference + Hash Map	1
Prefix Sum Hash Map / Two-Sum Pattern	1
Prefix Sum Modulo K + Trie/String Hashing	1
Prefix Sum on Trees (Parity)	1
Prefix Sum Range (Max - Min).	1
Prefix Sum Simulation	1
Prefix Sum Transformation + Fenwick Tree / Sliding Window.	1
Prefix Sums (Brute Force is expected for N=1000)	1
Prefix Sums (Cost Optimization)	1
Prefix Sums (Even/Odd Tracking)	1
Prefix Sums (Mathematical Simplification)	1
Prefix Sums + Binary Search (or Two Pointers).	1
Prefix Sums + Bitmasking	1
Prefix Sums + Combinatorics (nC2)	1
Prefix Sums + First Occurrence Hash Map.	1
Prefix Sums + Frequency Counting (HashMap).	1
Prefix Sums + Frequency Map (Balance Tracking)	1
Prefix Sums + Frequency Map (Remainder Counting)	1
Prefix Sums + Frequency Maps.	1
Prefix Sums + Hash Map	1
Prefix Sums + Hash Map (Frequency of remainders)	1
Prefix Sums + Hash Map (Frequency Tracking)	1
Prefix Sums + Hash Maps for Difference Tracking	1
Prefix Sums + Last Occurrence Tracking	1
Prefix Sums + Nearest Candle Precomputation	1
Prefix Sums + Parity Symmetry Check	1
Prefix Sums + Range Intersection (Inequality Check).	1
Prefix Sums + Relative State Tracking (+1/-1)	1
Prefix Sums / Difference Array (Line Sweep)	1
Prefix Sums / Frequency Mapping.	1
Prefix Sums / Running Totals	1
Prefix Sums / Running Totals.	1
Prefix Sums / Single Pass Logic	1
Prefix Sums of Coordinates. Calculate (x, y) for all prefixes and suffixes. Endpoint = Prefix[i-1] + (Total - Prefix[j]) .	1
Prefix Sums of Skill + Gap Analysis Formula. O(N · M)	1
Prefix Sums on 2D Coordinates / Hash Map caching.	1
Prefix Sums on Doubled Array + Median of Differences	1
Prefix Sums over Frequency Array	1
Prefix Sums. Calculate all cumulative sums once. Sum of right half = TotalSum - PrefixSum[i] .	1
Prefix XOR + Binary Trie with Range Counting.	1
Prefix XOR + Bitwise Complement.	1
Prefix XOR + Dynamic Programming with HashMap.	1
Prefix XOR + Hash Map (First Occurrence)	1
Prefix XOR + Hash Map (Frequency Counting).	1
Prefix XOR + Property of XOR	1

Prefix XOR + Target Search.	1
Prefix XOR / Greedy Mathematical Observation	1
Prefix XOR Array (Cumulative State Cache)	1
Prefix XOR Array combined with a Binary Trie (Prefix Tree) to count valid pairs in O(N log(Max_Value)) time.	1
Prefix/Suffix Counts	1
Prefix/Suffix DP (Length 5)	1
Prefix/Suffix Dynamic Programming (Precomputation)	1
Prefix/Suffix OR + Greedy Power of 2	1
Preorder DFS with Conditional Appending.	1
Prepared Statements / Dynamic SQL	1
Preprocessing (Row/Col Sums)	1
Prim's/Kruskal's + Greedy / DP with Knapsack.	1
Prime Factorization (Greedy/DP).	1
Prime Factorization + Range Merging / Interval Coverage.	1
Prime Factorization + Stars and Bars (Combinatorics).	1
Prime Product Hashing or Frequency Array	1
Priority Queue (Max-Heap) + Greedy Shrinking	1
Priority Queue (Slope Trick)	1
Priority Queue / Array of Stacks	1
Priority Queue Heap State (Visualizing the heap re-balancing for every element used in a group).	1
Probabilistic Multi-Level Linked List	1
Promise.allSettled Polyfill	1
Promise.race() with Timer	1
Promises + setTimeout	1
Proxy + Structural Sharing	1
Proxy Object / Recursive Object.freeze	1
Pruned DFS (Early Exit)	1
Pruning + Constant-Time Symmetry Check.	1
Query/Pair Stack (Drawing the search range for every individual query).	1
Queue + 3 TreeMaps (Multisets for Bottom K, Middle M-2K, Top K) + Running Sum.	1
Queue + HashSet (Available Pool)	1
Queue Memory (Visualizing the BFS queue filling with millions of possible intermediate numbers).	1
Queue Memory (Visualizing the BFS queue growing linearly with K, potentially millions of strings).	1
Queue of Iterators	1
Queue-based Level Order State	1
Quickselect (Hoare's Partition) OR Min-Heap	1
Quickselect + Virtual Indexing (3-way Partition)	1
QuickSelect Partitioning (nth_element)	1
Radix Sort Logic / Precomputed Sorted States	1
Range Narrowing (Iterative Intersection). O(N)	1
Rank Mapping (Hash Map / 2D Array)	1
Re-rooting DP (Post-order + Pre-order DFS).	1
Recursion + Bitmask Memoization Use an integer bitmask to represent which numbers are used. Cache result in a Map `mask: win/loss`.	1
Recursion + String Sorting (Divide & Conquer)	1
Recursion / Divide and Conquer	1
Recursion / Stack with HashMaps	1
Recursion with Backtracking	1
Recursion with Memoization	1
Recursion with Memoization (Divide and Conquer).	1
Recursion with Memoization (Top-Down DP)	1
Recursion with Range (Min/Max)	1
Recursion with Valid Range (Upper/Lower Bounds).	1
Recursive Backtracking (Implicit Stack)	1
Recursive Backtracking (Inside-Out Construction)	1
Recursive Backtracking (Post-Order Traversal)	1
Recursive Backtracking with Frequency Map (or House Robber DP).	1
Recursive Branch Pruning.	1

Recursive Call Stack (Visualize memory frames for every nested level of the object).	1
Recursive Call Stack (Visualizing deep paths and redundant calculations of the same sub-problems).	1
Recursive Call Stack (Visualizing depth-first search for the longest possible valid string path).	1
Recursive Call Stack (Visualizing the depth-first search through all valid and invalid partition combinations).	1
Recursive Call Stack (Visualizing the massive overhead of re-calculating the best path for the same index multiple times).	1
Recursive Canonical Comparison	1
Recursive Chain / Worker Pool	1
Recursive Closure / Function Arity	1
Recursive CTE (Common Table Expression)	1
Recursive CTE (Date Generation) + Left Join	1
Recursive CTE (Month Generator) + LEFT JOIN + Window Functions	1
Recursive CTE (Range Generation) + Left Join	1
Recursive CTE (SQL) / DFS with Aggregate Accumulation. O(N)	1
Recursive CTE (SQL) / Graph Cycle Detection with Visited Set.	1
Recursive CTE (WITH RECURSIVE)	1
Recursive CTE (Year-Level Pivot)	1
Recursive CTE + LEFT JOIN	1
Recursive CTE + LEFT JOIN + SUM/COUNT Window Functions	1
Recursive CTE + LEFT JOIN.	1
Recursive CTE + Window Function (ROWS BETWEEN)	1
Recursive Descent Parser + Scoped Symbol Table	1
Recursive Descent Parser / Stack	1
Recursive Descent Parsing / Stack.	1
Recursive Descent with Operator Precedence	1
Recursive DFS (Object Traversal)	1
Recursive DFS (Root arrow Right arrow Left).	1
Recursive DFS (Structural Comparison)	1
Recursive DFS Flattening + Sorted Key Set	1
Recursive DFS or Iterative "Reverse Preorder".	1
Recursive DFS or Iterative Stack.	1
Recursive DFS Serialization	1
Recursive DFS Type Checking	1
Recursive DFS with Depth Tracking	1
Recursive DFS with Depth Tracking.	1
Recursive Divide and Conquer (Sign Partitioning)	1
Recursive Generation with Range Pruning	1
Recursive In-Place Merge.	1
Recursive In-place Pointer Reversal	1
Recursive Length-Based Search	1
Recursive Logical AND	1
Recursive Matrix Filling (Height-based).	1
Recursive or Iterative In-Place Swap (DFS)	1
Recursive Pattern / Head-Pointer Tracking	1
Recursive Pointer Re-assignment.	1
Recursive Post-Order Split	1
Recursive Pruning (BST Property)	1
Recursive Range Splitting (Divide and Conquer).	1
Recursive Re-linking Find node. If it's a leaf, remove. If 1 child, replace with child. If 2 children, replace with In-order Successor (min in right subtree).	1
Recursive setTimeout via Closure	1
Recursive Symmetry / Divide & Conquer	1
Recursive Tree Merging (Divide & Conquer).	1
Recursive Triplet Grouping (Base 1000)	1
Recursive Yield* (Delegation)	1
Redundant Array Rotation Buffers (Storing the state of the array for every single shift).	1
Redundant Decision Paths (Visualizing same wall counts being re-calculated in different branches).	1
Redundant Min/Max Scans (Drawing a search arrow within every window).	1
Redundant String Slicing (Visualizing thousands of temporary substring copies being checked against a HashSet).	1

Redundant Traversal Stack (Visualizing the CPU re-visiting the same bottom nodes for every ancestor).	1
Regex (Regular Expressions) or SQL Pattern Matching. O(N)	1
Regex Extraction + Group By + Order By	1
Regex Matching or Structured Single-Pass Scan	1
Regex or Modular Parsing Use pre-defined Regex patterns for IPv4 and IPv6 to handle boundaries and valid character ranges in one pass.	1
Regex Pattern Matching (grep -P or awk).	1
Regex/String Parsing + Hash Set + Hash Map.	1
REGEXP_SUBSTR + Window Aggregation	1
Regular Expression (Regex) Matching. O(N)	1
Regular Expression (Regex) Parsing. O(N)	1
Regular Expressions (Regex)	1
Rejection Sampling (7x7 Matrix) Create a range of 1-49 using `7 * (rand7() - 1) + rand7()`. If the result is > 40, reject it. Otherwise, return `result % 10 + 1`.	1
Relative Parity / Math	1
Remainder Frequency Counting	1
Repeated Digit Extraction (Drawing the logic to pull digits from numbers inside the nested loops).	1
Repeated Function Execution Stack (Multiple redundant calls to the selector function).	1
Repeated String Concatenation (Visualizing memory churn from adding one char at a time to a string).	1
Reservoir Sampling	1
Reservoir Sampling (Memory Efficient)	1
Resource Hierarchy / Semaphores	1
Reverse Encryption Mapping (Trie/Hash)	1
Reverse Graph Traversal or Topological Sort.	1
Reverse In-order Traversal (Right -> Root -> Left).	1
Reverse In-Order Traversal (Right → Root → Left).	1
Reverse Linear Scan	1
Reverse Linear Scan (Right-to-Left)	1
Reverse Post-order Traversal (Right, Left, Root) maintaining a global `prev` pointer.	1
Reverse Processing + Union Find (DSU)	1
Reverse Recursive Math / Divide and Conquer	1
Reverse Scan + Hash Set	1
Reverse Search (King outward to Queens)	1
Reverse Simulation with Max-Heap	1
Reverse Time Union-Find (Disjoint Set).	1
Reverse Traversal + 32-Bit Position Tracking	1
Reverse Traversal + HashSet	1
Reverse Traversal / String Builder	1
Reverse Traversal Start from the end of the string. Collect K characters, add a dash, and repeat. Reverse the result at the end.	1
Reverse Trie (Suffix Trie) with Precomputed Min	1
Reverse Work-Backwards with Modulo.	1
Revert Half the Number	1
Right Max Array + Sorted Set	1
Right-Spine Insertion	1
Right-to-Left Array Reduce	1
Right-to-Left Carry Simulation (Greedy)	1
Right-to-Left Monotonic Scan.	1
Rolling Hash (Rabin-Karp) + Hash Set	1
Rolling Hash (Rabin-Karp) - Reversed.	1
Rolling Hash (Rolling Frequencies) + Trie.	1
Rook-Centric Scan	1
Row Max, Column Max, and Non-Zero Count.	1
Row Numbering / Window Functions.	1
Row Sorting + Column Maximums	1
Row Sorting + Global Priority Queue. O(N · M log M + NM log K)	1
Row-based Parity Processing + Array Slicing. O(M · N)	1
Row-by-Row Comparison Buffer (Visualizing the SQL engine re-scanning the bookings table for every single record).	1
Row-by-Row Memory Copy (Intermediate tables filling up Cartesian space).	1
Row-Pair Comparison (DP-lite)	1

Row-wise Iteration (Simple Scan)	1
Row/Column Frequency Counting	1
Row/Column Tally Arrays.	1
Row/Column/Diagonal Counters	1
Row/Column/Diagonal Sum Counters	1
Rule-Based Combinatorics (Independent Slots)	1
Running Max + Prefix Sum (Single Pass)	1
Running Maximum Comparison	1
Running Maximum Property	1
Running Total (Cumulative Frequency) Analysis.	1
Schoolbook Addition (Right-to-Left)	1
sed or tail + head .	1
Segment Tree	1
Segment Tree (DP State Merging)	1
Segment Tree (Dynamic) / Churning Map	1
Segment Tree (Max-Range) + Binary Search for First Index. O(N log N)	1
Segment Tree (Point Updates) tracking Max Subarray Sum or DP Prefix mapping. (O(N log N))	1
Segment Tree (Range Sum & Max)	1
Segment Tree + Balanced BST (Sorted Set)	1
Segment Tree / Balanced BST (TreeMap)	1
Segment Tree / Fenwick Tree	1
Segment Tree / Fenwick Tree (Range Maximum Query)	1
Segment Tree or Binary Indexed Tree (Fenwick Tree)	1
Segment Tree or Fenwick Tree (BIT) for O(N log N) LIS.	1
Segment Tree or Fenwick Tree with Lazy Propagation. Store XOR values in nodes and apply multiplications as "Lazy" updates.	1
Segment Tree or Square Root Decomposition where each "node" or "bucket" stores partial Trionic counts.	1
Segment Tree where each node stores a "Bit Count" array (size 32 or 64). Multiplication by a power of 2 becomes a "Bit Shift" on the array.	1
Segment Tree with Boyer-Moore Voting or Random Sampling + Range Binary Search (using a map of indices).	1
Segment Tree with Lazy Propagation	1
Segment Tree with Lazy Propagation + Sweepline	1
Segment Tree with Lazy Propagation.	1
SELECT DISTINCT + ORDER BY	1
Self Join with absolute difference = 1 OR Window Functions (LAG/LEAD).	1
Self-Anti-Join / WHERE NOT IN	1
Self-Balancing BST or Linked List (Ideal)	1
Self-Join / Aggregate CASE WHEN	1
Self-Join Delete.	1
Self-Join on Common Friends	1
Self-Join on Follower Column	1
Self-Join on ID = Reports_To + Group By.	1
Self-Join on ManagerId = Id .	1
Self-Join with Anti-Join (NOT IN)	1
Self-Join with DATEDIFF .	1
Self-Join with Inequality Constraints	1
Self-Join with ROW_NUMBER()	1
Semaphores + CyclicBarrier / Phasers	1
Sentinel Node + Two Pointers	1
Serialized Memory Wait (The closure for the addition waits twice).	1
Set Difference / Hash Map	1
Set Size vs. Half-Length Comparison.	1
Set-Based Adjacency List + DFS/BFS for Components. O(N^2 · M) (Optimized constant factor).	1
Set-Based Unique Value Counting	1
setTimeout + Promise Wrapper	1
Shrinking Bitwise Set	1
Shunting Yard Algorithm / Two Stacks (Operands & Operators)	1
Shunting-yard Algorithm / Stack	1

Sieve + Sliding Window / Prefix Sums.	1
Sieve of Eratosthenes	1
Sieve of Eratosthenes (Precomputation) + Linear Array Scan.	1
Sieve of Eratosthenes (precompute primes up to N) + Single Pass Prefix/Suffix sums.	1
Sieve of Eratosthenes (precompute primes) + Unbounded Knapsack Dynamic Programming (Coin Change variant).	1
Sieve of Eratosthenes + BFS with a "furthest reached" pointer to strictly avoid re-scanning identical prime gaps.	1
Sieve of Eratosthenes + Binary Search (Greedy)	1
Sieve of Eratosthenes + Combinatorics	1
Sieve of Eratosthenes + Digit Mask	1
Sieve of Eratosthenes + Disjoint Set Union (DSU) or DFS	1
Sieve of Eratosthenes + Math	1
Sieve of Eratosthenes + Prefix Prime Frequencies.	1
Sieve of Eratosthenes + Rolling Hash / DP.	1
Sieve of Eratosthenes + Sliding Window / Prefix Sums	1
Sieve of Eratosthenes + Two Pointers	1
Sieve of Eratosthenes / Harmonic Series Mapping. O(M log M + N)	1
Simple conditional logic: IF or CASE WHEN.	1
Simple DFS Recursion.	1
Simple Greedy Simulation	1
Simple Iteration (Small Constraints)	1
Simple Linear Scan	1
Simple WHERE clause with OR.	1
Simulation (Direction Flag) Use a direction flag (Up/Down). Calculate new i, j based on boundaries (e.g., if at last column, move down and flip direction).	1
Simulation (due to small N=100). O(N^2)	1
Simulation (Two-Pointer Logic)	1
Simulation + 3D State Tracking (r, c, direction)	1
Simulation + Hash Set	1
Simulation + Modular Exponentiation	1
Simulation + Prime Factorization	1
Simulation / Greedy	1
Simulation / Minute-by-Minute Increment.	1
Simulation or Mathematical Arithmetic Progression.	1
Simulation with Condition Validation	1
Simulation with Leftmost Greedy Search. O(N^2) is acceptable given N=100.	1
Simulation with Queue (Work Backwards/Index Mapping)	1
Simulation with Real Stack	1
Simulation with Step Increment (Greedy Walking).	1
Single Counter Tracking	1
Single Loop Math	1
Single Pass (Hash Sets)	1
Single Pass (Head Counting) Iterate through the grid. Only count an 'X' if it doesn't have an 'X' above it or to its left.	1
Single Pass (In-Place)	1
Single Pass (Running Sum and Count)	1
Single Pass (Shared Edge) Count land cells x 4. Subtract 2 for every shared edge (horizontal or vertical neighbor).	1
Single Pass (Space Detection)	1
Single Pass + Prev Pointer	1
Single Pass / Linear Indexing	1
Single Pass / Offset Comparison	1
Single Pass / State Tracking (Current Streak)	1
Single Pass Boolean Flag Scan.	1
Single Pass Bound Check. O(N)	1
Single Pass Bucketing (or SQL UNION/GROUP BY)	1
Single Pass Comparison (Pythagorean Theorem)	1
Single Pass Contraction Check.	1
Single Pass Count Iterate through characters. Count a segment whenever you see a non-space character that follows a space (or is at index 0).	1
Single Pass Counter (Greedy).	1
Single Pass Counter Iterate through the array. Increment `count` if 1; if 0, update `max` and reset `count`.	1

Single Pass Counting (Independent Moves)	1
Single Pass First-Occurrence Map	1
Single Pass Greedy Tracking	1
Single Pass Greedy Training	1
Single Pass Greedy Violation Detection	1
Single Pass Linear Scan	1
Single Pass Linear Scan / Prefix Sums.	1
Single Pass Matrix Traversal	1
Single Pass Max Tracking	1
Single Pass Max/Min Tracker (Greedy).	1
Single Pass Minimum Tracking.	1
Single Pass State Toggle	1
Single Pass State Tracking.	1
Single Pass Streaming Maximum (King of the Hill).	1
Single Pass Tokenization / On-the-fly Parsing	1
Single Pass Tracking (Max/Min Greedy)	1
Single Pass with Hash Map/Frequency Array.	1
Single Pass with Lookahead (In-Place)	1
Single Pass with Running Minimum (Prefix Min)	1
Single Pass with Three-Node Window + Edge Point Tracking	1
Single Pointer Iteration (count spaces and substring once).	1
Single Queue Rotation	1
Single-Pass Accumulation with Modulo Digit Extraction.	1
Single-Pass Accumulator / State Machine	1
Single-Pass Aggregation with CASE/IF statements	1
Single-Pass Array Mapping	1
Single-Pass Bottom-Up / Top-Down DFS	1
Single-Pass Character Counting.	1
Single-Pass Constant Space Scan	1
Single-Pass Early Exit	1
Single-Pass Filtering	1
Single-Pass Flagging	2
Single-Pass Frequency Array	1
Single-pass Frequency Array (size 26) + Max Frequency tracking.	1
Single-Pass Frequency Map.	1
Single-Pass Greedy Accumulation (Kadane's Variant).	1
Single-Pass Greedy Accumulation / Simulation.	1
Single-Pass Greedy Counter	1
Single-Pass Greedy Scan	1
Single-pass Hash Map / Frequency Array. Count all frequencies first, then perform a single pass to filter.	1
Single-Pass Index Mapping	1
Single-Pass Index Traversal (Step K)	1
Single-Pass Iteration / In-Place Bit Manipulation.	1
Single-pass iteration with a running bitwise OR variable and a parity check.	1
Single-Pass Level Accumulation	1
Single-Pass Linear Scan	2
Single-Pass Mathematical Extraction	1
Single-Pass Max & Second Max Tracking.	1
Single-Pass Parity Check	1
Single-Pass Pointer Distance	1
Single-Pass Pointer Scan	1
Single-Pass Range Intersection	1
Single-Pass Reverse Scan (Right-to-Left Pointer)	1
Single-Pass Scalar Tracking.	1
Single-pass Simulation using a Stack or a dynamically sized Array/StringBuilder.	1
Single-Pass Simulation with ASCII Math. O(N)	1
Single-Pass Stack (Precedence Handling)	1

Single-pass state machine or boolean flag tracking within a single iteration.	1
Single-Pass Two-Pointer (In-place if bits allow)	1
Single-Pass Two-Pointer / StringBuilder	1
Single-pass: Find the first and last occurrence of different words. The max distance is either LastIndex - FirstIndex or LastIndex - SecondFirstIndex .	1
Single-Pivot Circularity Check	1
Single/Double Pass Hash Map (Frequency Array)	1
Sliding Boundary Pointers	1
Sliding Window	2
Sliding Window (At Most K) Transformation.	1
Sliding Window (atLeast(k) - atLeast(k+1))	1
Sliding Window (Constant Space)	1
Sliding Window (Exact = AtMost(K) - AtMost(K-1))	1
Sliding Window (Expand/Contract)	1
Sliding Window (Expand/Contract) Use two pointers. Keep track of `maxFreq` in current window. If `windowSize - maxFreq > k`, shrink left.	1
Sliding Window (Expandable) Maintain a window with at most one '0'. If zeros > 1, shrink left pointer.	1
Sliding Window (Expanding Right, Shrinking Left)	1
Sliding Window (Fixed 2x2) / Counting	1
Sliding Window (Fixed Size)	2
Sliding Window (Fixed Size) / Prefix Sums	1
Sliding Window (Fixed Size).	2
Sliding Window (Fixed)	1
Sliding Window (Flexible)	1
Sliding Window (Hash Set)	1
Sliding Window (Inverse approach)	1
Sliding Window (Monotonic Property)	1
Sliding Window (Observation: minSize only)	1
Sliding Window (Optimised Summing)	1
Sliding Window (Queue)	1
Sliding Window (Shrinkable)	1
Sliding Window (String or Math)	1
Sliding Window (Target = TotalSum - x)	1
Sliding Window (Two Pointers + Map)	1
Sliding Window (Two Pointers)	6
Sliding Window (Two Pointers) Run Twice	1
Sliding Window (Two Pointers) with Hash Map.	1
Sliding Window (Two Pointers) with Sub-problem Logic	1
Sliding Window (Two Pointers).	1
Sliding Window (Two-Pointer / At-Most)	1
Sliding Window (Two-Pointers)	1
Sliding Window (Variable Size)	1
Sliding Window + 1D DP (Prefix Min)	1
Sliding Window + 2D Data Structure (Segment Tree/Fenwick Tree) or Sorted Set.	1
Sliding Window + Binary Trie	1
Sliding Window + Bit Count Array	1
Sliding Window + Deficit Counter	1
Sliding Window + Dynamic Programming (Prefix Max).	1
Sliding Window + Fenwick Tree / Segment Tree	1
Sliding Window + Frequency Array (Counting Sort)	1
Sliding Window + Frequency Array/Map.	1
Sliding Window + Frequency Array/Set.	1
Sliding Window + Frequency HashMap.	1
Sliding Window + Frequency Map	2
Sliding Window + Frequency Map (Fixed Size).	1
Sliding Window + Frequency Map (or Set)	1
Sliding Window + Frequency Map / Bitmask.	1
Sliding Window + Frequency Map Use an array of size 26. As the window moves, increment the "entering" char and decrement the "leaving" char.	1
Sliding Window + Hash Map (Frequencies)	1

Sliding Window + Hash Map (Two Pointers)	1
Sliding Window + Hash Map of Sets	1
Sliding Window + Hash Maps	1
Sliding Window + Hash Set	2
Sliding Window + Hash Set (or Rolling Hash)	1
Sliding Window + Hash Set (or Rolling Hash).	1
Sliding Window + Mathematical Logic.	1
Sliding Window + Median Distance + Prefix Sums	1
Sliding Window + Monotonic Deque	1
Sliding Window + Monotonic Deques	1
Sliding Window + Monotonic Deques (Min/Max Tracking)	1
Sliding Window + Ordered Set / Hash Map	1
Sliding Window + Prefix Sums	2
Sliding Window + Rabin-Karp / Trie	1
Sliding Window + Running Medians (Two Heaps).	1
Sliding Window + Segment Splitting	1
Sliding Window + Two Balanced BSTs (Heaps)	1
Sliding Window + Two Heaps (Min/Max) to maintain dynamic median. O(N log K)	1
Sliding Window + Two Heaps (or Balanced BST)	1
Sliding Window / DP	1
Sliding Window / Greedy.	1
Sliding Window / Hash Map	1
Sliding Window / Hash Set.	1
Sliding Window / Monotonic Queue.	1
Sliding Window / Prefix Sum Array	1
Sliding Window / Prefix Sum Hashing	1
Sliding Window / Randomized Search	1
Sliding Window / Regex Pattern Matching. O(N)	1
Sliding Window / Sweep Line	1
Sliding Window / Two Pointers with Bitmask Running OR	1
Sliding Window DP.	1
Sliding Window Maximum	1
Sliding Window on concatenated string (s + s).	1
Sliding Window on Digit String "123456789"	1
Sliding Window over an array of indices + Median greediness.	1
Sliding Window over Gap Arrays. O(N)	1
Sliding Window with a single counter for "duplicate pairs".	1
Sliding Window with Frequency Map.	1
Sliding Window with Hash Map (Frequency Counter).	1
Sliding Window with Min-Heap.	1
Sliding Window with Monotonic Deque OR Segment Tree (depending on exact stability constraint).	1
Sliding Window: exact(k) = atLeast(k) - atLeast(k+1)	1
Slow/Fast Pointers + In-Place Reversal of Second Half	1
Sort & Interleave (Wiggle Sort)	1
Sort + Anchor & Squeeze	2
Sort + Backtracking (Sibling Skip)	1
Sort + Backtracking (Skip Dupes)	1
Sort + Backtracking (Used Array / Skip)	1
Sort + Binary Search (Two Pointers)	1
Sort + Binary Search LIS	1
Sort + Binary Search on Answer + Greedy	1
Sort + De-duplicate + Hash Map	1
Sort + Deduplicate + Sliding Window (Two Pointers)	1
Sort + Greedy (Adjacent Triplet)	1
Sort + Greedy Linear Scan	1
Sort + Insert Sort descending by height (h), then ascending by (k). Insert people into a list at index k.	1
Sort + Longest Increasing Subsequence (LIS) via Binary Search	1

Sort + Prefix Check (or Trie)	1
Sort + Tracking Next Available	1
Sort + Two Anchors + Squeeze	1
Sort + Two Pointers (Greedy)	1
Sort + Two Pointers (Left/Right)	1
Sort + Two Pointers.	1
Sort Both Arrays + Sequential Pairing. O(N log N)	1
Sort Buffer (Visualize the O(N) or O(log N) extra space used by sorting).	1
Sort Buffer (Visualizing the O(N) memory used to rearrange the array).	1
Sort Buffer (Visualizing the O(N) or O(log N) extra space used by the sorting algorithm).	1
Sort by End Time + Binary Search + 2D Dynamic Programming (dp[i][quota]). (O(N log N))	1
Sort by Start (Asc) and End (Desc) + Greedy Scan	1
Sort Dictionary + Two Pointers.	1
Sort edges by time, use Disjoint Set Union (DSU). Start with C=V components. Decrement C for every successful union. Stop when C == K.	1
Sort Intervals + Sliding Window snapped to Interval Boundaries (Two passes: Left/Right). (O(N log N))	1
Sort OR Linear Scan for Top 3 and Bottom 2.	1
Sort the array + Binary Search on the Answer Space + Greedy Checking.	1
Sorted Time Processing + DSU with Reset	1
Sorting & Median Finding	1
Sorting (Attack Desc, Defense Asc) + 1-Pass Greedy	1
Sorting (by Ratio) + Max-Heap (for Quality).	1
Sorting + 1D Dynamic Programming (Knapsack variant)	1
Sorting + 2D Dynamic Programming (or Max-Heap Greedy).	1
Sorting + Adjacent Comparison	1
Sorting + Adjacent Comparison OR Trie Insertion. O(N log N · L)	1
Sorting + Adjacent Swaps	1
Sorting + Arithmetic Progression.	1
Sorting + Binary Search	1
Sorting + Binary Search (By Height)	1
Sorting + Binary Search (Prefix Sums).	1
Sorting + Binary Search (Range Check)	1
Sorting + Binary Search + Suffix Max	1
Sorting + Binary Search on Answer Space.	1
Sorting + Combinatorics + Hash Map	1
Sorting + Comparison Math	1
Sorting + Contribution of Each Element.	1
Sorting + Contribution Technique + Combinatorics. O(N log N)	1
Sorting + Cross-Multiplication	1
Sorting + DP (Longest Increasing Subsequence Variant)	1
Sorting + DP on Chains	1
Sorting + Dynamic Programming	1
Sorting + Dynamic Programming (Prefix Sum)	1
Sorting + Dynamic Programming + Binary Search	1
Sorting + Dynamic Programming / Two Pointers	1
Sorting + Fenwick Tree (Binary Indexed Tree)	1
Sorting + Fixed-Size Sliding Window	1
Sorting + Frequency Counting	1
Sorting + Frequency Map (or Queue)	1
Sorting + Gap Calculation	1
Sorting + Greedy (or 1D DP)	1
Sorting + Greedy (Sliding Window)	1
Sorting + Greedy / Two-Pass Max Search.	1
Sorting + Greedy Accumulation	2
Sorting + Greedy Accumulation.	1
Sorting + Greedy Allocation	1
Sorting + Greedy Alternating Distribution.	1
Sorting + Greedy Boundary Check	1

Sorting + Greedy Candidate Validation.	1
Sorting + Greedy Counting.	1
Sorting + Greedy Extremes	1
Sorting + Greedy Matching	1
Sorting + Greedy Pairing.	1
Sorting + Greedy Pass (or Counting Sort for O(N)).	1
Sorting + Greedy Row/Col Max Tracking	1
Sorting + Greedy Subsequences	1
Sorting + Greedy Swapping	1
Sorting + Greedy Target Matching	1
Sorting + Greedy Two-Pointer (Split Search).	1
Sorting + Greedy Two-Pointer.	1
Sorting + Grouping (Disjoint Connectivity)	1
Sorting + Linear Interval Merge	1
Sorting + Linear Scan	1
Sorting + Math (Prefix Sum Logic).	1
Sorting + Merge Intervals + Modular Exponentiation.	1
Sorting + Merging Intervals	1
Sorting + Min-Heap (Sliding Window of Top K).	1
Sorting + Monotonic Arrival Times.	1
Sorting + Parity Segregation	1
Sorting + Prefix Maximum + Binary Search	1
Sorting + Prefix Sum + Binary Search	1
Sorting + Prefix Sum + Two Pointers/Binary Search	1
Sorting + Prefix Sums + Binary Search	2
Sorting + Prefix Sums + Binary Search (Two-Pointer Optimization)	1
Sorting + Single Pass Optimization	1
Sorting + Sliding Window (Fixed Size 3)	1
Sorting + Sliding Window (Fixed Size K)	1
Sorting + Sliding Window (Queue)	1
Sorting + Sliding Window + Prefix Sums	1
Sorting + Sliding Window with Min-Heap of size K. O(N log N + N log K)	1
Sorting + Sliding Window.	1
Sorting + Small Constant Search	1
Sorting + Sweeping Max-Y	1
Sorting + Two Pointers	5
Sorting + Two Pointers (Greedy)	1
Sorting + Two Pointers (Greedy).	2
Sorting + Two Pointers (or Binary Search).	1
Sorting + Two Pointers + Backtracking	1
Sorting + Two-pass scan (Left-to-Right & Right-to-Left).	1
Sorting + Two-Pointer	1
Sorting + Window Functions (Lead/Lag) or Inequality Joins.	1
Sorting / 1D Greedy Sweep (Assuming 1D coordinate plane).	1
Sorting / Binary Search on Answer	1
Sorting / Frequency Map + Suffix Sum	1
Sorting / Window Functions (LEAD/LAG).	1
Sorting by Parity / Max Heap	1
Sorting by Start Time	1
Sorting by Value + Sorting by Index	1
Sorting Comparison or Counting Sort.	1
Sorting Events + Lazy Counter for "ALL" mentions. O(M log M + N)	1
Sorting or Bucket Sort (Boolean Array).	1
Sorting Pair Sums (Boundary Logic).	1
Sorting Rows	1
Sorting with Index Tracking + Boolean Marking.	1
Sorting with Index Tracking / Max-Heap.	1

Sparse Table + Decreasing GCD Property.	1
Sparse Table / GCD Property (Unique Prefix GCDs)	1
Spatial Hashing / Grid-based Priority Queues.	1
Sprague-Grundy Theorem (Bitwise XOR / Nim-Sum)	1
SQL `CASE` statements with `LIKE` and `REGEXP` for vectorized pattern matching. O(N)	1
SQL `GROUP BY` with Aggregate Function `MIN()`.	1
SQL Aggregation / JOIN	1
SQL Aggregation: `GROUP BY` + `COUNT()`.	1
SQL Aggregation: `GROUP BY` on multiple columns.	1
SQL Common Table Expression (CTE) with Filtering and Grouping. O(N)	1
SQL Common Table Expressions (CTE) + SUM() Aggregate	1
SQL DATE_FORMAT() function.	1
SQL GROUP BY + COUNT(DISTINCT)	1
SQL GROUP BY + String Aggregation + HAVING	1
SQL IS NULL	1
SQL JOIN + GROUP BY + HAVING	1
SQL Join + String Aggregation.	1
SQL JOINs + Window Functions (RANK)	1
SQL LEAST() and GREATEST() Functions	1
SQL Left Outer Join.	1
SQL Length Function (LENGTH/CHAR_LENGTH)	1
SQL LIKE Operator / String Matching	1
SQL Logical Indexing (`AND` condition).	1
SQL Recursive CTE + Anti-Join	1
SQL REGEXP_REPLACE / Nested String Functions	1
SQL Self-Join with Inequality Constraints.	1
SQL String Aggregation (`GROUP_CONCAT` / `STRING_AGG`).	1
SQL String Functions (CONCAT , LEFT) running directly on the database engine.	1
SQL String Functions (INITCAP / Regex)	1
SQL UNION ALL (Unpivoting).	1
SQL UPDATE + CASE WHEN	1
SQL Window Function (Cumulative Sum)	1
SQL Window Function: `LEAD()` over Partition.	1
SQL Window Functions (`RANK()` or `DENSE_RANK()`)	1
SQL Window Functions (Cumulative Sum/Running Total) + CTEs	1
SQL Window Functions (LAG/LEAD) / Pandas `groupby().rolling()`.	1
SQL Window Functions (RANK / DENSE_RANK)	1
SQL Window Functions + Common Table Expressions (CTEs)	1
Square of Prime Property	1
Square Root Decomposition (Block separation).	1
Square Root Decomposition or Fenwick Tree.	1
Square Root Divisor Hunt	1
Square Root Factor Pairing	1
Square Root Factorization + Dynamic Programming. Only check up to √N to find all factors, then use DP to find balanced subsets.	1
Square Root Search (Greedy)	1
Square-Free Part Grouping	1
Stable Sort / Map of Lists	1
Stack (Greedy Elimination)	1
Stack (Greedy Simplification).	1
Stack (Iterative Evaluation)	1
Stack (LIFO processing).	1
Stack (LIFO)	2
Stack (LIFO) / Two Pointers	1
Stack (Push/Pop on Chunks)	1
Stack (Right-to-Left) Scan the string backwards. If you hit a `?`, pop the next two values from the stack and push the result based on the condition.	1
Stack (Simulation of BODMAS/PEMDAS).	1
Stack / Amortized Character Matching	1

Stack / Amortized Sliding Window	1
Stack / Array Depth Tracking	1
Stack / Integer Counter	1
Stack / Previous-Word Pointer	1
Stack / Two-Pass Filtering	1
Stack Data Structure.	1
Stack Frame Explosion (Visualizing a call stack N-levels deep repeated billions of times).	1
Stack or "Core Value" Summation.	1
Stack Simulation	1
Stack with Frequency Pairs	1
Stack-Based Character Matching	1
Stack-based Evaluation.	1
Stack-based Expression Parsing + Dynamic Programming (bottom-up costs).	1
Stack-based Interval Calculation.	1
Stack-based Iterative Construction.	1
Stack-Based Iterative Parsing	1
Stack-based Parsing + State Machine.	1
Stack-Based Parsing / Grammar state machine	1
Stack-based Pattern Matching	1
Stack-Based Reversal Use a stack to push 1, 2, 3 dots If you see 'I', pop everything to the result. If you see 'D', keep pushing.	1
Stack-Based Segment Reversal	1
Stack-Based Single Pass with Sign Tracking	1
Stack-based Single Pass.	1
Staircase Search (Saddleback)	1
Staircase Search (Top-Right Start)	2
Standard BFS/DFS Connected Components. Accumulate a `running_sum` during the traversal. Modulo K at the very end of the island.	1
Standard Binary Search	1
Standard Dijkstra's Algorithm using an Adjacency List and a Min-Heap (Priority Queue).	1
Standard Library formatting (bin/format)	1
Standard Simulation (Loop) or Math (Square root approach).	1
Standard Trie with wordCount and prefixCount variables at each node.	1
Standardization via Pre-filtering + Template Mapping.	1
Start at Square Root Start W at lfloor√{N}rfloor and decrement until N % W == 0. This ensures L and W are as close as possible.	1
State Array Snapshots (Drawing the entire array state for every microscopic time increment).	1
State Encoding (Bitmask) + Hash Map	1
State Machine / 1D Dynamic Programming	1
State Machine / Concurrent Counters	1
State Machine / Deterministic Parsing	1
State Machine / Single Counter tracking contiguous valid states.	1
State Machine DP / Greedy	1
State Machine DP / Left-Right Precomputation.	1
State Machine DP / Math	1
State Tracking (Depth Counter).	1
State Tracking (Parity / Flip Count)	1
State Tracking / Greedy	1
State Transition Counting (Adjacent Differences)	1
State-Layered Dijkstra (BFS with 2-Layer States).	1
State-Space Dijkstra's Algorithm. State = (Node, Arriving_Direction).	1
Stateful Counter + Hash Map	1
Static Array Memory (Visualizing a large block of heap memory occupied by fixed integers).	1
String Aggregation (GROUP_CONCAT)	1
String Builder / Pre-allocated Array	1
String Builder / Two Pointers (In-place if allowed).	1
String Chunking / Substring Math.	1
String Comparison (In-place `startsWith`).	1
String Comparison / Line Overlap	1
String Concatenation (Better for scale/many conditions). Append "Fizz" if mod 3, append "Buzz" if mod 5. Output if empty.	1

String Concatenation (Double Check)	1
String Concatenation + Substring functions	1
String Concatenation Buffer (Visualizing the creation of 6 massive new strings in the heap).	1
String Doubling Trick Concatenate S+S. Remove the first and last characters. If S is still a substring of the result, it has a repeated pattern.	1
String Functions + Aggregation	1
String Hashing + Center Expansion.	1
String Iteration / Character Counting.	1
String Iteration / Simulation	1
String Manipulation / Math	1
String Mutation Buffer (Visualizing repeated string recreations for each swap in languages with immutable strings).	1
String Parsing + Hash Map (Frequency Counter).	1
String Parsing + HashSet	1
String Parsing with Strict Validation Pruning.	1
String Predicate Filtering	1
String Slice Memory (Draw many temporary substring copies being created in the heap).	1
String Slicing / Cyclic Shift	1
String Split + Bucket Array mapping.	1
String Splitting (by '#') + Segment Matching	1
String Splitting + Hash Map	1
StringBuilder / String Join with Index-based Suffix.	1
Subarray Buffer (Visualizing the heap filling with transient slices of the input array).	1
Subquery / CTE with Date Arithmetic.	1
Subquery / NOT IN clause.	1
Subquery Grouping with HAVING count = 1.	1
Subquery with In-Clause or Window Functions.	1
Subquery with LIMIT 1 OFFSET 1 wrapped in IFNULL .	1
Substring Slicing + Direct Integer Comparison	1
SUBSTRING_INDEX / Regex + Group By	1
Suffix Array Traversal (Right-to-Left Scan).	1
Suffix Automaton (SAM) or Rolling Hash	1
Suffix Greedy Search (Targeting "00", "25", "50", "75")	1
Suffix Sum of Shifts.	1
Suffix Trie (Inverted Prefix Tree).	1
SUM(CASE) / AVG(IF)	1
Sweep Line + Segment Tree	1
Sweep Line + Segment Tree (with Lazy Propagation/Coverage Count). O(N log N)	1
Sweep Line / Difference Array	1
Sweep Line Algorithm (TreeMap)	1
Sweep Line Algorithm + Max Heap / TreeMap	1
Symmetric Construction (Pairing)	1
Symmetry Grouping (2x2 Blocks + Centroids)	1
Synchronization (Mutex/Locking)	1
Synchronization (Semaphores / Mutex + Conditions)	1
Synchronized DFS / BFS	1
Target Index Mapping + Longest Increasing Subsequence (LIS) via Patience Sorting.	1
Targeted Filter + Distinct (SQL)	1
Tarjan's / Bridges (DFS Discovery Time)	1
Temporal State Snapshots (Visualizing the position of every robot at every tick of time).	1
Temporary Join Tables (Visualizing the SQL engine creating massive intermediate tables for each join).	1
Temporary Object Accumulation (Creating massive amounts of intermediate merged objects).	1
Temporary Property Allocation (Mutation of the target object).	1
Temporary Reversed String Buffers (Creating a reversed copy for every comparison).	1
Temporary String Buffers (Visualizing multiple string concatenations and copies).	1
The Circular Link (Ring)	1
The Reversal Algorithm.	1
Thread Pool + Thread-Safe Queue/Set	1
Three Pointers	1

Three Pointers (Backward Fill)	1
Three Pointers (Dynamic Programming)	1
Three-Case Greedy Analysis	1
Three-Phase Linear Scan	1
Three-Pointer Simulation (Single-Pass Result Array).	1
Three-Pointer Sliding Window (Tracking Indices)	1
Tokenization + `startsWith`	1
Top-3 Largest Gaps + Prefix/Suffix Max Gaps. O(N)	1
Top-3 Length Tracking per Character	1
Top-3 Row Candidates + Backtracking	1
Top-3 Variables Keep `first`, `second`, `third` initialized to -Infinity. Update them sequentially as you iterate O(N).	1
Top-Down DFS (Tree Path Sum)	1
Top-Down DFS with State Passing	2
Topological Sort (DFS with 3 States or Kahn's BFS)	1
Topological Sort (Kahn's Algorithm / BFS)	2
Topological Sort (Kahn's Algorithm)	1
Topological Sort (Kahn's Algorithm) / BFS.	1
Topological Sort (Kahn's or DFS)	1
Topological Sort (Kahn's) + Dynamic Programming	1
Topological Sort (Kahn's) + Dynamic Programming.	1
Topological Sort (Kahn's) Build a graph of dependencies. Check if at every step there is exactly 1 node with an in-degree of 0.	1
Topological Sort (Kahn’s) or Memoized DFS.	1
Topological Sort / BFS from Leaves (Peeling the Onion)	1
Topological Sorting (Peeling leaves like an onion)	1
Tortoise and Hare (Slow & Fast Pointers).	1
Total Cost - Max Character Frequency Cost	1
Transpose + Reverse Rows	1
Traveling Salesperson (TSP) with Bitmask DP	1
Tree Centroid Decomposition / Subtree Weight Sums.	1
Tree DFS/BFS Depth Mapping + Frequency Hashing + Mathematical Convolution/Lookup.	1
Tree Diameter Endpoints (2/3 DFS Passes)	1
Tree DP (Bottom-Up State)	1
Tree DP (Post-order Traversal)	2
Tree DP (Re-rooting Technique).	1
Tree DP (Rerooting / 2-Pass DFS)	1
Tree DP + Centroid Decomposition / Contribution of Edges	1
Tree DP + Frequency Map (BFS/DFS)	1
Tree DP + Memoization (State Reduction via Bit Shifts)	1
Tree DP / Re-rooting Technique	1
Tree DP with Combinatorics	1
Tree DP with Greedy Sorting	1
Tree DP with Rerooting (Change of Root).	1
Tree Dynamic Programming (Bottom-Up).	1
Tree Dynamic Programming (House Robber Style)	1
Tree Dynamic Programming (Kadane's on Trees)	1
Tree Dynamic Programming (Min Cost to Satisfy)	1
Tree Dynamic Programming (Post-order Traversal)	1
Tree Dynamic Programming (Rerooting or Diameters).	1
Tree Partitioning / Subtree Counting	1
Treemap (Sorted Dictionary)	1
TreeMap + Max-Heap (or Sorting).	1
TreeMap / Frequency Map + Smallest Element Processing.	1
TreeMap / Ordered Map (Next/Prev Greater Element)	1
TreeMap of Sorted Lists (Coordinate Grouping)	1
Trie (Prefix Tree)	1
Trie (Prefix Tree) + DFS Backtracking for '.'	1

Trie (Prefix Tree) + DFS.	1
Trie (Prefix Tree) + DFS/BFS	1
Trie (Prefix Tree) + Min-Heap/Caching.	1
Trie (Prefix Tree) / Hash Map	1
Trie (Prefix Tree) Node Structure	1
Trie (Prefix Tree) with Frequency Counters	1
Trie (Prefix Tree) with Reversed Search.	1
Trie (Prefix Tree) with Wildcard/Parameter Support.	1
Trie (Prefix Tree).	2
Trie (Prefix/Suffix) or Hashing with Word Splitting	1
Trie + Backtracking DFS on Grid	1
Trie + DFS/Memoization Sort words by length. Insert into a Trie. For each word, use DFS to see if it can be formed by existing Trie words.	1
Trie + Dynamic Programming	1
Trie + Floyd-Warshall + DP	1
Trie + Merkle Hashing (Post-order)	1
Trie of Combined Strings (suffix + { + prefix)	1
Trie with DFS / Generalized Neighbors.	1
Trie with Pairwise Character Keys	1
Trie with Weight Accumulation.	1
Trie-Based Backtracking	1
Trie-Based Backtracking Use a Trie to instantly find all words that start with a specific prefix (the prefix being the current column's characters).	1
Trie-based Directory Structure.	1
Trie-Based Dynamic Programming	1
Trillions of Digit Arrays (Visualizing the CPU bottleneck from converting numbers to strings repeatedly).	1
Trillions of temporary string allocations for digit extraction.	1
Trust Score Array (In-Degree vs Out-Degree)	1
Two Counters (Left/Right Passes)	1
Two Deques (Balanced Halves)	1
Two Dummy Nodes (Merging Streams)	1
Two Hash Maps (Double Tally)	1
Two Hash Maps + Min-Heap (Sorted Set)	1
Two Hash Maps + Priority Queues (Sorted Sets)	1
Two Hash Maps / Frequency-of-Frequency Tracking.	1
Two Hash Maps / One Map + Indexing	1
Two Hash Maps / Two Frequency Arrays	1
Two HashSets (Unique Intersection)	1
Two Heaps (Balance) / TreeSet Use a Max-Heap for the left half and a Min-Heap for the right half. Use "Lazy Removal" with a Hash Map to handle elements leaving the window.	1
Two Heaps (Median) + Hash Map & Treap/SortedSet (Mode)	1
Two Heaps (Min-Heap & Max-Heap)	1
Two Heaps (Min-Heap / Max-Heap) or Balanced BST.	1
Two Heaps (Min-Heap for Capital, Max-Heap for Profit) / Greedy.	1
Two Heaps (Priority Queues) + Prefix/Suffix Sums.	1
Two Heaps (Priority Queues) / TreeSet	1
Two Independent BFS/DFS Passes	1
Two Min-Heaps (Priority Queues)	2
Two Min-Heaps (Priority Queues): One for unused rooms, one for engaged rooms (endTime, roomIdx).	1
Two Monotonic Stacks + Priority Queue	1
Two Passes (Prefix and Suffix) using Max-Heaps of size K.	1
Two Pointers (Expanding from k).	1
Two Pointers (Fast & Slow Ruler Slide).	1
Two Pointers (Generator/Consumer) Use an `int[]` or `char[]` and two pointers: `head` (reading how many) and `tail` (writing what's next).	1
Two Pointers (Gravity logic) + Standard Matrix Rotation.	1
Two Pointers (Greedy Matching)	1
Two Pointers (Greedy Simulation).	1
Two Pointers (Greedy Squeeze)	1
Two Pointers (Greedy Subsequence Match)	1
Two Pointers (Greedy)	2

Two Pointers (Greedy).	1
Two Pointers (In-place Swap)	3
Two Pointers (In-Place Write)	1
Two Pointers (In-place)	2
Two Pointers (Inward Convergence)	1
Two Pointers (Inward Convergence).	1
Two Pointers (Inward Meeting)	1
Two Pointers (Inward Search)	1
Two Pointers (Inward Squeeze)	1
Two Pointers (Left & Right) converging to center, using a Hash Set for vowels.	1
Two Pointers (Left and Right).	1
Two Pointers (Linked List Intersection Style)	1
Two Pointers (Marker Placement)	1
Two Pointers (Merge Sort Style)	1
Two Pointers (Nested Indexing)	1
Two Pointers (One-Pass).	1
Two Pointers (Optimal Suffix Comparison)	1
Two Pointers (Optimized Prefix/Suffix)	1
Two Pointers (Outside-In) + Middle Checking	1
Two Pointers (Parabolic Extremity)	1
Two Pointers (Parallel Pointers).	1
Two Pointers (Parallel Scan)	1
Two Pointers (Prefix/Suffix Scan)	1
Two Pointers (Prefix/Suffix)	1
Two Pointers (Read & Write)	1
Two Pointers (Read/Write) Use `read` and `write` pointers. `read` finds the end of a group; `write` updates the array in-place.	1
Two Pointers (Reader/Writer)	1
Two Pointers (Relative Order & Constraints)	1
Two Pointers (Reverse Scanning).	1
Two Pointers (Right to Left) Traverse both strings backwards. Add digits plus carry. Append to StringBuilder and reverse.	1
Two Pointers (Sequential Match)	1
Two Pointers (Sliding Window) + Arithmetic Progression.	1
Two Pointers (Sliding Window) + Frequency Map.	1
Two Pointers (Slow/Fast with Offset)	1
Two Pointers (Slow/Fast) Use Floyd's Cycle-Finding algorithm. A fast pointer moves 2 steps, slow moves 1. Check for direction changes or single-node loops.	1
Two Pointers (Sort + Shrink).	1
Two Pointers (Sorted Array Property).	1
Two Pointers (Sorted) or Hash Map (Frequency)	1
Two Pointers (Start and End of Run).	1
Two Pointers (String validation).	1
Two Pointers (Subsequence Match with Constraints).	1
Two Pointers (Vowel-Selective Swap)	1
Two Pointers + Dummy Node	1
Two Pointers + Greedy Backtrack	1
Two Pointers + Greedy Sort both arrays. Give the smallest cookie that satisfies the child with the smallest greed factor.	1
Two Pointers + Hash Set (skip leading zeros directly).	1
Two Pointers + Mirror Map	1
Two Pointers + Pre-allocated Array/StringBuilder.	1
Two Pointers + Prefix Min/Max Tracking	1
Two Pointers + Prefix Sum	1
Two Pointers + String Builder	1
Two Pointers / Cyclic Replacement or Sub-segment Reversal Trick.	1
Two Pointers / Deque (Match from front, match from back).	1
Two Pointers / Greedy (Counting 1s)	1
Two Pointers / Greedy Prefix Sum	1
Two Pointers / Hash Map (First & Last Occurrence)	1
Two Pointers / Linear Comparison (Shift checking).	1

Two Pointers / Math (Sum of N)	1
Two Pointers / Maximum Overlap	1
Two Pointers / Mirror Matching	1
Two Pointers / Run-Length Encoding (RLE) grouping.	1
Two Pointers / Single-Pass Array Traversal checking alternating non-zero boundary states.	1
Two Pointers / Single-Pass Sliding Window.	1
Two Pointers / Single-Pass String Check	1
Two Pointers / Sliding Window	1
Two Pointers / Stack-based simulation.	1
Two Pointers / String Trimming	1
Two Pointers / StringBuilder / RegEx	1
Two Pointers / Suffix Analysis / Greedy Char Selection.	1
Two Pointers / Tracking Min & Max limits	1
Two Pointers In-Place (Symmetry Swap).	1
Two Pointers on Compressed Pairs	1
Two Pointers on compressed segments.	1
Two Pointers on Difference Array.	1
Two Pointers on Monotonic Surface	1
Two Pointers on Sorted Degrees + Shared Edge Subtraction.	1
Two Pointers on Sorted Intervals	1
Two Pointers or Fermat's Theorem.	1
Two Pointers Pointer `i` traverses the word, `j` traverses abbreviation. Parse numbers to skip `i` forward.	1
Two Pointers with "Skip" Flag. O(N)	1
Two Pointers with One-Time Divergence.	1
Two Priority Queues (Available Heap, Busy Heap).	1
Two Priority Queues (Min-Heap for Sell, Max-Heap for Buy).	1
Two Queues (Enter and Exit) with State Tracking.	1
Two Queues (R_index, D_index).	1
Two separate Binary Searches (Upper Bound for negatives, Lower Bound for positives) leveraging the sorted nature of the array.	1
Two Stacks (Amortized O(1) - "Lazy Transfer")	1
Two Stacks (Count Stack & String Stack)	1
Two Stacks (Data Stack + Min Stack) or One Stack with Pairs.	1
Two Stacks (Predecessor & Successor)	1
Two Stacks / Gap Buffer	1
Two Stacks or Doubly Linked List	1
Two Stacks Push all nodes of both lists onto two stacks. Pop from both to add from right-to-left.	1
Two Variables / Counting Consecutive Blocks	1
Two-Pass / Prefix Tracking	1
Two-Pass BFS (Level Sums)	1
Two-Pass DFS (Time Map + Profit DFS)	1
Two-Pass Distance Arrays OR Two Pointers (Gap Tracking).	1
Two-Pass Force Calculation or Two-Pointer Segmenting.	1
Two-Pass Greedy (Left-to-Right and Right-to-Left).	1
Two-Pass Greedy (Max Tracking)	1
Two-Pass Hash Set (Prefix/Suffix)	1
Two-Pass Inclusion/Exclusion (At-Most K)	1
Two-Pass Linear Traversal (Head-to-Tail)	1
Two-Pass Min-Sweep (Left-to-Right + Right-to-Left).	1
Two-Pass Monotonic Logic (Min/Max Tracking).	1
Two-Pass Precomputation OR One-Pass State Machine.	1
Two-Pass Prefix and Suffix Sum.	1
Two-Pass Prefix/Suffix Accumulation	1
Two-Pass Prefix/Suffix Matching + Sliding Window.	1
Two-Pointer "Climb"	1
Two-Pointer (In-Place).	1
Two-Pointer (Sentinel + Last Non-9)	1
Two-Pointer / Boundary Tracking (Min and Second Min). O(N + M)	1

Two-Pointer / Bucket Splitting	1
Two-Pointer / Greedy Counting	1
Two-Pointer / Max Tracking	1
Two-Pointer / Sliding Boundary	1
Two-pointer approach or Hash Map tracking based on the property target - (arr[i] + arr[j]) .	1
Two-Pointer Array with Modulo Arithmetic.	1
Two-Pointer Binary Search	1
Two-Pointer Bitwise Addition	1
Two-Pointer Greedy / Sorting.	1
Two-Pointer Greedy Matching	1
Two-Pointer Greedy Replacement	1
Two-Pointer Greedy Scan or Edit Distance variant. If only specific operations are allowed (like moving to front), focus on the "Longest Matching Suffix".	1
Two-Pointer Greedy Simulation	1
Two-Pointer Greedy Swapping (Bubble Swaps for target positions).	1
Two-Pointer In-Place Re-linking	1
Two-Pointer In-place Reverse (or String Builder).	1
Two-Pointer In-Place Swap	1
Two-Pointer In-Place Swap (Jumping by 2k).	1
Two-Pointer In-Place Swap.	1
Two-Pointer Index Tracking (One Pass)	1
Two-Pointer Integer Parsing.	1
Two-Pointer Iteration + Mathematical Concatenation.	1
Two-Pointer Iterators (Row & Col)	1
Two-Pointer Jump (Step of 2)	1
Two-Pointer Meeting Point + Distance Logic.	1
Two-Pointer Merging (Sorted Property).	1
Two-Pointer Position Invariant	1
Two-pointer recursive DFS (or BFS with queue).	1
Two-Pointer Ring Traversal	1
Two-Pointer Sync (Head Switching).	1
Two-Pointer Technique (Meeting in the center).	1
Two-way expansion from 'start' index.	1
Two-Way Summation (Circular Array)	1
Unbounded Call Stack (Deep recursion for every valid path permutation).	1
Unbounded Integer Accumulator (Memory usage of the counter variable growing as it fails to terminate).	1
Unbounded Knapsack + Case Addition	1
Unbounded Step Counter.	1
Unified Hash Map (Frequency Counting).	1
UNION ALL + CTE / Subquery	1
UNION ALL + GROUP BY.	1
UNION ALL + Unified GROUP BY (CTEs)	1
UNION ALL + Window Functions	1
UNION ALL followed by GROUP BY and ORDER BY LIMIT 1.	1
UNION ALL for Event Splitting + GROUP BY	1
UNION ALL within a CTE + Group By.	1
UNION CTE + Window/Aggregate	1
Union Find (DSU) / Greedy	1
Union Find with Path Compression	1
UNION of two filtered sets.	1
Union-Find (Connected Components)	2
Union-Find (Consistency Check)	1
Union-Find (Disjoint Set Union)	1
Union-Find (Disjoint Set Union).	1
Union-Find (Disjoint Set)	1
Union-Find (Disjoint Set) with Dynamic Group Tracking.	1
Union-Find (Disjoint Set) with Math Multiples	1
Union-Find (DSU) with Component Counting.	1

Union-Find (DSU) with Path Compression.	1
Union-Find + Prime Factorization	1
Union-Find + Sieve of Eratosthenes	1
Union-Find + Topological Sort (Indegree)	1
Union-Find / Greedy Coloring + LCP Verification.	1
Union-Find or Boundary Mapping	1
Union-Find with Component Size Tracking + Adjacency Map.	1
Union-Find with Optimized Similarity Check.	1
Union-Find with Weights / Directed Graph DFS	1
Unique Char Counting Observe unique identifiers: 'z' is only in zero, 'w' in two, 'x' in six. Subtract counts in a specific order.	1
Unique Positive Integers (Hash Set)	1
Unix Pipeline (cat | tr | sort | uniq -c | sort -nr).	1
Update-History Log (Inverse Search)	1
Upper Bound Estimation + String Search.	1
Value Overwriting / "Node Identity Theft"	1
valueOf() and toString() Overriding	1
Variable Sliding Window / State Counting	1
Variable-Size Sliding Window (Caterpillar Method).	1
Vector Math / State Check.	1
Vectorized Assignment: df['bonus'] = df['salary'] * 2	1
Virtual 1D Array + Hash Map mapping (Fisher-Yates Shuffle).	1
Visited Node HashSet (Visualizing extra memory to store every node ID to prevent infinite loops).	1
Visited Set / In-Place Marking.	1
Weekday Function (SQL CASE/WEEKDAY)	1
Weighted Average Formula	1
Weighted Median / Binary Search on Answer	1
Welzl's Algorithm (Recursive Smallest Enclosing Circle)	1
WHERE clause filtering directly in SQL engine.	1
Win-Condition Invariance Check	1
Window Function (`DENSE_RANK()`)	1
Window Function (`FIRST_VALUE`) / `RANK()`	1
Window Function (`RANK()` / `ROW_NUMBER()`)	1
Window Function (`RANK()`) + Pivoting	1
Window Function (AVG over Partition)	1
Window Function (COUNT OVER PARTITION)	1
Window Function (DENSE_RANK)	1
Window Function (LAG/LEAD)	1
Window Function (LEAD) + CTE	1
Window Function (LEAD/LAG) or PERIOD_DIFF.	1
Window Function (MAX OVER)	1
Window Function (MAX) / GROUP BY	1
Window Function (MIN) or Grouped Filtering	1
Window Function (PERCENT_RANK)	1
Window Function (RANK)	3
Window Function (RANK/DENSE_RANK) or Join with Max-Group.	1
Window Function (ROW_NUMBER + COUNT)	1
Window Function (ROW_NUMBER)	3
Window Function (ROW_NUMBER) + CTE	1
Window Function (ROW_NUMBER) + Pivot (CASE WHEN).	1
Window Function (ROWS BETWEEN).	1
Window Function / LAG (SQL)	1
Window Function `ROW_NUMBER() OVER(PARTITION BY player_id ORDER BY event_date)`.	1
Window Function: `SUM(games_played) OVER(PARTITION BY player_id ORDER BY event_date)`.	1
Window Function: DENSE_RANK() .	1
Window Function: DENSE_RANK() OVER(ORDER BY Score DESC) .	1
Window Function: RANK() or ALL operator.	1

Window Functions (`RANK()` or `MAX() OVER`)	1
Window Functions (`RANK()` or `ROW_NUMBER()`).	1
Window Functions (`ROW_NUMBER`).	1
Window Functions (AVG ... OVER ROWS BETWEEN 2 PRECEDING)	1
Window Functions (AVG OVER PARTITION).	1
Window Functions (COUNT with Range)	1
Window Functions (Dense_Rank / Max over Partition)	1
Window Functions (DENSE_RANK)	1
Window Functions (Gaps and Islands).	1
Window Functions (id - ROW_NUMBER() grouping).	1
Window Functions (LAG / Difference)	1
Window Functions (LAG / LEAD)	1
Window Functions (LEAD / DENSE_RANK)	1
Window Functions (Max Over Partition)	1
Window Functions (RANK / CTE)	1
Window Functions (RANK / DENSE_RANK)	1
Window Functions (RANK / MAX OVER)	1
Window Functions (RANK() / PERCENT_RANK())	1
Window Functions (RANK, COUNT, CEIL)	1
Window Functions (ROW_NUMBER) to group consecutive sequences + CTEs.	1
Window Functions (ROW_NUMBER/RANK)	1
Window Functions (Sliding Window)	1
Window Functions (SUM OVER)	1
Window Functions (SUM OVER) + CASE	1
Window Functions + CTE Aggregation	1
Window Functions + Floor Math	1
Window Functions / CTE (Common Table Expressions)	1
Window Functions / CTEs	1
Window Functions / Order By + Limit 1	1
Window Functions or Common Table Expressions (CTEs) with MAX() and AVG().	1
Window Functions or Subqueries with Group By.	1
Window Functions with IGNORE NULLS	1
Window Functions: LEAD() or LAG() .	1
Word-by-Word Parsing	1
Wormhole Teleportation (O(N) Pre-matching)	1
X-Coordinate Sorting	1
XOR Inverse Property	1
XOR Math or Frequency Array	1
XOR Parity Property (Even/Odd Counts)	1
XOR Sum Property	1
Z-Algorithm	1
Z-Algorithm (Prefix & Suffix Matching)	1
Z-Algorithm / KMP Prefix Function	1
Zeller’s Congruence / Sum of Days	1
Zero-Skipping Sparse Traversal / Compressed Sparse Row (CSR)	1